

—— // REWRITING THE DESKTOP // ——

The GUI Automation BUS

First Edition

THE BUS

the surface · the bridge · the socket

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTAPP

Contents

Before You Begin	4
Chapter 1 — The Dead End	6
Chapter 2 — Five Layers, One Contract	8
Chapter 3 — The Surface	11
Chapter 4 — The Bridge	15
Chapter 5 — The App Module	18
Chapter 6 — The Transport	20
Chapter 7 — The Hybrid Handler	24
Chapter 8 — The Surface, App by App	26
Chapter 9 — Three Front-Ends, One Module	30
Chapter 10 — Security and the Allow-List	32
Chapter 11 — Novelty — An Honest Prior-Art Analysis	34
Chapter 12 — Rollout and Open Questions	36
Chapter 13 — The Value Model at the Edge	38
Chapter 14 — Failure Is a Reply	41
Chapter 15 — Order, Correlation, and the Pump	43
Chapter 16 — Orchestrations	45
Chapter 17 — The Surface as Its Own Documentation	50
Chapter 18 — The Long Arc	52
Chapter 19 — Writing a New App Module	54
Chapter 20 — The Subscription and Event Model	59
Chapter 21 — Testing the Bus	62
Chapter 22 — Versioning the Contract	64
Chapter 23 — Debugging and Tracing Requests	66
Chapter 24 — Performance and Backpressure	68

Chapter 25 — A Per-App Orchestration Cookbook	71
Chapter 26 — The Verb Reference	75
Chapter 27 — State Queries in Depth	82
Chapter 28 — The Frame Reference	85
Chapter 29 — The Error Reference	88
Chapter 30 — Cookbook II: Reporting, Maintenance, Migration	90
Chapter 31 — Implementing the Handler	94
Chapter 32 — Cookbook III: Interactive and Composed Sessions	97
Chapter 33 — The Bus and the Connector Family	99
Chapter 34 — Patterns and Anti-Patterns	101
Chapter 35 — Cookbook IV: Domain Recipes	103
Chapter 36 — A Design FAQ	107
Chapter 37 — Glossary	110
Chapter 38 — Tooling: The REPL and Completion in Practice	112
Chapter 39 — The Step Chain, Reconsidered	115
Chapter 40 — Concurrency: Many Scripts, Many Apps	118
Chapter 41 — Restarts, Partial Runs, and Resumability	121
Chapter 42 — Naming a Surface	124
Chapter 43 — What the Bus Replaces	127
Chapter 44 — Long Work: Jobs, Progress, and the Cancellation Gap	130
Chapter 45 — The Filesystem as the Second Wire	135
Chapter 46 — The Catalog: Generation, Diffing, and the Gate	139
Chapter 47 — The Audit Trail	143
Chapter 48 — Driving an App a Human Is Using	147
Chapter 49 — A Complete Session, Narrated	150
Chapter 50 — Text: Encoding, Normalization, and the Bytes in a String	153
Chapter 51 — Time: Timestamps, Deadlines, and the Clock the Bus Does Not Have	157
Chapter 52 — One User, More Than One Session	160
Chapter 53 — Secrets, and What Outlives the Run	163
Chapter 54 — What a Verb Should Say When It Says No	166

Chapter 55 — Retiring a Verb	169
Chapter 56 — The Bus in the Shell	172

Before You Begin

This is the book about a single sentence: **one language should be able to drive every app you own, by name, and get an answer back.**

Not by moving the mouse. Not by matching pixels. Not by shelling out to a different scripting runtime for each platform. By calling a named verb — a verb the app itself declares — passing typed arguments, and receiving the return value straight into the calling program's variables. `library.search` with a query returns rows. `request.send` returns a response. `document.open` returns a handle. The app is not a screen to be poked; it is a set of functions to be called.

The suite this book describes was built the way described in its companion, *Rewriting the Desktop in Rust*: every app is a thin Tauri v2 shell over a pure-Rust `*-core` engine, all sharing one component library, `zgui-core`. That architecture left a gift on the table. If each engine already exposes a clean command surface to its own shell, then that same surface — enumerated, typed, and reachable over a small local protocol — is an automation API for free. The GUI Automation Bus is the collection of the small pieces that turn “the shell can call the engine” into “any stryke script, anywhere on the machine, can call the engine, in-process or over a socket, and orchestrate a dozen apps in one run.”

The language is stryke — the author's own from-scratch, Cranelift-JIT scripting language. The bus is not a new language and not a network service. It is an `App` module for stryke, a typed surface declared once per app, a request/response bridge, and a per-app Unix socket carrying newline-delimited JSON. Five small layers, each with a reference implementation already present somewhere in the stack, assembled into a capability the individual pieces did not have alone.

How to read this book

The first six chapters build the bus from the bottom up: the problem it closes (Chapter 1), the five-layer model (Chapter 2), then each layer in turn — the surface (3), the bridge (4), the `App` module (5), and the transport (6). Chapter 7 reads the real hybrid handler that every app ships. Chapter 8 is the census: the live surface, app by app — more than three thousand verbs across eleven apps. Chapters 9 through 12 cover the three front-ends onto the bus, the security model, an honest accounting of what is and is not novel here, and the rollout.

The remaining chapters turn from the contract to the craft. Chapters 13 through 15 go down to the seams the earlier chapters glide over — how a JSON value becomes a stryke value and back, how a failure becomes a reply, and how correlation, ordering, and event delivery actually work on a full-duplex socket. Chapter 16 is worked orchestrations: whole cross-app scripts, read line by line, that exist only because the bus does. Chapter 17 shows the surface documenting itself, and Chapter 18 traces the arc past version one. The first half earns the design; the second half lives inside it.

The last stretch is the practitioner's half — the material you reach for once the design is settled and you are building on top of it. Chapter 19 is a full tutorial: take an app that has no surface and put it on the bus, one line at a time, from the first `register` call to a passing `.stk` script. Chapter 20 is the subscription and event model in depth — filters, replay, backpressure on the stream, and the lifecycle of a subscription from `sub` frame to `unsub`. Chapter 21 is how you test a bus surface without a running desktop; Chapter 22 is how the contract changes without breaking the scripts already written against it; Chapter 23 is what you do when a call misbehaves — tracing, logging, and reading the wire. Chapter 24 is performance and backpressure under real load. Chapter 25 is a cookbook: a run of worked, per-app orchestrations organized by the problem they solve rather than the mechanism they use. Chapter 26 is the Verb Reference — a namespace-by-namespace walk of every app's surface, the printed companion to the generated catalog. The final four are reference

matter for the desk: Chapter 27 is state queries in depth — when to read state and when to call a verb; Chapter 28 is the frame reference, every wire frame with its fields and a worked transcript; Chapter 29 is the error reference, every failure the bus can hand a script with its cause and its fix; and Chapter 30 is a second cookbook of reporting, maintenance, and migration runs. Chapter 31 goes to the app author's side — the `zgui-bridge` crate, the `Handler`, and what `serve` actually does. Chapter 32 is a third cookbook of interactive and composed sessions, and Chapter 33 places the `App` module in the connector family it belongs to, alongside `stryke-gui` and `stryke-aws`. Chapters 34 and 35 collect the idioms and a fourth cookbook of single-app recipes; Chapter 36 is a design FAQ that answers the questions the earlier chapters raise but do not stop for; Chapter 37 is a glossary of every load-bearing term; and Chapter 38 builds the REPL and the shell completion far enough to use.

The last stretch before the closing session collects the questions that only come up once the bus is in daily use. Chapter 39 revisits the palette's six-step grammar now that one of its steps can drive the whole bus, and gives the rule for when a chain should collapse into a script and when it should not. Chapter 40 is concurrency — many scripts against one app, one script against many apps, and the writer the design cannot control at all, the human at the keyboard. Chapter 41 is failure above the level of a single call: what a broken call's effect actually is, how a handle and its subscriptions come back after a restart, and how to make a long run resumable. Chapter 42 reads the naming conventions off the shipped surface and states them as rules, because the verb id is the one artifact that is expensive to change. Chapter 43 is the practitioner's comparison — for each thing you would otherwise have written, what it would have cost and which of the bus's properties it lacks. Chapter 44 is the shape a surface takes when the work is too long to block on — jobs, progress, and the cancellation the wire deliberately does not offer. Chapter 45 is the second carrier every large orchestration ends up using, the filesystem, and the ownership discipline that makes passing paths safe. Chapter 46 is the generated catalog read as a build artifact: how it is extracted, how two generations of it are diffed, and what a compatibility gate built on that diff should and should not block. Chapter 47 is the audit trail — what a run should write down, and why the bus refuses to write it for you. Chapter 48 is the fact the whole book has been quietly assuming, that the app being driven is an app somebody may be looking at. Then Chapter 49 runs one whole session end to end.

The final stretch takes six questions the earlier chapters raise only glancingly, each of which becomes a bug in a real orchestration before it becomes a topic. Chapter 50 is text — the encoding the wire assumes, the control-character rule the framing silently depends on, and the normalization hazard sitting under every cross-app join. Chapter 51 is time, which JSON has no type for, and the clock a deadline has to be measured against. Chapter 52 reads the security model's file mode precisely: what a uid is and is not, what happens when one user has two sessions, and why a scheduled script sometimes reports that a visibly running app is not running. Chapter 53 follows a secret through every artifact that outlives a run. Chapter 54 is the app author's side of failure — what a verb should say when it says no, and the one field the reply frame does not have. Chapter 55 is removal, the half of the versioning rule that is easy to state and hard to execute, including the three removals that break silently. Chapter 56 places the bus in the terminal it lives in. Read the first eighteen chapters to understand the bus; read the rest to live in it day to day.

Every architectural claim is grounded in the design document `GUI_AUTOMATION_BUS.md` and in the shipped code — each app's `bus.rs`, each engine's command list, the shared `zgui-core` surface registry. Where a number appears — a verb count, an app count — it is derived from the source of truth (`GUI_SCRIPT_ACTIONS.md` is generated from each app's verb list), not asserted from memory. This is a book about a built thing, and it tries to earn every sentence.

Chapter 1 – The Dead End

A palette that cannot talk back

Before the bus, every app already shipped a user-programmable command palette. `ZGui.userCommands`, in `zgui-core/webui/user-commands.js`, stores a shared, cross-app list of command entries, each a chain of typed steps drawn from a small grammar: `url`, `js`, `scheme`, `event`, `stryke`, `bash`. Two of those steps are the whole story of why the bus had to exist.

The `event` step dispatches exactly one app action by id. Apps published their action vocabulary through `setActions([{id, label}])` — id and label, nothing more. No parameters. No return value. No way to read the app's state first, or to react to what the action did. It is a doorbell: you press it, something happens inside, and you are told nothing.

The `stryke` step ran a script on the backend through `run_stryke_hook`. The script received a single context argument and returned. That was the entire contract. The script was blind: it could compute, it could format a string, it could call out to the wider world — but it had **no handle back into the app it was running inside**. It could not call an app action and read the result. It could not query the current selection, the open document, the contents of a list. It could not subscribe to an event. And it certainly could not reach a *different* app.

So the palette offered a cruel choice. Either a blind script that could not touch the app, or a single fire-and-forget action that returned nothing. Never the obvious thing in between: a script that drives the app, waits for the answer, and decides what to do next. That gap — the missing return path from app to script — is the entire problem the bus closes.

What “driving” actually requires

Reduce the requirement to its verbs and it is short. A running script must be able to **enumerate** an app's actions, **call** one with typed arguments and receive the **return value**, **read** a named piece of state, and **subscribe** to an event stream. Four operations. And it must be able to do all four whether the script runs *inside* the app — a palette step, a hook — or *outside* it, as a standalone file in the shell driving a separately running app.

Those two execution contexts look different and are the same. In-process, the script and the app share an address space; the call is a function call. Out-of-process, they are two operating-system processes and the call must cross a socket. The bus's central design decision is that the script cannot tell which it is in. `App::here()->call(...)` and `App::open("zcite")->call(...)` return handles that behave identically. One language, one API, two transports underneath.

Why the existing steps could not be stretched

It is fair to ask why the palette's other step kinds could not simply be pressed into the role, since the grammar already had six. Walk them and each fails the same test for a different reason. The `url` step opens a location; it is a navigation primitive with no return channel and no notion of an app action at all. The `scheme` step fires a custom-scheme URL — `zcite://item/42` — which is one-way by construction: a scheme handler is invoked, not queried, and the sender learns nothing about what the handler did. The `js` step evaluates JavaScript in the front-most webview, which *can* read app state, but only the app the palette is attached to, only

in the frontend, and only by writing app-specific JavaScript that reaches into private internals — the opposite of a stable, typed, cross-app contract. The `bash` step shells out, which is the widest escape hatch of all and precisely the thing the whole suite was built to stop needing: a `bash` step cannot call an app verb because there is no command-line surface for the app's verbs to call. Six steps, and not one of them could read a value out of an app and hand it to the next step. The gap was not a missing feature in any single step; it was a missing *layer* that all the steps could sit on top of.

The workarounds, and what each one cost

The gap did not stop anyone from automating; it made them automate badly, in three recognizable shapes, and each shape is worth naming because each is a symptom of the same missing layer and each is what the bus's arrival actually retires.

The first shape is the **chain of doorbells with a sleep between them**. Wanting a palette command that did three things in order, an author strung three `event` steps together and, because the second depended on the first having finished, inserted a `bash` step that slept. The chain worked on the machine it was written on and failed under load, because the sleep was a guess about a duration nobody measured. It could not be made correct, either, since correctness would have required knowing when the first action finished — a fact that no step could obtain, because no step had a return value to obtain it from. Every such chain is a race that the author negotiated with rather than closed.

The second shape is the **js step reaching into private internals**. When a script genuinely needed to read something out of the app — which item is selected, what document is open — the only step that could do it was `js`, and the only way `js` could do it was by naming the frontend's own variables and functions. That produced automation coupled to the app's implementation rather than its capabilities: it worked, it was fast, and it broke the first time the frontend was refactored, with a failure that pointed at a JavaScript expression rather than at the design decision that made the expression necessary. It also could only ever address the app the palette was attached to, so it solved a single-app problem in a way that could not be extended to two.

The third shape is the **workflow that stayed manual**. Anything spanning two apps — the invoice run of Chapter 16, the reconciliation of a library against a remote store — simply was not automated, because there was no seam at which two apps' values could meet in one program. It was performed by hand with the clipboard and a temp directory, one export at a time, and repeated as often as the work demanded. That is the largest cost of the three and the least visible, because unwritten automation leaves no artifact to point at. It shows up only as time.

Read together, the three shapes make the diagnosis precise. The first was a missing completion signal, the second a missing typed read, the third a missing cross-app seam — and all three are the same absence, the return path from app to script, showing up at three different scales. One layer was missing, and it was missing everywhere at once.

The four operations, made concrete

The four operations read as abstractions — enumerate, call, read, subscribe — so it is worth grounding each in the single line of stryke that will, by Chapter 5, express it. *Enumerate* is `$h->verbs()`, which returns the typed manifest a script reads to discover what an app can do. *Call* is `$h->call("item.add", %{ doi => $d })`, which invokes a verb with typed arguments and — the entire point — binds the return value into a stryke variable. *Read* is `$h->get("selection")`, which pulls a named piece of live state without disturbing it. *Subscribe* is `$h->on("itemAdded", $fn)`, which registers a callback against an event stream. Four verbs of the meta-language, and every automation in this book is built from nothing but their composition. The claim of the chapter is narrow and total at once: before the bus, a script inside an app could do none of the four against the app it lived in, and a script outside could do none of the four against anything. After the bus, a script does all four, from either side, through one handle. That is the whole distance the rest of the book travels.

Chapter 2 – Five Layers, One Contract

The stack, top to bottom

The bus is five layers, and the discipline of the design is that each one already had a working precedent in the stack — nothing here is invented where something could be reused.

Layer 1, the automation surface. Each app declares a typed dictionary of verbs, state queries, and events. This is the formalization of `setActions`: the same idea, upgraded from id-and-label to id, parameters, return type, and a runnable body. It is owned by the app’s frontend and registered once on boot.

Layer 2, the bridge. A request/response mechanism so that a stryke `call` returns a value from the app back into the VM. The reference implementation is zcontainer’s “sync invoke plus streaming subscribe” substrate — the exact shape the bus needs, ported into a shared component rather than rebuilt per app.

Layer 3, the stryke App module. A connector package in the same family as `stryke-gui` and `stryke-aws`: a cdylib with C-ABI exports and a stryke-facing module. It gives the language `App::open`, `App::here`, `App::list`, and the handle methods `verbs`, `call`, `get`, and `on`.

Layer 4, the transport. In-process, a direct host callback — no socket, lowest latency. Out-of-process, a per-app Unix domain socket speaking newline-delimited JSON. The precedent is zterminal, which already talks a raw wire protocol straight to a socket — the tmux imsg protocol, no subprocess — proving the discipline.

Layer 5, the front-ends. The palette `stryke` step, the hooks runtime, and the standalone `.stk` file. All three get `App` in scope and use the same module, so there is no per-surface special-casing. The palette does not become a client of a special palette API; it becomes one client of the same bus every other surface uses.

The ownership rule

One rule governs where each piece lives, and it is inherited directly from the architecture book’s shell-versus-engine boundary: **the automation surface is owned by the core, and the socket host is owned by the top-level app.** A core knows its own verbs — it is the thing that implements them — so the verb list is core-owned. But a running app is one process with one front door, the way it has one command palette; so it opens exactly one socket, and any engine embedded inside it contributes its verbs *into that app’s surface* rather than opening a socket of its own. Namespacing keeps them distinct. The rule falls straight out of the existing architecture; the bus did not have to invent a new notion of who owns what.

Each layer’s precedent, named

The design’s central discipline — reuse a working shape rather than invent one — is worth making explicit layer by layer, because it is what keeps the per-app cost near zero and what makes the whole thing credible as a *built* rather than *proposed* system. Layer 1 is `setActions` grown a type signature: the registration call was already there, already invoked on boot in every app, already read by the palette’s action picker; the bus widens its payload and adds two readers. Layer 2 is zcontainer’s invoke-plus-subscribe substrate, a request/response channel with an attached stream that was written, shipped, and debugged before the bus existed as a word. Layer 3 is the `stryke-gui` / `stryke-aws` connector pattern — a cdylib with C-ABI exports and a stryke-facing module — instantiated a third time with different verbs. Layer 4 is zterminal’s socket discipline: a real app already speaks a raw wire protocol straight to a Unix socket with no subprocess between, so the bus is not

proving that the pattern works, only pointing a second protocol down the same proven pipe. Layer 5 is the palette's existing step runner, which already put `stryke` in scope on three surfaces; the bus adds one binding to that scope and changes nothing else. Five layers, five precedents. The novelty, as Chapter 11 argues at length, is the assembly, not any single course of brick.

What each layer is forbidden to know

A layered design is defined as much by what each layer is denied as by what it provides, and stating the prohibitions is the fastest way to check whether the stack is real or merely drawn. Five prohibitions hold here, and each one is what makes some later chapter's claim possible.

Layer 1, the surface, must not know that a transport exists. A verb's `run` body receives decoded arguments and returns a value; it is never handed a connection, never told whether its caller dialed a socket or came through a host callback, and never given a way to write to the wire directly. This is what lets the identical registration serve the palette's action picker, an in-process script, and a remote dialer without modification, and it is why an app author writing a verb writes only domain code.

Layer 2, the bridge, must not know what a verb means. It correlates, validates against the declared parameters, awaits, and composes a reply; it never inspects a verb's semantics to decide anything. Chapter 31 states the same rule from the crate's side — the host is a pipe with a correlation table — and the consequence is that one implementation serves eleven apps of unrelated domains without a line of domain code in it.

Layer 3, the module, must not know which transport it is on. `App::call` composes a frame and waits for a correlated reply; whether the frame went down a socket or through a callback is resolved beneath it. This is the prohibition that makes the central promise of Chapter 1 enforceable rather than aspirational: a script cannot tell which context it is in, because the layer that would have to tell it has no access to the fact.

Layer 4, the transport, must not interpret a frame beyond its `t`. It moves bytes and frames them at newlines; it does not know a verb from a state query except as two values of a discriminant field. This is what allows the frame set of Chapter 28 to be closed and grown by adding fields — a transport that parsed frame bodies would have to be taught every extension, and one that only routes on `t` does not.

Layer 5, a front-end, must not know it is one of three. The palette step, the hook, and the `.stk` file each receive `App` in scope and contain no branch on which surface they are. This is the prohibition that makes the portability of Chapter 39 possible: a script developed at the prompt can be pasted into a palette step because there was never a line in it that depended on where it ran.

The one deliberate leak

Against those five, the design permits exactly one place where a layer knows something a strict reading would deny it, and naming it honestly is the point of the section. The hybrid handler of Chapter 7 knows the difference between an engine verb and a frontend verb, and routes on it. That is knowledge about the app's internal structure sitting in the dispatch path, and a purist stack would push it down or hide it entirely.

The design keeps it, deliberately, and confines it to the one component the *app* owns rather than the one the shared crate owns. `zgui-bridge` remains ignorant: it calls `is_engine_cmd` and dispatches to whichever arm the answer selects, without knowing what either arm means. The app's `bus.rs` holds the knowledge because the app is the only party that has it — only the app knows which of its own verbs its engine implements. The leak is therefore not a layering violation but a correctly placed responsibility: the shared layer stays generic by delegating the one question it cannot answer to the one component that can. A stack with no such delegation point would have to either give the shared crate per-app knowledge or force every call through the slowest path, and both are worse than one narrow, named interface.

The contract, stated once

Underneath the five layers is one contract, and it can be written in a sentence that every layer either implements or consumes: *a caller names a verb and supplies typed arguments; the app runs the verb and returns exactly one typed value, or one typed failure, correlated to the call.* Layer 1 declares the verbs and their types. Layer 2 carries the call and its reply. Layer 3 gives the caller a handle to name the app and the verb. Layer 4 moves the bytes, in-process or across a socket, without changing the contract's shape. Layer 5 is three different callers all speaking that one sentence. Reading the stack this way explains why the transport can be swapped under the caller's feet without the caller noticing: the transport is the one layer the contract deliberately says nothing about. A verb call means the same thing whether a host callback or a socket carried it, because the contract is defined in terms of names, types, and correlation — never in terms of wires.

Chapter 3 – The Surface

From label to manifest

Layer 1 replaces `setActions([ {id, label} ])` with a typed manifest registered once per app, through a `zgui-core` module, `automation.js`, exposed as `window.ZGui.automation`. The registration is declarative:

```
ZGui.automation.register({
  app: "zcite",
  verbs: [
    { id: "library.search",
      label: "Search library",
      params: [{ name: "q", type: "string", required: true },
                { name: "collection", type: "string", required: false }],
      returns: "list<item>",
      run: (a) => zcite.searchLibrary(a.q, a.collection) },
    { id: "item.add",
      label: "Add item",
      params: [{ name: "doi", type: "string", required: true }],
      returns: "item",
      run: (a) => zcite.addByDoi(a.doi) },
  ],
  state: [
    { id: "selection", returns: "list<item>", get: () => zcite.currentSelection() },
    { id: "activeCollection", returns: "string", get: () => zcite.activeCollection() },
  ],
  events: [
    { id: "itemAdded", payload: "item" },
  ],
});
```

Three kinds of entry. A **verb** is callable and typed and returns a value; its `run` may be asynchronous, and the bridge awaits it before replying. A **state** query is read-only; its `get` returns a value on demand. An **event** is a name the app emits, via `ZGui.automation.emit(id, payload)`, and that subscribers receive.

Introspection is the same surface

`ZGui.automation.surface()` returns the manifest with the function bodies stripped — types only, no `run` or `get` closures. That stripped manifest is exactly what a script sees when it asks a handle for its `verbs()`. The consequence is that the palette's own action-picker, which used to read the flat id-and-label list, now reads the typed surface for free: the same registration that makes an app scriptable from the outside makes its palette editor richer on the inside. One declaration, two readers.

What the types are, and what they are not

The `type` and `returns` strings in the manifest — `string`, `list<item>`, `item` — look like a type system, and it is worth being precise about how much of one they are. They are a **contract the bridge checks at the boundary and a document the surface publishes**, not a compiler that has proven anything about the `run` body. When a call arrives, the bridge validates the supplied arguments against the declared `params`: a missing `required` argument is rejected before `run` is ever invoked, and a value of the wrong JSON kind is rejected with the verb's signature in the error. That much is enforced. But `returns: "list<item>"` is a promise the app makes about its own output, not a shape the bridge coerces the result into — if a verb's `run` returns something else, the bridge forwards it faithfully and the mismatch is the app's bug to fix, surfaced the next time a script reads a field that is not there.

The choice is deliberate and it mirrors the whole suite's posture toward its own frontend: the core is trusted because the same author wrote it, so the manifest's job is to describe the surface accurately and reject *callers* who misuse it, not to defend the app against itself. The `item` in `list<item>` is documentation with teeth only at its edges — it tells a script author what fields to expect and it lets `verbs()` render a readable signature, while the interior of the value is governed by the app's own discipline. A richer scheme — nominal record types shared between the manifest and the engine — is a thing the design leaves for a later edition, named in Chapter 18, precisely because the cheap version already carries its weight.

The parameter grammar in full

The `params` array is where the surface earns its keep, so it repays a closer read than the opening example gave it. Each entry is an object with a `name`, a `type`, and a `required` flag, and the bridge treats the three as a checklist it runs before `run` is ever entered. A missing `required` parameter is a rejection, not a call with a nil hole in it. An `optional` parameter absent from the arguments is simply not passed, and the `run` body sees it as undefined — the app decides its own default. The `type` string is drawn from the same small vocabulary the `returns` field uses: the scalars `string`, `int`, `float`, `bool`; the container forms `list<T>` and `map<T>`; and the app's own nominal shapes such as `item`, `message`, `transfer`, which the surface treats as opaque object types with named fields the app documents. A verb that takes a search query and an optional collection reads:

```
{ id: "library.search",
  label: "Search library",
  params: [{ name: "q", type: "string", required: true },
            { name: "collection", type: "string", required: false },
            { name: "limit", type: "int", required: false }],
  returns: "list<item>",
  run: async (a) => zcite.searchlibrary(a.q, a.collection, a.limit ?? 50) }
```

Two properties of that entry are load-bearing. First, `run` is `async` and returns a Promise, and the bridge awaits it before composing the reply — so a verb backed by a database query or a network fetch reports its real result, not a placeholder, and the calling script blocks exactly as long as the work takes. Second, the default `a.limit ?? 50` lives in the `run` body, not in the manifest: the manifest declares that `limit` is an optional int, and the app decides what absence means. The surface describes the shape of the conversation; the engine owns the semantics.

The validator, in order

The bridge's boundary check is a short, ordered sequence, and knowing its order explains every error message a script author will ever see from it — the errors of Chapter 29 are just this sequence's rejection points, named.

First the **name resolves, or nothing else happens**. The incoming verb id is looked up against the engine's `COMMANDS` and the frontend's registered set, and a name in neither produces `no such verb: <name>` immediately. No argument is examined, because there is no signature to examine against — the parameter

list belongs to a verb, and there is no verb.

Second, **each declared parameter is checked in turn**. The validator walks the verb's `params` array, not the caller's argument map, which is the correct direction and worth noticing: the declaration drives the check, so a verb's contract is enforced exactly as written whether the caller supplied two arguments or twenty. For each declared parameter, if it is `required` and absent from the arguments, the call is rejected with the missing parameter named and the verb's full signature printed. If present, its JSON kind is checked against the declared `type` — a string where an `int` was declared is a rejection, again with the signature. If it is optional and absent, it is simply not passed, and the `run` body sees it as undefined and applies its own default, per the `a.limit ?? 50` pattern above.

Third, **`run` is entered with the validated arguments and nothing else happens at the boundary**. The return value is forwarded faithfully, unchecked against `returns`, for the reason the previous section gave: the manifest's return type is a promise the app makes about itself, and the bridge is not in the business of second-guessing the engine it exists to call.

Extra arguments, and why they are tolerated

One case in that sequence deserves its own explanation, because it looks at first like a missing check. An argument the caller supplied that the verb never declared is *not* a rejection. The validator walks the declared parameters and ignores anything else in the map.

The tolerance is deliberate and it is the same instinct that governs the wire in Chapter 22: unknown fields are ignored so that the contract can grow additively. A script written against a newer build of an app, sending an argument that build introduced, does not catastrophically fail against an older build that has not learned the parameter — it gets the older behavior, which is the behavior it would have gotten before the parameter existed. Rejecting unknown arguments would make every new optional parameter a breaking change in the reverse direction, turning the cheapest kind of surface growth into a compatibility event.

The asymmetry with missing required arguments is the point. A missing required argument means the app cannot do the work at all, so the only useful response is a loud rejection at the boundary. An unrecognized extra argument means the caller knows about something this build does not, which is a mismatch the caller can survive and the app can ignore. Reject what makes the work impossible; tolerate what merely makes it older.

How deep the type check goes

The container forms deserve one precise sentence, because a reader may expect more of them than they promise. A parameter declared `list<item>` is checked to be a JSON array. Its elements are not walked, and their fields are not verified against whatever `item` means. The same holds for `map<T>`: the value must be a JSON object, and its contents are the app's business.

That is the same stance the chapter's opening took, applied consistently rather than abandoned at the first container. The bridge validates the *shape of the conversation* — that an array arrived where an array was declared — and leaves the interior to the app's own discipline, because element-wise validation would mean the bridge holding a real type system for nominal shapes it does not own, checked on every element of every call, to defend an app against its own author. The check is at the boundary because the boundary is where an untrusted caller sits, and Chapter 10's trust model has already established that the caller is the user's own script. The validator's job is to catch the honest mistake — a forgotten argument, a string where a number belongs — at the moment it is made, with the signature in hand. It is not a proof, it was never claimed to be one, and the cheap version carries its weight precisely because it stops where its authority stops.

Emitting an event

The third entry kind, the event, is the only one whose direction is app-to-script, and its declaration is the smallest of the three — a name and a payload type — but its runtime side has a second half the registration does not show. Declaring `{ id: "itemAdded", payload: "item" }` only names the channel; the app must actually push down it, which it does with one call at the point the thing happens:

```
async function addByDoi(doi) {
  const item = await zcite.resolveAndStore(doi);
  ZGui.automation.emit("itemAdded", item); // every subscriber receives this
  return item;
}
```

The `emit` is fire-and-forget from the app's side — it returns immediately, having handed the payload to the bridge, which fans it out to every subscription registered on that event. An app with no subscribers still calls `emit`; the bridge simply has no one to deliver to, and the cost is a membership check on an empty list. This is why an app can declare and emit events unconditionally without paying for machinery no script is using: the expensive half of a subscription — the delivery — exists only for events someone is actually watching, and Chapter 20 traces exactly how the bridge decides who that is.

Chapter 4 – The Bridge

Making a fire-and-forget call return

Layer 2 is the piece that turns `run_stryke_hook` from a dead end into a conversation. The bridge lets an in-flight stryke script call back into the app and — the whole point — get the value back into the VM. Trace a single in-process `call`:

```
stryke VM (App::call)
-> host callback app_call(verb, args_json)    [stryke FFI -> app backend]
-> backend routes to the webview: dispatch to the registered verb
-> verb.run(args) resolves (await if it returned a Promise)
-> JSON result travels back over the response channel
-> host callback returns the JSON to the stryke VM
-> App::call returns the decoded value
```

The one genuinely new backend primitive is a **synchronous-looking request/ response** between the stryke host and the webview surface: the script blocks on a call, the webview runs the verb, and the value comes back correlated to that call. This is not invented from nothing. zcontainer already implements exactly this shape — a synchronous invoke plus a streaming subscribe — and the bus's design is to lift that substrate into a shared component, not to reimplement it in each app.

One substrate, two channels

`get` travels the same request/response path, aimed at the state table instead of the verb table. Subscription is the streaming half of the same substrate: `on(event, fn)` registers on the subscribe channel, and the app's `emit` pushes frames to every subscriber. So the bridge is not three mechanisms but one — a correlated request/response with an attached event stream — used three ways. That economy is why per-app cost stays near zero: an app that wires the one substrate gets calls, state reads, and subscriptions together.

The correlation channel, concretely

The word “correlated” carries the weight in that trace, and it is worth making mechanical. When the host callback delivers a call into the backend, it assigns the request a numeric `id` and stashes a one-shot reply slot in a table keyed by that `id` — in the shipped code, a channel the backend can later fulfill exactly once. The backend then evaluates the dispatch shim in the webview, `window.__zguiBridgeDispatch(id, kind, name, args)`, passing the same `id` along. The shim looks up the registered verb by `name`, runs it, awaits the result, and calls back into the backend through a small Tauri command, `zgui_bridge_reply(id, ok, value)`. The backend takes the `id`, finds the waiting slot, and fulfills it with the value — which unblocks the host callback, which returns the JSON to the stryke VM. The `id` is the thread that runs through the whole loop: it is minted at the call, it rides into the webview and back, and it is what lets the reply find the exactly-one caller waiting for it even when several calls are in flight. Nothing about that loop is specific to `call`; `get` runs the identical machinery with `kind` set to state instead of verb, which is the concrete meaning of “one substrate, two channels.”

Why the webview is in the loop at all

A reasonable objection is that routing a call through the webview adds a hop, and the design answers it by *not* routing every call through the webview — which is exactly what the hybrid handler of Chapter 7 is for. Engine verbs, the ones listed in the core's `COMMANDS`, never enter the webview at all: they run in Rust, in the same `*-core` engine the UI calls, and the reply is composed without a single frame crossing into JavaScript. The webview hop exists only for the surface that genuinely lives in the frontend — the `appshell.*` verbs, an embedded core's verbs, anything a plugin registered through `ZGui.automation` at runtime. For those, the frontend is not an overhead in the path; it *is* the path, because the frontend is where the verb was defined. The bridge's two-target shape is therefore not a compromise between speed and reach but a recognition that an app is two things at once — a Rust engine and a web frontend — and a call has to be able to land on whichever one actually implements the verb.

The reply slot, and what happens when nobody fulfils it

The correlation table described above is a table of *slots*, and a slot is a resource with a lifetime, so it repays asking what the lifetime is and who ends it. A slot is created when the host callback mints an id, and it is consumed exactly once — by `zgui_bridge_reply` arriving with that id and fulfilling the waiting channel. One slot, one fulfilment, then the entry is gone from the table.

The interesting question is the case where the fulfilment never comes. A frontend verb whose `run` throws before the shim can report, a shim that never runs because the webview was not ready, a `run` that awaits a promise nobody will settle — each leaves a slot occupied by a caller waiting on a channel that will never be written. Without a second mechanism, that is a permanently blocked script and a permanently leaked table entry, and the two failures compound: a script that hangs is the worst outcome Chapter 14 names, and a table that grows an entry per hung call is a slow leak in a long-lived app.

The bounded wait of Chapter 14 is what closes both at once, and this is where the deadline earns more than its stated purpose. When the deadline fires, the module's side stops waiting and manufactures a death naming the verb — which is the part a script author sees. The backend's side abandons the slot, which is the part nobody sees and which matters just as much, because it means the correlation table's size is bounded by the number of calls in flight rather than by the number of calls ever made. A late reply arriving after the deadline finds no slot for its id and is dropped rather than fulfilling something that has moved on, which is the correct handling of a message whose recipient has given up: the alternative — keeping the slot indefinitely in case a reply eventually appears — trades an unbounded table for a value nobody is waiting for any more.

Boot ordering, and the window before the shim exists

There is one honest race in the webview path and it lives at startup. The dispatch shim, `window.__zguiBridgeDispatch`, is installed by the frontend as it initialises, and the socket the host serves is bound in `main`. Those are two different components coming up on two different schedules, and there is a window — short, but real — in which the socket is dialable and the shim is not yet installed.

A frontend call arriving in that window has no shim to evaluate, and the honest outcome is a failed call rather than a queued one: the caller learns that this app is not ready for frontend verbs yet, rather than blocking on a webview that may take an arbitrary amount of time to finish loading. An engine call arriving in the same window is unaffected, because it never touches the webview — a second and rather elegant consequence of the hybrid handler's split, and one worth knowing when a script's very first call against a just-launched app behaves differently from every call after it.

The practical guidance is the one the discovery idiom of Chapter 34 already gives for a different reason. A script that has just launched an app, or that runs early in a session, should establish that the app is answering before it depends on it, and the cheapest way to establish that is the call it was going to make anyway wrapped in the tolerance the run can afford. There is no readiness verb and there deliberately is not one: a readiness

flag that said “the frontend is up” would be a second thing to keep true, and the surface already has a way to ask — call something and see.

Chapter 5 – The App Module

A connector, not a new dialect

Layer 3 is a stryke connector package, `stryke-app`, a sibling of `stryke-gui` and `stryke-aws` and built to the same cdylib-plus-FFI shape: C-ABI functions taking and returning C strings, with a stryke-facing module on top. It is semantic where `stryke-gui` is physical — one calls `library.search` and gets rows; the other moves the mouse — and the two are complementary rather than competing.

The grammar is ordinary stryke, the same seen in `stryke-gui` examples: `use App`, `val` bindings, `Module::fn(...)` calls, `$handle->method(...)`, `p` to print, and `"${x}"` interpolation.

```
#!/usr/bin/env stryke
use App

# in-process: the script runs inside the app (palette step / hook)
val $me = App::here()
val @hits = @{ $me->call("library.search", %{ q => "graphene" }) }
p "found ${\ scalar @hits} items"

# out-of-process: drive running apps by name, from the shell
val $cite = App::open("zcite")
val $req = App::open("zreq")

# cross-app: every DOI in the zcite selection fires a request in zreq
for val $it (@{ $cite->get("selection") }) {
  $req->call("request.send", %{ url => "https://api.crossref.org/works/${ $it->{doi} }"
    ~ })
}

# subscribe: when zcite adds an item, log it
$cite->on("itemAdded", fn ($item) { p "added: ${ $item->{title} }" })
```

The whole surface, in seven calls

Call	Returns	Notes
<code>App::here()</code>	handle	the app the script runs inside (in-process only)
<code>App::open(\$name)</code>	handle	dial a running app's socket; dies if absent
<code>App::list()</code>	list	bus names of running apps
<code>\$h->verbs()</code>	manifest	typed verbs / state / events, for introspection
<code>\$h->call(\$verb, %args)</code>	value	invoke a verb, await the result, decode it
<code>\$h->get(\$state)</code>	value	read a state query

Call	Returns	Notes
<code>\$h->on(\$event, \$fn)</code>	subscription	run a callback per emitted event

Seven calls is the entire vocabulary a script needs to enumerate, drive, read, and watch any app in the suite. Everything else is the app's own verbs.

The handle is the unit of connection

Each call in that table returns or consumes a *handle*, and understanding what a handle is explains why the vocabulary is exactly seven and not more. A handle is a live binding to one app: in-process it wraps the host callback into the local surface; out-of-process it wraps an open socket to one app's `.sock` file. `App::here` and `App::open` are the two ways to mint one — the first for the app the script runs inside, the second for a named app across the socket — and `App::list` is the way to discover which names `App::open` would succeed against right now. Everything else is a *method on a handle*, because everything else is a thing you do *to a particular app*: enumerate its verbs, call one, read its state, watch its events. The vocabulary is small because the model is small — a script holds one handle per app it drives, and it drives each app through that app's handle and no other. There is no global `call(app, verb)` free function, and the absence is deliberate: routing every call through a handle keeps the target explicit at every call site, so a script that drives four apps reads as four clearly separated conversations rather than one undifferentiated stream of `calls` distinguished only by their first argument.

A fuller worked handle

The opening example showed the four operations one line each; a real handle is used across many lines, and the shape of that use is worth seeing whole. Here a single `zreq` handle enumerates itself, reads state, sends a request, and subscribes, all against the one open connection:

```
#!/usr/bin/env stryke
use App

val $req = App::open("zreq")

# 1. enumerate - discover the request verbs this build actually exposes
for val $v (@{ $req->verbs()->{verbs} }) {
  if $v->{id} =~ /^request\.\/ { p "  ${ $v->{id} }" }
}

# 2. read - what environment is active right now
val $env = $req->get("activeEnvironment")
p "active environment: ${ $env->{name} }"

# 3. call - send one request and bind its response
val $res = $req->call("request.send", %{ url => "https://example.com/health" })
p "status ${ $res->{status} } in ${ $res->{ms} }ms"

# 4. subscribe - log every future send this app performs
$req->on("requestSent", fn ($r) { p "sent: ${ $r->{method} } ${ $r->{url} }" })
```

One handle, four operations, one app. Every larger orchestration in Chapter 16 is this pattern multiplied across handles: one handle per app, each handle driven through the same four methods, the script the only place any two apps' values ever meet.

Chapter 6 – The Transport

Two modes, one API

The script never learns which transport it is using, but there are two.

In-process is for a script running inside the app — a palette `stryke` step or a hook. `App::here()` binds directly to the local `ZGui.automation` surface through the host callback. There is no socket and no serialization round trip beyond the JSON the bridge already speaks; it is the lowest-latency path.

Out-of-process is for a standalone file in the shell driving a separately running app. Each app process, on boot, opens a Unix domain socket:

```
$XDG_RUNTIME_DIR/zgui/<app>.sock      # Linux
$TMPDIR/zgui/<app>.sock                # macOS (XDG_RUNTIME_DIR usually unset)
```

`App::open("zcite")` dials `zgui/zcite.sock`. The host is a shared Rust helper — the `zgui-bridge` crate — so the per-app cost is a single line in `main`: `serve(app_name, surface)`. The precedent is again concrete: `zterminal` already speaks a raw wire protocol straight to a socket, the `tmux` `imsg` protocol, with no subprocess in between. The bus applies the same discipline.

A newline of JSON

The wire is deliberately small: newline-delimited JSON frames, request/response with an attached subscription stream.

```
-> {"t":"call","id":1,"verb":"library.search","args":{"q":"graphene"}}
<- {"t":"reply","id":1,"ok":true,"value":[{"...}, {...} ]}

-> {"t":"get","id":2,"state":"selection"}
<- {"t":"reply","id":2,"ok":true,"value":[{"...} ]}

-> {"t":"verbs","id":3}
<- {"t":"reply","id":3,"ok":true,"value":{"verbs":[{"...}], "state":[{"...}], "events":[{"...} ]}}

-> {"t":"sub","id":4,"event":"itemAdded"}
<- {"t":"reply","id":4,"ok":true}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"...}}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"...}}

<- {"t":"reply","id":N,"ok":false,"error":"no such verb: foo.bar"}
```

The `id` field correlates each reply to its request; the `sub` field correlates each pushed event to its subscription. Crucially, the in-process transport speaks the *same frames* over the host callback rather than a socket, which means the `App` module has exactly one codec to maintain. The two transports differ only in what carries the bytes.

The life of a socket

A socket file is a small piece of shared state on the filesystem, and shared state on the filesystem has a life cycle that has to be managed or it rots. The `zgui-bridge` host manages it in the obvious, boring way, which is the correct way. On boot, `serve(app_name, surface)` computes the path — `zgui/<app>.sock` under the runtime directory — creates the `zgui/` parent if it is missing, unlinks any stale file already sitting at that path, binds a fresh `UnixListener`, and sets the mode to `0600` so that only the owning user can dial it. The unlink-before-bind step is what makes a crashed app recoverable: a process that dies without cleaning up leaves a dead socket file behind, and the next launch simply removes it and takes the name.

This produces one honest race, and the design names rather than hides it. Two copies of the same app launched at once will both try to own `zcite.sock`; the second to bind wins the file, and the first is left serving a socket no dialer will find because the name now points elsewhere. The stance is that this is correct: the suite's apps are single-instance by intent — one running `zcite`, one command palette, one front door — so “the last one to bind owns the name” is not a bug to paper over but the single-instance rule expressing itself at the socket layer. A future launch-on-demand, if it comes, is where this would be tightened; version one lets the last writer win.

On clean shutdown the host unlinks its own socket, so a script's `App::list()` — which enumerates the live `*.sock` files — reflects reality within the window between a crash and the next launch, and exactly reflects it otherwise. `App::open` against a name whose file is gone fails at `connect`, not at some later call, so a script learns immediately that its target is down rather than sending a frame into a socket that no one is reading.

Framing: why newlines, and what they cost

The wire is newline-delimited JSON, and the choice of a newline as the frame boundary is the kind of small decision that pays for itself every day it is not noticed. A newline delimiter means the reader is a `BufReader::read_line` loop and the writer appends `\n` after each serialized value — no length prefix to compute and keep in sync with the body, no chunked framing to reassemble, no binary header to version. The cost is exactly one constraint: a frame's JSON must contain no literal newline, which is free because JSON string escaping already turns an embedded newline into `\n` inside the encoded string, so a valid JSON value serialized without pretty-printing is always a single physical line. The bus leans on that guarantee rather than fighting it. It also means a human can watch the wire with `socat - UNIX-CONNECT: $TMPDIR/zgui/zcite.sock` and read the traffic directly, one frame per line, which is the debugging affordance Chapter 23 builds on. A binary protocol would be marginally smaller and considerably less legible; for a single-user local bus, legible wins.

One listener, many connections

A single `.sock` file is one *listener*, but a listener accepts many *connections*, and the distinction matters for how several scripts share one app. When two `.stk` files both `App::open("zcite")` at once, each `connect` produces its own accepted stream inside the app; the app serves them concurrently, each with its own `id` space, its own set of subscriptions, and its own reply slots. There is no shared per-dialer state to collide because the state that matters — the correlation table, the subscription list — is per-connection. Two scripts subscribing to `itemAdded` each get their own delivery; one script's `unsub` or disconnect never disturbs the other's stream. The single-instance rule of the previous section governs which *process* owns the name; within that one process, the number of concurrent dialers is bounded only by the app's willingness to serve them, and the shared `zgui-bridge` host serves each on its own task. So `App::list()` reporting one `zcite` does not mean one script may drive it; it means one process answers, to as many scripts as ask.

One socket per app, not one broker

The obvious alternative to a socket per app is a single socket for the whole suite — one `zgui.sock`, one broker process, every app registering with it, every script dialing it and naming its target in the frame. That shape is common enough in desktop stacks to be the default assumption a reader brings, so the reasons the bus does not take it are worth setting out.

A broker introduces a process that must exist. Something has to start it, keep it running, restart it when it dies, and decide what happens to apps that boot before it or outlive it. It is a supervision problem and a lifecycle problem where currently there is neither: an app's socket comes up when the app does and goes away when the app does, because it is part of the app. There is no state to reconcile between “what the broker thinks is running” and “what is running,” because nothing holds an opinion separate from the filesystem.

A broker also introduces a registration protocol, which is a second contract to design, version, and debug — apps announcing themselves, withdrawing themselves, and being timed out when they crash without withdrawing. The per-app socket needs none of it. As the previous sections established, the presence of a bound, connectable socket file *is* the registration and its absence *is* the deregistration, so `App::list()` is a directory scan, and the registry cannot disagree with reality because there is no registry.

A broker adds a hop. Every call would cross from the script to the broker and from the broker to the app, with the reply retracing both, doubling the round trips on the path Chapter 24 accounts for. And it adds a failure mode with no analogue in the current design: a broker that dies takes the whole bus down at once, whereas an app that dies takes only itself off the bus, leaving every other app dialable and every other script unaffected.

Finally, a broker would have to be *owned*. Which crate ships it, which product's release it belongs to, and whether an app is allowed to be automated when it is absent are all questions with no clean answer in a suite of peer applications. The ownership rule of Chapter 2 — one app, one front door — has no such question, because the front door is part of the app that owns it.

What the per-app choice costs

The honest accounting has a debit side, and it is small but real. A script driving four apps opens four connections rather than one, which is four `connect` calls and four file descriptors — trivially cheap, but four rather than one. `App::list()` is a directory scan, which means it reports what has a socket file rather than what has affirmatively said “I am ready,” and the gap between a crashed app and its next launch is a window in which a stale file can briefly misreport, as the socket lifecycle section already noted. A broker could close that window by holding a liveness view; the per-app design closes it by having `App::open` fail at `connect` rather than later, so the misreport costs one failed dial and never a hung call.

There is also no place to put a cross-app facility that would genuinely want one — a suite-wide event bus, a global subscription across every app, a single connection multiplexing all traffic. Version one does not want any of those, and Chapter 30's event tap shows the reason: a script that subscribes to two apps and pumps once already gets the cross-app view, assembled in the VM where the star topology says every cross-app value belongs. The facility a broker would provide is one the script already provides, and it provides it without a process in the middle.

The path, resolved once

The socket path deserves one more sentence of precision because scripts sometimes need to reason about it directly. The host computes the directory as `$XDG_RUNTIME_DIR/zgui` when `XDG_RUNTIME_DIR` is set — the Linux case — and `$TMPDIR/zgui` otherwise, which is the macOS case where `XDG_RUNTIME_DIR` is usually unset. The filename is always `<app>.sock`, where `<app>` is the app's bus name — the same name `App::open` takes and `App::list` returns. This is why `App::list()` can be implemented as a directory scan: it reads the `zgui/` directory, takes every entry ending in `.sock`, and strips the suffix to recover the bus name.

The directory is the registry; there is no separate discovery service to keep in sync with reality, because the presence of a bound, connectable socket file *is* the registration, and its absence *is* the deregistration.

Chapter 7 – The Hybrid Handler

What each app actually ships

The design in the earlier chapters is the contract; the shipped reality is a file named `bus.rs` in each app's Tauri backend, and it is worth reading because it resolves a tension the design implies but does not spell out. An app is not one surface. It is a Rust engine — the `*-core` crate the shell already calls — plus a frontend `appshell` with its own registered verbs. A remote `call` might target either. The handler is therefore **hybrid**, and its own header comment says so:

Engine commands — the verbs listed in the core's `commands::COMMANDS` — route straight to the same `*-core` engine the UI uses, in Rust, with no round trip through the webview. A membership test decides: `is_engine_cmd(cmd)` is simply `core::commands::COMMANDS.contains(&cmd)`. Everything else — the `appshell.*` verbs, an embedded core's verbs, anything registered through `ZGui.automation` — is forwarded to the webview: the backend evaluates a dispatch shim, `window.__zguiBridgeDispatch(id, kind, name, args)`, which runs the registered verb and reports the result back through a `zgui_bridge_reply` command that fulfills a per-request channel.

The effect is that the *entire* app shell is scriptable, not merely the engine. Fast verbs stay fast — an engine call never leaves Rust — while the full frontend surface remains reachable through the one forwarding path. The socket host itself is the shared `zgui-bridge` crate: `serve`, a `Bridge`, and a `Handler` the app implements. One socket per running process, exactly as the ownership rule requires.

Discovery versus dispatch

There is a deliberate asymmetry between what an app *lists* and what it *accepts*. `verbs()` returns the enumerable surface — the engine's `COMMANDS` plus the registered frontend verbs — so a script can discover what exists. But `call` accepts any command the handler can route, even one discovery does not advertise. `zwire` is the clearest case: its host lists a `SURFACE_VERBS` array for discovery, but the comment on that array notes that `call` accepts any `browser.<verb>`, because the Chromium HUD's own executor is the thing that runs it. Discovery is a courtesy to the script author; dispatch is governed by the allow-list the handler enforces. The two need not be identical, and where they differ, the book says so.

The routing decision, in one branch

The whole hybrid handler reduces to a single branch taken once per incoming call, and it is short enough to state exactly. The handler receives a verb name; it asks `is_engine_cmd(cmd)`, which is `core::commands::COMMANDS.contains(&cmd)`; and it forks. If the verb is an engine command, the handler decodes the arguments and calls the `*-core` engine directly, in Rust, composing the reply from the engine's return value — no webview, no JavaScript, no second process. If it is not, the verb belongs to the frontend, and the handler evaluates the dispatch shim in the webview and waits on the per-request reply channel described in Chapter 4. Two arms, one predicate. The predicate is a set membership test against a list the core already maintains for its own UI, so the handler adds no new registry of its own: the same `COMMANDS` list that tells the shell which commands the engine answers tells the bus which calls skip the webview. The handler is thin precisely because it delegates the hard question — “does the engine implement this?” — to the one authority that already knows, the core's own command list.

Namespacing an embedded core

The ownership rule of Chapter 2 says an embedded core contributes its verbs *into the host app's surface* rather than opening a socket of its own, and the hybrid handler is where that rule becomes code. An app that embeds a second engine — a mail client that embeds a calendar core, say — registers the embedded core's verbs under a distinct namespace prefix so that `calendar.event.add` and the mail engine's own `message.send` never collide, and both answer on the one socket the host app owns. The handler's routing does not change: an embedded core's verbs are either in some `COMMANDS` list it can test against, in which case they route straight to that engine in Rust, or they are frontend-registered, in which case they take the webview path like any other frontend verb. The single front door holds no matter how many engines sit behind it, because the door does not care how many rooms the house has — it cares only whether the verb a caller named is one this process, by any of its parts, knows how to answer.

Chapter 8 – The Surface, App by App

The census

The bus is no longer a proposal; it is a surface with a size. Counting the verbs each app exposes — its engine `COMMANDS` list, its host's verb array, or (for the coreless trading terminal) the entire `⌘K` palette catalog its webview registers as bus verbs — plus the shared `appShell` verbs every app inherits, the live surface is **3,073 actions across eleven apps** with an enumerable verb list, plus fifteen shared shell verbs. The catalog `GUI_SCRIPT_ACTIONS.md` is generated from each app's source, so the figure moves with the code rather than aging in prose.

App	Verbs	The surface, in one line
traderview	1726	every <code>view:</code> tile and <code>action:</code> shortcut — a charting / trading terminal whose whole <code>⌘K</code> palette is scriptable
zpdf	309	render, edit, annotate, forms, OCR, redact — an Acrobat-scale PDF engine
zcite	178	library, collections, citations, PDF, sync — a reference manager
zemail	174	accounts, folders, messages, PGP and S/MIME, search
zwire	161	tabs, windows, tab groups, downloads, reading list, power — a browser
zftp	123	FTP, SFTP, WebDAV, S3 and cloud stores, transfers, sync
zreq	121	requests, collections, auth, codegen, gRPC and WebSocket
ztunnel	97	OpenVPN and WireGuard configuration and control
zoffice	96	writer, calc, impress, over ODF and OOXML
zthrottle	54	system monitoring, process and network throttling
zgo	19	launcher system commands — sleep, lock, empty trash, and the rest
appShell	15	terminal, files, hooks, palette, theme, CRT, neon — on every app

The shared fifteen

Because every app is a shell over `zgui-core`, every app inherits the same `appshell.*` verbs without declaring them: the terminal (`appshell.terminal.open`, `.close`), the file browser (`appshell.files.open`,

.close), the settings and shortcuts panels, the command palette, and the visual toggles for the light and dark themes, the CRT scanline effect, and the neon glow. A script that knows only these fifteen can already open a terminal, flip a theme, or summon the palette in any app in the suite — before it learns a single app-specific verb.

Shape, not just size

The verbs are namespaced, and the namespace is the app's own vocabulary made legible. zreq's surface reads as a map of Postman: `collection.*`, `environment.*`, `request.*`, `codegen.*`, `graphql.*`, `grpc.*`. zcite's reads as Zotero: `library.*`, `collection.*`, `item.*`, `citation.*`. Reading a namespace is reading the product's domain model. That is what makes the surface scriptable in the human sense as well as the machine sense: the verb you want is named the thing you would call it.

Not every app keeps its verbs in a Rust list. traderview has no `-core` engine at all — its `bus.rs` forwards everything to the webview's `ZGui.automation` surface — yet it is the *largest* app on the bus, because its front-end registers the whole ⌘K palette catalog as verbs: every `view:` tile and `action:` shortcut, the ephemeral recents and favourites deliberately filtered out. That surface is enumerable from source (the tile table and the shortcut list), so the catalog counts it honestly. Four other coreless apps — zphoto, zstation, ztranslator, and zcontainer — carry the bus but register no comparable catalog; they expose the shared `appShell` verbs plus their Tauri backend commands, and the catalog enumerates only the shell surface for them, saying so rather than inventing a count. Honesty about the boundary of what is enumerable is part of the catalog's contract.

Reading the census critically

A table of counts invites a ranking, and the ranking would be wrong. The number in the verbs column measures one thing precisely — how many distinct ids the generator could enumerate from that app's source — and a reader who takes it for anything else will draw conclusions the figure does not support. Three distortions are worth holding in mind while reading the table above.

The first is **width against depth**. The largest surface on the bus belongs to an app with no engine at all, because its front-end registers an entire palette catalog: every tile and every shortcut becomes a verb. Those are real, callable, useful verbs, and there are a great many of them, but each does a small, single thing — show this view, run this shortcut. An engine-derived surface is built differently. A verb like `calc.render` or `export.marxml` or `sync.plan` carries a whole subsystem behind one id, takes parameters that meaningfully change what it does, and returns a structured value a script reasons over. One hundred engine verbs and one thousand palette verbs are not comparable quantities; they are two different units that the catalog, honestly, counts the same way because counting them the same way is the only thing it can prove.

The second is **namespace count as the better shape signal**. A surface's namespaces say more about what a script can accomplish than its verb total does. An app with a dozen namespaces has a dozen distinct domains a script can operate in; an app with one namespace and four hundred verbs has one domain enumerated exhaustively. When sizing up an unfamiliar app for automation, the useful first move is not to read the count but to list the namespaces — which the `verbs()` filter of Chapter 32 makes a two-line habit — because the namespaces are the app's domain model and the count is only its granularity.

The third is that **the census is a floor, not a measurement of power**. The under-counted apps are under-counted because their surface is not enumerable from source, not because they do little. The same asymmetry runs the other way for every app on the table: a verb whose parameters open up a large space of behaviors counts once, exactly as a verb with no parameters does. The figure is a lower bound on reachable capability expressed in the one unit the generator can defend, and its value is precisely that it is defensible. Treat it as evidence that the surface is large and real, which it supports completely, rather than as a ranking of the apps, which it does not support at all.

zcite, read as a surface

Take the reference manager as the first close read, because its surface maps cleanly onto a working librarian's day. The `library.*` namespace is the entry point — search, the entire holdings, the recently added — and it is the namespace the opening example of the book called into. `collection.*` is the shelving system: `collection.add` and `collection.rename` manage the folders, `collection.add_item` and `collection.remove_item` move items between them, and the read-only `collection.tree`, `collection.list`, and `collection.diff` report their structure. `item.*` is the item lifecycle — `item.add`, `item.add_attachment`, and the rest — and it is the namespace whose returns matter most, because `item.add` hands back the item it created, the fact Chapter 16's reading-list recipe turns on. Around that core sit the interchange namespaces: `bib.*` renders citations and bibliographies (`bib.bibliography`, `bib.citation`, `bib.csl`, `bib.styles`), while `export.*` and `import.*` are the widest namespaces of all, each carrying a couched knowledge of a format — `export.bibtex`, `export.ris`, `export.csl_json`, `export.marcxml`, `export.zotero_rdf` outbound, and the matching `import.crossref_json`, `import.pubmed_xml`, `import.datacite_json` inbound. A script that reads `library.*` to find work and `export.*` to render it has automated the two ends of the manager without touching a window.

zreq, read as a surface

The API client's surface reads as a map of the request lifecycle. `request.*` is the verb-dense center: `request.send` fires one and returns the response, `request.add`, `request.update`, `request.get`, and `request.remove` manage saved requests, and `request.fuzz` drives a request repeatedly under varied inputs. `collection.*` groups requests the way `zcite`'s groups items, and `environment.*` — `environment.activate`, `environment.add`, `environment.list`, `environment.remove` — is the variable-binding layer that makes one saved request run against staging or production by swapping the active environment rather than editing the request. The protocol namespaces widen the client past plain HTTP: `graphql.*` introspects and parses schemas (`graphql.introspection_query`, `graphql.schema.parse`), `grpc.call` invokes a gRPC method, and `codegen.generate` turns a saved request into client code in a chosen language. `history.*` closes the loop — `history.list`, `history.get`, `history.replay` — so a script can re-run past traffic without reconstructing it. The surface is Postman rendered as verbs, and the verb you want is named the thing you would call it.

zoffice, read as a surface

The office engine splits its surface by document kind, and the split is legible in the namespaces. `calc.*` is the largest and the most script-friendly, because a spreadsheet is the most naturally programmable of the document types: `calc.open`, `calc.eval` and `calc.evaluate` for formula evaluation, `calc.edit_cell`, `calc.find` and `calc.replace`, `calc.sort`, `calc.named_ranges`, `calc.pivot_tables`, `calc.charts`, `calc.validations`, `calc.conditional_formats`, and the render/export family `calc.pdf`, `calc.csv`, `calc.html`, `calc.markdown`, `calc.render`. `writer.*` is the prose engine and `impress.*` the presentation engine (`impress.charts_render`, `impress.graphic_objects`, `impress.find`), while `draw.*` handles vector documents (`draw.connectors`, `draw.svg`, `draw.render`). The invoice recipe of Chapter 16 leans on this surface: it renders a billing range out of `calc` and hands the resulting file to the PDF engine, one export verb feeding one open verb across the socket.

zemail, zftp, and ztunnel in brief

Three more surfaces round out the picture without a full walk each. The mail client's namespaces track a mail app's nouns: `account.*` (`account.add`, `account.autoconfig`, `account.list`), `folder.*` (`folder.create`, `folder.inbox_load`, `folder.digest`), `message.*` for composition and sending, `contact.*` (`contact.import_vcard`, `contact.find_duplicates`, `contact.merge`),

`filter.*` (`filter.run`, `filter.to_sieve`), and the protocol-level `imap.*` (`imap.folders`, `imap.idle`, `imap.build_search`), alongside the PGP and S/MIME security surface the census line names. The transfer client's surface is a catalog of transfer protocols: `transfer.*` (`transfer.add`, `transfer.cancel`, `transfer.resume_check`, `transfer.multipart_plan`), the wire-level `sftp.*`, `ftp.*`, and `s3.*` (`s3.presign`, `s3.list_objects`), the remote-filesystem `fs.*` (`fs.list`, `fs.mkdir`, `fs.chmod`, `fs.rename`), and `sync.*` (`sync.compare`, `sync.plan`, `sync.resolve`). The VPN client is control and configuration: `vpn.*` (`vpn.connect`, `vpn.disconnect`, `vpn.status`, `vpn.reconnect_schedule`, `vpn.killswitch_plan`), the WireGuard `wg.*` (`wg.genkey`, `wg.pubkey`, `wg.handshake_model`), the OpenVPN `openvpn.*` (`openvpn.push_reply`, `openvpn.inline_blocks`), and `dns.*` (`dns.leak_check`, `dns.split_horizon`). The tunnel recipe of Chapter 16 gates sensitive work on `vpn.status` reporting connected — the read path the old palette never had.

The coreless apps and the honest gap

The census closes on the apps whose surface the catalog deliberately under-counts, and the honesty there is worth restating as a property rather than an apology. `traderview` is the extreme in one direction: no `-core` engine at all, yet the largest surface on the bus, because its whole `⌘K` palette — every `view:` tile and `action:` shortcut — is registered as bus verbs, and the tile table and shortcut list are enumerable from source so the catalog counts them exactly. `zphoto`, `zstation`, `ztranslator`, and `zcontainer` are the extreme in the other direction: they carry the bus and answer on a socket, but they register no comparable frontend catalog, so what the generator can enumerate for them is the shared `appShell` verbs plus their Tauri backend commands, and the catalog reports that surface and no more. The number for those apps is small not because the apps are small but because the *enumerable* surface is, and the catalog's contract is to report what it can prove, never to estimate what it cannot. That discipline is why every count in Chapter 8 can be trusted: it is a floor the generator measured, not a figure a writer hoped for.

Chapter 9 — Three Front-Ends, One Module

Layer 5 is the promise that there is no per-surface logic. Three different places run `stryke`; all three get `App` in scope and use the same module.

The `palette stryke step` used to pass only a context argument. Extended, it binds `App::here()` to the current app, so a palette command stops being a single `event` fire and becomes a real orchestration — read the selection, call a verb, branch on the result — while the step-chain grammar (`url | js | scheme | event | stryke | bash`) is otherwise unchanged. The `stryke` step simply got more powerful.

The `hooks runtime` gets the identical binding, so an app's hooks can react to emitted events and call verbs in response — the app scripting itself as things happen inside it.

The `standalone .stk` file runs from the shell and uses `App::open(name)` over the socket. This is the cross-app bus: one file, one terminal, several running apps driven together. The same module serves all three because the transport difference is hidden beneath it — the palette step and the shell file call the identical `App` methods, and only the layer below knows one used a callback and the other a socket.

The palette step, before and after

The clearest way to feel the change is to put the old palette step next to the new one. Before the bus, a palette command that wanted to react to a search had no way to; the best it could do was fire one `event` step and stop, learning nothing. After the bus, the same command becomes a real branch, because the `stryke` step now has `App::here()` bound to the current app:

```
# a palette command's stryke step - runs inside the app that owns the palette
val $me = App::here()
val @hits = @{ $me->call("library.search", %{ q => $ctx->{query} }) }
if scalar @hits == 0 {
    $me->call("appshell.palette.toast", %{ text => "no matches" })
} else {
    $me->call("collection.add", %{ name => "search: ${ $ctx->{query} }" })
}
```

The step still receives its context argument, `$ctx`, exactly as it always did — that part of the contract did not change — but now the same step can read the app, branch on the result, and call a second verb. The palette did not gain a special palette API; it gained the one binding that puts the whole bus in the step's scope, and everything the standalone `.stk` file can do, the palette step can now do too, against `here()` instead of `open()`.

The hook, reacting to the app

The hooks runtime is the same binding aimed at a different trigger. A palette step runs when the user invokes it; a hook runs when the app emits an event the hook subscribes to. The identical `App::here()` is in scope, so a hook can read state and call verbs in response to the app scripting itself as things happen inside it:

```
# a hook fired on the app's own "itemAdded" event
val $me = App::here()
val $item = $ctx->{payload} # the emitted item
if $item->{type} == "journalArticle" && !$item->{doi} {
  # a new article with no DOI – try to resolve one from its title
  val @found = @{$me->call("library.search", %{ q => $item->{title} }) }
  if scalar @found > 0 { p "possible match: ${ @found->[0]->{doi} }" }
}
```

Three surfaces, one module, one binding. The palette step reacts to the user, the hook reacts to the app, and the `.stk` file reacts to nothing — it drives from the outside. All three write the same `App` calls, and none of them contains a line of code that knows which surface it is running on. That is the whole content of Layer 5: not three integrations, but one module placed in three scopes.

Choosing which of the three to write in

Because the module is identical across the three surfaces, the choice between them is not a technical one — any of the three can express any automation — and that makes it a design question rather than a capability question. Three properties separate them, and each one decides a different class of script.

The first is **what starts it**. A palette step starts because the user chose it from the palette. A hook starts because the app emitted something. A `.stk` file starts because the shell ran it, which may mean a person typed its name or may mean a timer fired. If the automation's natural trigger is a moment in the user's workflow, the palette step is not merely convenient, it is the correct home, because putting it in a file means inventing a second way to invoke it that duplicates a launcher the app already has.

The second is **how many apps it touches**. A palette step and a hook run inside one app and reach it through `App::here()`; they can still `App::open` a second app, but doing so puts a cross-app orchestration inside one app's palette, where it is discoverable only by the person who remembers which app they filed it under. A run that genuinely spans four apps belongs in a file, where its name describes the run rather than the app it happens to have been launched from. The star topology of Chapter 13 has a centre, and the centre should live somewhere that does not belong to any one spoke.

The third is **whether it must survive the app**. A palette step and a hook exist inside a running app's process; they are gone when the app is. A `.stk` file exists on disk, is version-controlled with everything else, is testable by the script-logic method of Chapter 21, and composes with the shell per Chapter 45. Anything that is going to be maintained rather than merely used wants to be a file, and the fact that a file developed at the prompt can be pasted into a palette step unchanged — because no line in it knows which surface it is on — means the file is also the better place to *develop* a palette step before it becomes one.

The defaults those three produce are worth stating outright. A one-off convenience the user invokes: palette step. An automatic reaction to something happening inside one app: hook. Anything else — anything scheduled, anything spanning apps, anything with enough logic to be worth testing, anything anyone else will read — a file. The portability guarantee means none of those choices is expensive to revisit, which is the point of Layer 5 having no per-surface logic in the first place: the decision can be made late and changed cheaply, because the code does not encode it.

Chapter 10 – Security and the Allow-List

The bus is powerful, so its security model is stated in three flat rules rather than left implicit.

First, **the socket is local and private**. It lives in the per-user runtime directory at mode 0600. There is no network listener, ever. The bus is a single-user, single-machine facility by construction, not by configuration — there is no setting that opens it to the network because there is no network code to enable.

Second, **the target must be running**. An out-of-process `call` dials a socket that exists only while the app is up; dialing a dead app fails and the script dies. Version one has no launch-on-demand, so a script cannot silently spawn apps as a side effect of automating them.

Third, **verbs are an allow-list**. Only what an app registered through `ZGui.automation.register` is reachable. There is deliberately no generic “evaluate this JavaScript in the app” verb, because such a verb would dissolve the typed surface into arbitrary code and make the allow-list meaningless. The escape hatches that do run arbitrary code — the `js` and `bash` palette steps — stay exactly where they are: explicit, user-authored steps in a local palette, never remotely callable verbs. The line between “a script the user wrote and ran” and “a verb a remote caller can invoke” is the line the security model protects.

The trust boundary, drawn precisely

It helps to state exactly where the trust boundary runs, because the three rules above are its consequences rather than its definition. The boundary is the socket file’s owner. Everything on the inside — the user’s own scripts, the user’s palette steps, the user’s hooks — is trusted with the full surface, because it is all code the one user wrote or chose to run under their own account. Everything on the outside — another user on the same machine, a process running as a different uid, anything across a network — is trusted with nothing, because mode 0600 means the socket file cannot even be opened by them. There is no middle tier, no partially-trusted caller, no capability token to scope down a dialer, because the model has exactly two states: you are the user, and you have everything, or you are not, and you have no socket to dial. The simplicity is the security property. A model with a middle tier has to get the middle tier right; a model with no middle tier has only the file-permission bit to get right, and the operating system gets that bit right for a living.

Why there is no “eval” verb

The refusal to ship a generic “run this code in the app” verb deserves its own treatment, because it is the single decision that keeps the allow-list meaningful. A verb that evaluated arbitrary JavaScript, or ran an arbitrary shell command, or read an arbitrary file, would be one entry in the allow-list that made every other entry irrelevant: a caller with that one verb could do anything the app’s process could do, and the careful enumeration of `library.search`, `item.add`, and the rest would be decoration over a hole. So the surface has no such verb, anywhere, by policy. The apps that legitimately need to run arbitrary code — and some do — expose it only through the palette’s `js` and `bash` steps, which are not bus verbs at all: they are steps in a command the user typed into their own palette, on their own machine, and they are never reachable through `call`. The distinction is exact. `App:open("zcite") ->call("bash", ...)` does not exist and cannot be made to exist, because `bash` is a palette step kind, not a registered verb, and only registered verbs are

callable. The escape hatch stays an escape hatch — a thing the user reaches for deliberately in a local editor — and never becomes a remote capability.

What the allow-list is, mechanically

“Allow-list” is a precise claim about the dispatch path, not a policy statement, so it is worth grounding in the handler. When a `call` frame arrives, the handler resolves the verb name against exactly two sources: the engine’s `COMMANDS` list and the set of verbs registered through `ZGui.automation`. A name in neither set is not dispatched to some default handler that guesses; it is rejected with `ok:false` and the message `no such verb: <name>`, and no app code runs. This is what makes the surface an allow-list rather than a deny-list: the default answer to an unrecognized verb is *no*, and a verb becomes callable only by being explicitly added to one of the two known sets. The one deliberate widening, described in Chapter 7, is *zwire*’s host accepting any `browser.<verb>` because its Chromium HUD is the executor — but even that is an allow-list with a wildcard *namespace*, not an open door: a frame for `foo.bar` against *zwire* is still rejected, because it is not under the one prefix the handler was told to forward. The wildcard is scoped, declared, and documented; it is the exception that shows the rule is real.

The threat the model does not address, named

An honest security chapter names what it does not defend, and this one leaves one threat explicitly out of scope: a malicious script the user runs themselves. The bus gives a script the full surface of every running app, so a hostile `.stk` file the user is tricked into executing can do real damage — send mail, delete files over `fs.delete`, bring a tunnel down. The bus does not defend against this and does not try, because it cannot: a script the user chose to run is, by the model’s own definition, inside the trust boundary, indistinguishable from a script the user wrote. The defense against a hostile script is the same defense as against any hostile program the user runs — do not run programs you do not trust — and it lives at the level of the user’s judgment, not the bus’s dispatch. Naming this is part of the model’s honesty: the bus secures the *boundary*, not the user’s choices inside it.

Chapter 11 — Novelty — An Honest Prior-Art Analysis

A combination, not a single invention

This chapter is written in the house style of [INVENTIONS.md](#): claim the smallest true thing, name the prior art, and record confidence honestly. The bus is a **combination** first, not a single new capability. Embedding a language in an application, scripting across applications, and shipping a vendor-authored automation language each predate this work, separately and by decades. The claim is the *conjunction*, under constraints — and the honest form of that claim is “none found, low confidence,” a packaging novelty, not a proven categorical first.

The four nearest prior arts each match part of the shape and fail a load-bearing leg:

Prior art	Matches	Fails on
AppleScript / OSA (1993) — 20+ scriptable first-party apps	app count, a vendor language	macOS-only; apps are Apple-Event targets and the interpreter is external, not embedded in the app
VBA — hosted in-process in Word, Excel, and the rest LibreOffice with Basic and UNO	in-process embedding, a vendor language, cross-app via COM cross-platform, embedded, scriptable across apps	not cross-platform; single domain (office) single domain; interpreted Basic with no JIT; a component model, not shared embeddable cores; not solo-authored
KDE with Kross and D-Bus	domain diversity, cross-app	Kross bridges Python, Ruby, and JavaScript rather than being a language; no owner-authored language, no JIT, IPC not shared cores

What survives the intersection

Take the intersection of all the legs and no prior art holds it: a domain-diverse suite — a PDF editor, an email client, a VPN, a browser, an office suite, and more — that is cross-platform, solo-authored, driven by the author’s own from-scratch Cranelift-JIT language, embedded in-process across shared embeddable engine cores. AppleScript is macOS-only and external; VBA is Windows-first and single-domain; LibreOffice is single-domain with no JIT; KDE has no owner language. The combination is the claim, and it is recorded as a combination — low confidence, non-exhaustive search — precisely so that the claim survives scrutiny.

The **speed leg** is the part that is not a packaging argument. `stryke` is a Cranelift-JIT VM; an in-process call is a host callback, not a fork; an out-of-process call is a Unix socket, not HTTP. There is no subprocess per call — the same discipline as `stryke-gui` holding one persistent input handle rather than spawning a tool per keystroke. Whatever else is or is not first here, the bus is built to be fast in a way that shelling out to per-platform scripting runtimes structurally cannot be.

The modern comparison, and why it also misses

The prior-art table reaches back to the nineties because the *shape* the bus occupies is old, but a fair reading has to address the modern tools too, since a reader will reach for them first. Browser automation frameworks drive one class of application through one wire protocol, but they drive an external target and speak no vendor language of their own — they are a client library, not an embedded interpreter, and they cover exactly one app. General RPC frameworks give typed cross-process calls with generated stubs, which is genuinely close on the transport leg, but they are a plumbing layer a developer builds a system *on top of*, not a scripting surface a user drives apps *through*; there is no user-facing language and no notion of a running app's live state to read. Shell-level automation tools chain commands, but a command-line tool is not an application's internal surface — it is whatever the tool's author chose to expose on the command line, which for a GUI app is usually nothing. Each modern tool matches the leg the old ones missed and misses a leg the old ones had. The bus's claim survives the modern intersection for the same reason it survives the historical one: the conjunction of an owner-authored JIT language, embedded in-process across a domain-diverse suite of shared cores, with one uniform local transport, is not the thing any single existing tool set out to be.

The confidence, stated as a discipline

The chapter ends where the house style demands: on the confidence, stated plainly and low. “None found” is a claim about a search, and the search was not exhaustive, so the honest label is “none found, low confidence” — a packaging novelty asserted as a combination, not a categorical first proven against the whole history of software. This is not modesty for its own sake; it is the only claim that survives an adversarial reading. A stronger claim — “the first ever” — invites a single counterexample to falsify it, and the space of scripting systems is too large and too poorly indexed for anyone to have checked it all. The combination claim is different: it does not assert that no one has ever built anything like this, only that the specific conjunction of legs, held all at once, is one this author has not found elsewhere and believes to be rare. That claim a counterexample refines rather than destroys, which is exactly the property a claim published in a book about a shipped thing should have.

Chapter 12 — Rollout and Open Questions

Shared once, then per app

The rollout separates the work that happens once from the work that repeats. The shared pieces are four: the `zgui-core automation.js` surface registry with its JS bridge dispatch and `emit`; the `zgui-bridge` Rust crate with the Unix-socket host, the frame codec, and the request router; the `stryke-app` package in `strykelang` with its C-ABI exports and `App` module; and the `run_stryke_hook` change in each app backend that binds `App::here()` into the script's environment.

The per-app work is deliberately thin, because the architecture front-loaded it. An app declares its surface with one `ZGui.automation.register` call — and most of those verbs already existed as palette actions, now merely typed — and serves its socket with one `zgui_bridge::serve` line in `main`. Then it is verified twice: a `.stk` script drives it in-process through a palette step, and out-of-process from the shell. The pilots were chosen for the richest existing surfaces, so the loop could be proven before the fan-out; the census in Chapter 8 is what the fan-out produced.

The questions left open

A design honest enough to publish is honest about what it has not settled. Five questions remain live. Whether `App::open` holds its socket for the script's life or redials per call — the proposal holds and reconnects on a broken pipe. How strict parameter typing is at the JSON boundary — the proposal validates against the manifest and dies with the verb's signature on a mismatch. Whether `on` callbacks run on a stryke event loop or drain a queue the script pumps — the proposal drains at an explicit pump point, to stay inside stryke's execution model. Whether the socket host is best as a shared crate or a per-app module, given the vendorable-and-durable constraint every dependency in this stack must satisfy. And whether launch-on-demand belongs in a later version, so a script can open an app that is not yet running.

None of these blocks the surface that already exists. More than three thousand verbs answer today. The open questions are about making the edges of the bus as considered as its center — which is the right kind of question to still be asking about a thing that is already built.

The pilot loop, in detail

The rollout's "verified twice" deserves unpacking, because the two verifications are not redundant — they exercise the two transports, and a surface can pass one and fail the other. The in-process verification runs a `.stk` snippet as a palette step: it binds `App::here()`, calls a verb, reads a state query, and prints the result into the palette's own output, confirming that the surface registered, that the bridge's webview path resolves the verb, and that a value comes back into the VM. The out-of-process verification runs the same logical script as a standalone file from the shell, using `App::open(name)` instead of `here()`, which additionally confirms that the socket bound, that the frame codec round-trips, and that `App::list()` sees the app. Passing in-process but failing out-of-process points at the socket host — the `serve` line missing, the path wrong, a permissions problem. Passing out-of-process but failing in-process points at the palette binding. Two transports, two failure

signatures, and running both on every pilot is what let the fan-out proceed on the confidence that a green surface was green on both paths, not just the one the author happened to test.

Why the pilots were the richest apps

Choosing the pilots for their *richest existing surfaces* looks backward — surely the simplest app is the safest first move — but the logic is sound and worth stating. A rich surface exercises the most of the shared machinery on the first try: many verbs with many parameter shapes stress the argument validator, state queries stress the `get` path, events stress the subscription stream, and a large `COMMANDS` list stresses the hybrid handler's engine-versus-frontend split. A trivial app with three verbs would have proven almost nothing about the shared pieces, so a bug in the codec or the pump would have surfaced only later, during fan-out, when it was more expensive to trace across a dozen apps at once. Proving the shared substrate against the hardest surface first means the fan-out inherits a substrate already tested against everything it will face — the eleventh app to join found nothing left to break that the first, hardest one had not already found.

Chapter 13 – The Value Model at the Edge

What crosses the wire

JSON is the only thing on the wire, so the value model on the wire is JSON's: null, boolean, number, string, array, object, and nothing else. stryke's model is richer — scalars, arrays written with `@`, maps written with `%`, live handles, closures — and so every argument going out and every result coming back is a projection between the two. The projection is total in one direction and partial in the other, and the partiality is where the interesting decisions live.

Outbound, from script to app, the projection is total. An argument map like `%{ q => "graphene", limit => 20 }` becomes a JSON object; a stryke array becomes a JSON array; scalars become the matching JSON scalars. Two stryke values have no JSON image and so cannot be arguments, and this is a feature rather than a gap. A closure cannot cross, because there is nothing on the far side to run it against and no way to serialize a body of JIT-compiled code. A handle cannot cross either, because a handle is a live connection and not a value — a point Chapter 5 states and this chapter makes load-bearing below. The one place a callback appears in the API, `on`, never serializes it: the closure stays resident in the VM and only the subscription `id` travels, so events are delivered by pumping frames through a function that never left home.

Inbound, from app to script, the projection has one lossy seam and is otherwise clean. A JSON array decodes to a stryke array a script walks with `for val $x (@{ ... })`; a JSON object decodes to a map indexed as `$it->{doi}`; JSON `null` decodes to stryke `nil`, which is exactly the value a script tests for when a state query has no answer. The seam is numbers. JSON has a single numeric type; stryke distinguishes integer from float. The codec restores an integer when the decoded value is integral and a float otherwise, which is precisely right for the overwhelming majority of verbs — a row count, an id, a page number are integers and arrive as integers — and merely conventional for the rare verb whose contract is an integral-valued float. A script that needs to be certain coerces at the point of use; the codec's rule is the one that surprises the fewest callers.

Why a handle is not a value

The rule that a handle cannot be an argument is worth dwelling on, because it shapes the whole topology of a multi-app run. A handle names a transport plus a target: it is, concretely, an open socket to `zcite` or a bound callback into the local surface. Serializing that and shipping it to another process would mean shipping a socket, which means nothing on the far end. So handles are VM-local by construction. `App::open` mints one, the handle methods consume it, and it dies when the script does.

The consequence is architectural, not merely mechanical. A script that wants `zreq` to act on data living in `zcite` does not — cannot — hand `zreq` a handle to `zcite` and step back. It reads from `zcite` itself and calls into `zreq` itself, staying the hub through which every value passes. The topology is therefore always a star, with the script at the center and each app a spoke, and no app ever holds a live connection to another app. That is the same shape the security model wants for its own reasons — an app is never a client of another app, only ever a server to the user's own scripts — and it falls out for free from the single decision that a handle is not a value. Two chapters, one constraint, no coordination needed between them.

The number seam, worked

The one lossy seam — JSON's single numeric type against stryke's integer/float split — is worth working through a concrete case, because it is the projection decision a careful script author will eventually hit. Consider `library.search` returning items, each carrying a `year` field. On the wire the year is JSON `1998`, a bare number with no type tag. The codec's rule is: decode an integral value to a stryke integer and a non-integral value to a stryke float. So `1998` becomes the integer `1998`, and `$item->{year} + 1` is integer arithmetic, which is what a script author expects. The rule only surprises in the rare inverted case: a verb whose contract is a float that happens to be integral — a price of exactly `20.0`, say, arriving on the wire as `20` — decodes to the integer `20`, and a script that then formats it as currency has to know to coerce. The codec chooses the rule that surprises the fewest callers rather than the rule that is theoretically lossless, because there is no lossless choice: JSON threw the type away before the codec ever saw the value. A script that needs certainty coerces at the point of use — `$price * 1.0` forces a float — and the codec's convention handles the overwhelming common case for free.

Large values, and where the projection hurts

The projection is described so far as if size were free, and for the overwhelming majority of calls it is: a search result, a status map, a list of transfers are all small enough that the encode, the copy, and the decode disappear into the verb's own work. There is a regime where that stops being true, and knowing where it is prevents the one performance surprise the value model can produce.

Every value that crosses is serialized whole, copied through the socket whole, and decoded whole before the call returns. There is no streaming, no chunking, and no lazy handle to a value that stays on the app's side. `App::call` is a function that returns a value, and a value is a thing the VM holds in memory. So a verb that returns a hundred rows costs nothing worth measuring, and a verb asked to return an entire library, or the full contents of a large document, pays the encode and the decode in full, at once, on both sides.

The remedy is not in the transport; it is in how the surface is asked. Three moves cover essentially every case.

Ask for less. A verb with a `limit` parameter exists so a script can bound the result, and the pagination-shaped verbs across the census — a search with a limit, a history list with a count — are there for this reason. A script that wants the first twenty matches should say twenty rather than take everything and slice in the VM, because the slice in the VM happens after both serializations have already been paid.

Move the filter into the app. The general rule is the pure-helper seam of Chapter 34 read in the other direction: compute in stryke what stryke can compute, but do not drag a large intermediate across the wire in order to reduce it locally when the app can reduce it first. Filtering a large result set down to three rows in the script means the wire carried the whole set to produce three rows.

Pass paths, not contents. The invoice run of Chapter 16 is the model, and its shape is not accidental. `calc.exportRange` returns `$out->{path}`, and the PDF engine is then handed that path — the document itself never enters the script, never crosses a socket, and never occupies the VM. The apps exchange a filename and the filesystem carries the bytes. That is the idiomatic way to move anything large through the bus, and it generalizes: when two apps need to hand each other something substantial, the value that crosses should be a reference the receiving app can resolve, not the substance itself.

The through-line is that the wire is for *answers*, not for bulk. It is a control plane — typed, correlated, legible, small — and the design's decision to make it newline JSON was made on the assumption that it would carry decisions and identifiers rather than payloads. Used that way it is fast and, more importantly, legible in a `socat` transcript, which is the property Chapter 23 depends on. A protocol that routinely carried megabytes would have had to give up the legibility to stay reasonable, and it would have bought a capability the filesystem already provides better.

The three values that cannot cross, together

Chapter's earlier sections named the non-serializable values one at a time; seen together they form a single rule with one reason. A closure cannot cross because there is no far side to run it against. A handle cannot cross because it is a live connection, not a value. A stryke value carrying either transitively — a map with a closure inside it — cannot cross whole, because the projection to JSON has no image for the interior. The unifying reason is that JSON is a *data* format and these are *behavior and connection*, not data. The bus does not paper over the gap with a fake serialization that would deserialize to something broken on the far side; it refuses the value at the boundary, loudly, so the script author learns at the call site that they tried to send something that is not sendable. This is the same posture as the failure model of the next chapter: fail where the fault is, not three statements later where a broken value finally gets dereferenced.

Chapter 14 — Failure Is a Reply

The two ways a call can end

Every `call`, `get`, and `verbs` request ends in exactly one reply frame, and that frame's `ok` field is the fork in the road. `ok:true` carries a `value`; `ok:false` carries an `error` string. The `App` module turns the two into two different things inside the VM, and the difference is deliberate. A true reply decodes its `value` and returns it to the calling expression. A false reply raises, and an unhandled raise dies the script with the error string as its message. So `no such verb: foo.bar` is not a sentinel a script must remember to test for — it is a death, loud and immediate, the same kind of death as a nil dereference or a failed `App::open`.

This is a design choice with a defensible reason. A mistyped verb name in an automation script is a bug in the script, not a branch the script meant to take. Returning nil for an unknown verb would let the mistake propagate silently until some later line dereferenced the nil and blamed the wrong statement. Dying at the call site, with the verb's own name in the message, puts the error where the fault is. Scripts that genuinely want to probe for a verb's existence read `verbs()` and test membership; they do not fish by calling and catching. The fast path — call and use — stays free of defensive ceremony because the slow path fails honestly.

The failures the wire cannot phrase

Some failures never become a reply frame at all, because the channel that would carry the reply is itself gone. An out-of-process target that exits in the middle of a call breaks the pipe; there is no `ok:false` because there is no peer left to compose it. The `App` module surfaces a broken pipe as its own kind of death, naming the app that vanished, rather than letting the script block forever on a socket no one will ever write to — a hang being the single worst outcome for an automation script, worse than any error, because it produces no signal at all.

This is exactly the design surface behind the Chapter 12 open question about holding versus redialing a socket. The proposal holds the connection for the handle's life and reconnects once on a broken pipe: a target that restarts cleanly between two iterations of a loop does not kill a script that would otherwise have marched on. The retry is single and bounded, because a socket that refuses twice in a row is a target that is genuinely down, and a script driving a down app should learn so and stop, not spin.

Timeouts

There is a third failure that neither an `ok:false` nor a broken pipe describes: a verb that accepts the call, never errors, and never resolves. In-process, a `run` that awaits a promise that never settles would hang the calling script as completely as a dead socket. The bridge therefore treats every `call` as a bounded wait rather than an open-ended one. The request carries an `id`; the reply must arrive under a deadline; a deadline that passes with no reply becomes the same kind of death as a broken pipe, again naming the verb. This is the one moment the synchronous-looking call shows the asynchronous machine beneath it. To the script, `App::call` is a function that returns a value or dies — nothing more. Underneath, the bridge is running a timer against a correlation id, and the tidy synchronous surface is the timer's success case made to look like the only case.

The full taxonomy of a failed call

Gathered into one place, a `call` can fail in exactly four ways, and a script author who holds the taxonomy in mind writes calmer automations. First, the *rejected* call: the verb ran or was refused and the app composed an `ok:false` reply — an unknown verb, a missing required argument, an app-level error the verb chose to raise. This is the common case, it names the fault, and it raises in the VM. Second, the *broken* call: the peer vanished mid-flight, the pipe broke, and no reply frame is possible; the `App` module manufactures a death naming the app that disappeared. Third, the *timed-out* call: the peer is alive but the verb never resolved under the deadline; the bridge's timer fires and manufactures a death naming the verb. Fourth, the *undialed* call: the target was never reachable, `App::open` failed at `connect`, and the script died before any frame was sent. Four failures, and the design's through-line across all of them is that every one is a *death at the call site*, never a sentinel value that propagates. The reason is uniform: an automation script that hangs, or that marches on with a nil where a value should be, produces no signal or the wrong signal, and both are worse than a loud death with the fault named. The bus spends its complexity to guarantee that a call either returns the value or stops the script — nothing quieter.

Catching, for the scripts that mean to

A death is the default, not a mandate, and a script that genuinely wants to survive a failed call can. stryke's own error handling wraps a call whose failure is a branch the author actually meant to take rather than a bug — probing an app that may or may not answer a given verb, tolerating one flaky request in a batch, treating a down tunnel as a condition to report rather than a reason to abort:

```
val $res = try { $req->call("request.send", %{ url => $u }) }  
          catch ($e) { p "skipped ${ $u }": "${ $e }"; nil }  
if $res { record($res) }
```

The point is that catching is *opt-in*. The fast path — call and use the value — carries no defensive ceremony, because the default is a death that stops the script at the fault. Only a script that has a real reason to continue past a failure reaches for the `try`, and when it does, the failure it catches is the same four-way taxonomy above, delivered as a stryke error whose message is the reply's `error` string or the manufactured broken-pipe or timeout text. The author who wants a branch writes one; the author who wants correctness by default writes nothing and gets it.

Chapter 15 – Order, Correlation, and the Pump

Ids are a conversation, not a clock

The `id` on a request is a correlation token and not a timestamp, and reading it as a clock is the mistake the transport is built to survive. A Unix socket is full-duplex, so a script is free — in a future pipelined mode, or simply because the kernel buffers — to have several requests outstanding at once, and their replies may come back in any order the app happens to finish them. The `App` module matches each reply to the waiting request by `id`; it assumes nothing about arrival order.

In today's straight-line scripts this machinery is invisible, because the synchronous `App:call` blocks on its own `id` before the next statement runs, so there is only ever one request in flight and reordering has nothing to reorder. The correlation matters for the events that share the channel, and for any later mode that lets a script fire a batch and gather the answers. It is built now, cheaply, so that the wire never has to be redesigned to add pipelining later — the `id` was always going to be the thing that made concurrency safe, so it is the thing on every frame from version one.

Events do not interrupt

Replies and events travel the same socket, and the design flatly refuses to let an event preempt a running script. `on` registers a callback, but the callback does not fire the instant its frame arrives off the wire. Firing on arrival would mean arbitrary stryke code running in the middle of an unrelated expression — a value half-computed, a map half-built, the stack in a state the author never wrote — and stryke's execution model does not permit that, for the same reason no sane language lets a signal handler resume inside an arithmetic operation.

So events queue. They accumulate in a buffer attached to the subscription and they drain at an explicit pump point, and this is the concrete resolution of the Chapter 12 open question about whether `on` runs on an event loop or against a pumped queue. It is a pumped queue. A callback runs only when the script pumps, which means the script author always knows precisely the set of places a callback can run: the pump points, and nowhere else. Control flow is never violated from outside. The cost is symmetrical and honest — a script that subscribes and then never pumps will never see one event — and it is the right trade, because “my callbacks run where I say” is worth far more to an automation author than “my callbacks run the instant the frame lands.”

The pump in practice

The pump is ordinary stryke, not a special form. A long-running watcher subscribes, then loops, pumping and doing its own work between drains:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val @seen = @{}

$cite->on("itemAdded", fn ($item) {
  push @seen, $item->{doi}
  p "added: ${ $item->{title} }"
})

# drain the subscription queue on a cadence the script controls
loop {
  App::pump() # run any queued callbacks now
  if scalar @seen >= 10 { break } # stop after ten additions
  App::sleep(250)
}
p "collected ${\ scalar @seen} new DOIs"
```

The subscription is live for the whole loop, but a callback body runs only at the `App::pump()` line — never while `push` is mid-flight, never inside the `if`. The script reads top to bottom exactly as written, and the events fold into it at one named seam. That is the entire contract, and it is enough.

What the queue guarantees, and what it does not

The pumped queue makes two promises and withholds a third, and knowing which is which keeps a watcher correct. It promises *order*: events from one subscription drain in the order they arrived off the wire, so a script that watches a sequence of additions sees them in sequence, not shuffled. It promises *completeness up to the pump*: every event that landed before a `pump` call runs during that call, none skipped, so a script that pumps regularly misses nothing that arrived. What it withholds is *timeliness*: an event that lands the instant after a `pump` returns waits, unseen, until the next `pump`, and a script that stops pumping stops seeing events entirely. This is the deliberate trade named earlier — “my callbacks run where I say” bought at the price of “my callbacks do not run the instant the frame lands” — and the two withheld and promised properties are the same coin’s two faces. A watcher that wants tight latency pumps in a tight loop; a watcher that wants to do heavy work between drains pumps less often and accepts the lag. The queue never drops and never reorders; it only ever *delays*, and the delay is under the script’s own hand.

Backpressure at the queue

A subscription’s queue is a buffer, and a buffer that fills is a design question the pump model answers rather than dodges. If an app emits faster than a script pumps — a burst of a hundred `itemAdded` events while the script sleeps between drains — the events accumulate in the subscription’s buffer, and the next `pump` runs all hundred callbacks in a row. The buffer is bounded, so a script that subscribes to a firehose and never pumps does not grow memory without limit; past the bound, the oldest queued events are dropped and the drop is observable, so a watcher that cares can notice it fell behind rather than silently missing events it never knew existed. The design’s stance is that a script driving a busy app is responsible for pumping often enough to keep up, exactly as a script reading a busy socket is responsible for reading it, and the bounded buffer with an observable drop is the honest middle between an unbounded buffer that hides the problem and a zero buffer that loses every event between pumps. Chapter 24 returns to this as the general backpressure story; here it is enough that the queue has a ceiling and says so when it hits it.

Chapter 16 – Orchestrations

The preceding chapters justify the bus one mechanism at a time. This one spends the mechanism. Each recipe below is a whole script that exists only because the bus does — several running apps, one file, one terminal — and each is read for the same thing: the line where a value leaves one app and enters another, which before the bus had no way to be written.

From a selection to a batch of requests

The smallest orchestration is two apps and one loop, and it is the example the [App](#) module chapter opened with, now read for what each line costs. The script reads the current selection out of the reference manager and, for every item, fires a crossref lookup in the API client:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $req  = App::open("zreq")

for val $it (@{ $cite->get("selection") }) {
  val $doi = $it->{doi}
  if $doi {
    $req->call("request.send",
              %{ url => "https://api.crossref.org/works/${ $doi }" })
  }
}
```

`$cite->get("selection")` is a state read that returns a list of maps across the socket; `$it->{doi}` reaches into one of those maps; `$req->call("request.send", ...)` crosses to a second, unrelated app. Before the bus, the selection lived behind a window the script could not read and the request lived behind a palette button that returned nothing. The bus does not add a feature to either app. It makes the value that was always there in `zcite` reachable, and the verb that was always there in `zreq` callable, from the same six lines.

Harvesting the browser's reading list

A richer recipe crosses the browser and the reference manager in the other direction. The browser, [zwire](#), keeps a reading list; the script pulls it, and for every entry that looks like a paper it adds the item to the library and files it in a named collection:

```
#!/usr/bin/env stryke
use App

val $wire = App::open("zwire")
val $cite = App::open("zcite")

val @list = @{$wire->get("readinglist")}
p "reading list holds ${\ scalar @list} entries"

for val $entry (@{ @list }) {
  val $url = $entry->{url}
  val $doi = doi_from_url($url)          # a plain stryke helper, no app involved
  if $doi {
    val $item = $cite->call("item.add", %{ doi => $doi })
    $cite->call("collection.addItem",
      %{ collection => "from-reading-list", item => $item->{id} })
    p "filed: ${ $item->{title} }"
  }
}
```

Two things are worth naming. First, `item.add` returns the item it created — the whole reason the bus exists is that this return value lands in `$item` and the very next line uses `$item->{id}`; a fire-and-forget action could not have done this at all. Second, `doi_from_url` is not an app verb; it is ordinary stryke running in the VM between two calls. The bus does not force every step through an app. It lets app calls and plain computation sit in the same straight-line program, which is what makes a real workflow expressible as one readable file.

The invoice run: three apps, one pipeline

The strongest case for the bus is the one no single app could ever answer, because it lives in the space between three. A calc sheet holds a billing range; the office suite renders it, the PDF engine finishes it, and the mail client sends it:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")
val $pdf = App::open("zpdf")
val $mail = App::open("zemail")

# 1. export the current calc sheet's billing range to a PDF on disk
val $out = $office->call("calc.exportRange",
  %{ sheet => "Invoices", range => "A1:F40", format => "pdf" })

# 2. stamp it in the PDF engine - flatten, watermark, protect
val $doc = $pdf->call("document.open", %{ path => $out->{path} })
$pdf->call("document.watermark", %{ doc => $doc->{id}, text => "PAID" })
$pdf->call("document.flatten", %{ doc => $doc->{id} })
val $final = $pdf->call("document.save", %{ doc => $doc->{id} })

# 3. attach and send it
$mail->call("message.send", %{
  to => "billing@example.com",
  subject => "Invoice - ${ $out->{period} }",
  attach => @{$final->{path} },
})
p "sent ${ $final->{path} }"
```

Read the seams. `$out->{path}` leaves the office suite and becomes the argument that opens a document in

the PDF engine. `$doc->{id}` threads three PDF verbs together. `$final->{path}` leaves the PDF engine and becomes a mail attachment. No file was copied by hand, no app was clicked, no format was guessed — each app did the one thing it is best at and handed its result to the next, and the handoff is a stryke variable. This is the shape the opening sentence of the book promised: one language driving every app by name, and getting the answer back.

Gating on the tunnel

The last recipe uses the bus for control rather than data. Before running any verb that touches a sensitive host, the script brings a VPN up through `ztunnel`, waits for the state query to confirm the link, does its work, and tears the tunnel down — so the sensitive calls provably run only while the tunnel is up:

```
#!/usr/bin/env stryke
use App

val $vpn = App::open("ztunnel")

$vpn->call("connection.up", %{ profile => "work" })

# poll the state query until the link reports connected
val $tries = 0
loop {
  if $vpn->get("status")->{state} == "connected" { break }
  if ($tries += 1) > 20 { die "tunnel never came up" }
  App::sleep(500)
}

# ... sensitive verbs against internal hosts run here, tunnel guaranteed up ...

$vpn->call("connection.down", %{ profile => "work" })
p "tunnel down, run complete"
```

The `get("status")` read in the loop is the bus doing what the old palette never could: reading an app's live state and branching on it. A fire-and-forget `connection.up` followed by a blind `sleep` would only *hope* the tunnel was ready; reading `status` until it says `connected` *knows*. The difference between hoping and knowing is exactly the return path the bus adds, and here it is load-bearing for correctness, not merely convenience.

Watching one app to drive another

The recipes so far are straight-line; this one is event-driven, and it is the first to spend the pump of Chapter 15 in anger. The script watches the browser's downloads and, whenever a PDF finishes, opens it in the reference manager as a new attachment — a running bridge between two apps that reacts rather than polls:

```
#!/usr/bin/env stryke
use App

val $wire = App::open("zwire")
val $cite = App::open("zcite")

$wire->on("downloadComplete", fn ($dl) {
  if $dl->{path} =~ /\.pdf$/ {
    val $item = $cite->call("item.add", %{ file => $dl->{path} })
    p "filed download: ${ $item->{title} }"
  }
})

loop { App::pump(); App::sleep(500) } # drain the download stream forever
```

Two seams matter. The `on` callback fires only inside `App::pump()`, so the `item.add` call into `zcite` never interrupts a half-built expression — the guarantee of Chapter 15 doing real work. And the callback, running in the VM, freely calls a *second* app: the browser's event drove a reference-manager verb, and neither app knew the other existed. The script is the only place the download and the library ever met.

Fan-out with per-item failure tolerance

The invoice recipe assumed every step succeeded; a real batch does not. This recipe sends a request per row of a calc sheet and tolerates the ones that fail, collecting a result table rather than dying on the first bad row — the `try` of Chapter 14 spent across a fan-out:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")
val $req     = App::open("zreq")

val @rows = @{ $office->call("calc.cells", %{ sheet => "Endpoints", range => "A2:A200" }) }
val @ok    = @{}
val @bad   = @{}

for val $row (@{ @rows }) {
  val $url = $row->{value}
  if !$url { next }
  val $res = try { $req->call("request.send", %{ url => $url }) }
               catch ($e) { push @bad, %{ url => $url, why => "${ $e }" }; nil }
  if $res && $res->{status} < 400 { push @ok, $url }
  elsif $res { push @bad, %{ url => $url, why => "status ${ $res->{status} }" } }
}

p "ok: ${\ scalar @ok}   bad: ${\ scalar @bad}"
for val $b (@{ @bad }) { p "  ${ $b->{url} } - ${ $b->{why} }" }
```

The shape is the one every real batch wants and the old palette could not express: read a range out of one app, drive a second app once per row, distinguish the three outcomes — hard failure, HTTP failure, success — and finish with a report instead of a crash. `calc.cells` returns the range as data the script walks; `request.send` returns each response into a variable the script inspects; the `try` turns a broken call into a recorded row rather than a dead script. Every value that crosses between the two apps crosses through the script, and every failure that could have stopped the run becomes a line in the final table.

Reconciling two apps' views of the same data

The last recipe uses the bus to *compare* rather than to move or to gate. The reference manager and the transfer client both know about a set of files — the manager's attachments and the client's remote store — and the script reconciles them, reporting what exists in one place but not the other:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $ftp = App::open("zftp")

# every attachment the library knows about, by filename
val %have = %{}
for val $it (@{ $cite->call("library.search", %{ q => "" }) }) {
  for val $att (@{ $it->{attachments} }) { %have{ $att->{name} } = true }
}

# what the remote store holds
val @remote = @{ $ftp->call("fs.list", %{ path => "/backup/library" }) }

# the two-way diff, computed in the script
for val $f (@{ @remote }) {
  if !%have{ $f->{name} } { p "on server, not in library: ${ $f->{name} }" }
}
p "library holds ${\ scalar keys %have} attachments; server holds ${\ scalar @remote}
  ~ files"
```

No single app could answer “what is on the server that the library does not track,” because the question spans two apps' private views, and before the bus there was no seam where those two views could be read into one program and compared. The recipe is the plainest possible statement of what the bus is *for*: it is not that either app gained a feature, but that a script can now hold both apps' answers at once and reason across them.

Chapter 17 – The Surface as Its Own Documentation

`verbs()` is the manual

The manifest a script reads with `$h->verbs()` is not a second, hand-written document that can drift from the code — it is the same registration the app booted with, stripped of its function bodies. That identity is what lets the surface be its own documentation. A script author who does not remember whether the reference manager calls it `item.add` or `item.create` does not consult a wiki; they ask the running app:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
for val $v (@{ $cite->verbs()->{verbs} }) {
  p "${ $v->{id} }${ join(", ", @{$v->{params} } |map| .{name}) }" -> ${ $v->{returns} }
  ~ }
}
```

The output is a signature list generated from the exact surface the app will answer to a moment later. There is no version skew possible, because there is no second source: the thing that documents the verb and the thing that runs the verb are one registration read two ways, precisely the “one declaration, two readers” property Chapter 3 built and here collects its dividend.

The generated catalog

`GUI_SCRIPT_ACTIONS.md` — the catalog the census in Chapter 8 counts — is the same idea lifted from run time to build time. Rather than a human enumerating verbs into prose that ages, a generator walks each app's source: the engine's `commands::COMMANDS` list, the host's discovery array, and the frontend's `ZGui.automation.register` surface, plus the shared `appShell` verbs every app inherits. The catalog is therefore a projection of the code, and every count in it — the 3,073 across eleven apps, the fifteen shared shell verbs — is a number the generator recomputed, not a number a writer typed. When a verb is added to an app, the catalog grows on the next generation; when one is renamed, the catalog renames. Prose that claims a count is a snapshot that starts drifting the instant it is written; a generated catalog is a measurement that is re-taken on demand. The book leans on the second and is careful, wherever a number appears, to say which one it is.

A REPL and a completion source

Two small tools fall out of the surface being enumerable, and both are worth naming because they cost almost nothing once the manifest exists. The first is a REPL: a loop that reads a verb and an argument map from the terminal, calls it, and prints the decoded reply — a scriptable app's equivalent of a database shell, built from `verbs()` for discovery and `call` for execution, with no per-app code. The second is shell completion. Because `App::list()` enumerates the running apps and `verbs()` enumerates each app's verbs, a completion function can offer the live set of app names after `App::open(` and the live set of verb ids after a handle's `->call(`,

so a script author tab-completes against exactly what is reachable at that moment rather than against a static list that may be stale. Neither tool is a new capability of the bus. Both are the surface's own reflectiveness, spent twice.

The signature is the docstring

There is one more dividend the reflective surface pays, and it closes the loop between the machine reader and the human one. Because `verbs()` yields a verb's parameter names, parameter types, and return type, the same enumeration that lets a REPL offer completions lets a documentation generator emit a signature line no human wrote and no human can let drift:

```
library.search(q: string, collection?: string, limit?: int) -> list<item>
item.add(doi: string) -> item
collection.add_item(collection: string, item: string) -> bool
```

Those lines are not typed into a wiki; they are printed from the manifest the app booted with, and they are exactly what the app will answer to. A documentation page built this way cannot describe a verb the app does not have, cannot omit a verb the app does have, and cannot show a parameter the app dropped, because it is not a description of the surface at all — it is the surface, rendered. The distinction the whole chapter turns on is that between *documenting* a thing and *projecting* it: a document is a second artifact that must be kept in sync, and projection is the same artifact shown a second way, which cannot be out of sync because there is only one of it. Every tool in this chapter — the `verbs()` self-description, the generated catalog, the REPL, the completion source, the signature line — is projection, not documentation, and that is the single property that makes each of them free and none of them able to lie.

Chapter 18 – The Long Arc

Launch-on-demand

The sharpest edge version one leaves unground is that an out-of-process `call` requires its target to already be running; dial a dead app and the script dies at `connect`. This is a deliberate floor, not an oversight — a script that could silently spawn a dozen apps as a side effect of automating them would be a surprising and heavy thing — but it is also the most-requested next move. The shape of the answer is visible from here: a small resolver that, on a failed `connect`, consults a registry mapping bus name to launchable bundle, starts it, and waits for its socket to appear before retrying the dial once. The security model constrains the design more than it complicates it: launch-on-demand must be opt-in per script and must never cross the single-user boundary, because a facility that spawns apps is exactly the facility an attacker would most like to borrow. Version one earns the right to add this later by refusing to fake it now.

The remote temptation, refused

The most natural-seeming extension is the one the design most firmly declines. A newline-JSON protocol over a Unix socket is one `bind` away from being the same protocol over a TCP socket, and a TCP socket is one config flag away from being reachable across a network. The bus will not take that step, and Chapter 10's flat rule — there is no network listener, ever — is written precisely so that the temptation has no toehold. The reasoning is that the bus's whole security posture rests on the socket being a single-user, single-machine artifact at mode `0600`; the moment a frame can arrive from another host, every verb becomes a remote procedure call and the allow-list has to defend against callers it was never designed to distrust. Cross-machine automation is a real and different problem with real and different answers — authentication, transport security, capability scoping — and folding it into the local bus by loosening one bind call would be the kind of shortcut that dissolves a clean model. If cross-machine ever ships, it ships as its own thing with its own threat model, not as this socket with a wider bind.

Typed stubs for the language

Chapter 3 drew the line at the manifest's types being a boundary check and a document rather than a shared type system, and Chapter 18 is where the richer version is named. Because `verbs()` yields a machine-readable signature for every verb — parameter names, parameter types, return type — a generator could emit stryke stub modules per app: `Zcite:library_search($q, $collection)` as a real, autocompleting, arity-checked function rather than a `call("library.search", ...)` string the VM only validates at dispatch. The stubs would be regenerated from the live surface, so they could not drift, and a script that used them would fail at compile time on a misspelled verb or a missing argument instead of at run time on a dead reply. This is the same move the generated catalog makes — code projected from the surface — aimed at the language instead of at documentation. It is deferred, not dismissed: the string-keyed `call` is the honest primitive, and the typed stubs are sugar poured over it once the primitive has proven itself across the full census.

The bus as a platform primitive

Step back far enough and the bus stops looking like a feature of a scripting language and starts looking like a property of the whole suite. Every app in the architecture book was already a thin shell over a pure-Rust core sharing one component library; the bus is the observation that such a suite is, without any further invention, a single programmable surface — that “the shell can call the engine” and “any script can call the engine” are the same sentence with a socket inserted. What the suite gains is not a dozen automation APIs but one, uniform across a PDF editor and an email client and a browser and a VPN, spoken in the same seven calls, carried by the same newline of JSON. A new app joins the platform by registering a surface and serving a socket — two lines — and inherits the whole tooling, the REPL, the completion, the catalog, the orchestration idioms, for free.

That is the arc the book has been tracing from its first sentence. One language should be able to drive every app you own, by name, and get an answer back. The bus is the small set of pieces that makes the sentence true, and the reason it could be small is that the architecture had already done the hard part: it had already turned every app into a set of functions waiting to be called. The bus only had to hand them a caller.

Chapter 19 – Writing a New App Module

The two lines, expanded

Chapter 18 closed on the claim that a new app joins the bus in two lines — register a surface, serve a socket. That claim is true, and this chapter is what those two lines look like when they are the whole of a real morning's work rather than a slogan. The exercise is concrete: take an app that ships today with no automation surface and put it on the bus, from the first `register` call to a green `.stk` script driving it from the shell. Nothing here is new mechanism; it is the mechanism of the first eighteen chapters, walked once in the order a builder actually meets it.

Step one: declare the surface

The surface lives in the frontend, owned by the core, registered once on boot. The first move is to find the app's existing action list — the `setActions` call it already makes to populate its palette picker — and rewrite it as a typed manifest. An app that today does this:

```
// before: id and label, nothing more
appShell.setActions([
  { id: "note.new",   label: "New note" },
  { id: "note.find", label: "Find notes" },
]);
```

becomes this:

```
// after: typed verbs, state, and events on the automation surface
ZGui.automation.register({
  app: "znotes",
  verbs: [
    { id: "note.new",
      label: "New note",
      params: [{ name: "title", type: "string", required: true },
               { name: "body",  type: "string", required: false }],
      returns: "note",
      run: async (a) => znotes.create(a.title, a.body ?? "") },
    { id: "note.find",
      label: "Find notes",
      params: [{ name: "q", type: "string", required: true }],
      returns: "list<note>",
      run: async (a) => znotes.search(a.q) },
  ],
  state: [
    { id: "selection", returns: "list<note>", get: () => znotes.selection() },
  ],
  events: [
    { id: "noteCreated", payload: "note" },
  ],
});
```

Three things happened and none of them was expensive. The two actions kept their ids, so every palette command that already fired `note.new` still works — the id is the compatibility surface, and it did not move.

Each action grew a parameter list, a return type, and a `run` body that calls the engine method the palette action was already bound to. And two entries that did not exist before appeared: a `state` query for the current selection and an `event` the app already had a moment to emit. The picker in the palette, which read the flat list, now reads the typed surface and shows richer entries for free — the “one declaration, two readers” property of Chapter 3, collected the moment the surface is written.

Step two: emit the events

A surface that declares an event but never pushes down it is a promise unkept, so the second move is to find the one place in the engine where the thing happens and add the `emit`. For `noteCreated` that is wherever a note is actually created:

```
async function create(title, body) {
  const note = await znotes.engine.insert({ title, body });
  ZGui.automation.emit("noteCreated", note);
  return note;
}
```

One line, at the point of the event, fire-and-forget. If no script is subscribed the `emit` costs a membership check against an empty list; if one is, the bridge fans the payload out to it. The app does not know or care which, which is exactly why the `emit` can be added unconditionally and left in place forever.

Step three: serve the socket

The frontend now has a surface; the backend now needs a front door. That is the one line in `main`:

```
zgui_bridge::serve("znotes", surface_handle);
```

`serve` computes the socket path — `zgui/znotes.sock` under the runtime directory — creates the `zgui/` parent if missing, unlinks any stale file, binds a fresh `UnixListener` at mode `0600`, and begins accepting connections, each served on its own task by the shared host. The `surface_handle` is the backend’s route into the app: the hybrid `Handler` that tests each incoming verb against the engine’s `COMMANDS` and either runs it in Rust or forwards it to the webview’s dispatch shim. For an app whose verbs all live in the frontend — like this notes app — the handler forwards everything; for an app with a rich `-core`, the engine verbs stay in Rust. Either way the line in `main` is the same, because the routing lives in the shared crate, not in the app.

Step four: verify twice

The app is on the bus; now prove it, on both transports, exactly as the pilots did. In process first, as a palette `stryke` step:

```
val $me = App::here()
val $n = $me->call("note.new", %{ title => "from the palette" })
p "created note ${ $n->{id} }"
```

Then out of process, as a standalone file from the shell, which additionally proves the socket bound and `App::list` sees the app:

```
#!/usr/bin/env stryke
use App

p "running apps: ${\ join(", ", @{ App::list() }) }" # znotes should appear
val $nn = App::open("znotes")
val $n = $nn->call("note.new", %{ title => "from the shell" })
p "created note ${ $n->id }"
for val $hit (@{ $nn->call("note.find", %{ q => "shell" }) }) {
  p "  found: ${ $hit->title }"
}
```

If the palette step prints an id but the shell file cannot find the app, the fault is the socket line — missing, mis-pathed, or a permissions problem. If the shell file finds the app but a call is rejected with `no such verb`, the fault is the surface — a typo in an id, or a verb declared but not registered. The two-transport verification is not ceremony; it splits the failure space in half on the first run, which is the whole reason the rollout insisted on it.

The other shape: when the engine owns the verbs

The notes app in the walkthrough is coreless — every verb lives in the frontend, so the handler forwards everything to the webview and the tutorial never has to think about the routing decision. Most apps in the census are not built that way. They have a `*-core` crate that already implements the domain, already exports a `commands::COMMANDS` list, and already answers those commands for the app's own UI. Putting one of those on the bus is a different and, in a specific sense, easier job, so it is worth walking the difference.

The frontend registration shrinks. An engine-backed app does not re-declare its engine's verbs through `ZGui.automation.register`, because they are already enumerable from `COMMANDS` and already dispatchable in Rust; declaring them a second time in JavaScript would create a second registry to keep in sync with the first, which is exactly the drift the design refuses everywhere else. What the frontend registers is what genuinely lives in the frontend: the state queries, the events, and any verb whose implementation is frontend-side. The engine's verbs join the surface by being in `COMMANDS`, and the hybrid handler picks them up without anyone listing them twice.

That produces a registration that looks lopsided the first time an author writes one — a few frontend verbs, a couple of state entries, an event or two, against an engine surface of hundreds — and the lopsidedness is correct. The engine is the app's substance; the frontend registration is the seam where the parts of the app that are *not* the engine join the same surface.

```
// an engine-backed app: the engine's COMMANDS are already the bulk of the surface,
// so the frontend registers only what the engine does not implement
ZGui.automation.register({
  app: "zexample",
  verbs: [
    { id: "view.focus",
      label: "Focus a pane",
      params: [{ name: "pane", type: "string", required: true }],
      returns: "bool",
      run: (a) => appShell.focusPane(a.pane) },
  ],
  state: [
    { id: "selection", returns: "list<item>", get: () => ui.selection() },
    { id: "activeDoc", returns: "string", get: () => ui.activeDoc() },
  ],
  events: [
    { id: "documentOpened", payload: "string" },
  ],
});
```

The `serve` line in `main` is unchanged — the same one line, the same shared crate — and the only real difference is in the handler, where `is_engine_cmd` now returns true for a large set instead of for none, so most calls take the Rust path and never reach the webview at all. That is the fast path of Chapter 24 arriving for free: an engine-backed app is not merely equally scriptable but structurally faster to script, because the dense part of its surface is the part that skips the frontend.

There is one obligation the coreless walkthrough does not incur, and it is the one to be careful about. Engine commands were written to serve the app's own UI, and their argument shapes and return values were designed for that caller. Exposing them on the bus makes those shapes a public contract subject to Chapter 22's compatibility rules, so it is worth one pass over the list before the first script is written against it — a command whose return value is an internal enum, or whose arguments encode a UI concern, is a command worth reshaping while nothing depends on it. After the first script, it is a rename with a deprecation alias.

The five mistakes a first surface makes

Enough surfaces have been written to see the same five errors recur, and each has a cheap fix at declaration time and an expensive one later.

A mutating verb that returns nothing. The most costly mistake, because it reproduces the exact defect the bus exists to close. A verb that creates a thing and returns `true` forces every script that uses it to go and find what was created, by search or by convention or by guess. `item.add` returning the item it created is what makes the reading-list recipe of Chapter 16 a five-line loop instead of a search-after-write dance. The rule: a verb that creates returns what it created; a verb that changes returns the changed thing or enough of it to be useful; a verb that returns only success has thrown away the information the caller most needed.

Registering the UI's actions instead of the app's capabilities. A first surface frequently mirrors the menu bar, because the menu bar is the list the author has to hand. Some of that is right — those actions are real capabilities — but a menu item exists to be clicked once by a person, and a verb exists to be called a thousand times by a program. Anything whose only content is “open this panel” is a shell verb; anything a script would want to do in a loop deserves parameters and a return value that a menu item never needed.

Declaring `required` where a default exists. A parameter is `required` when the app genuinely cannot proceed without it, not when it is usually supplied. Every `required` parameter is a thing every call site must repeat forever, and the `a.limit ?? 50` idiom of Chapter 3 puts the default where it belongs — in the `run` body, owned by the app. A surface with too many required parameters is verbose at every call site in every script.

Declaring events and never emitting them. The registration is cheap and the `emit` is one line at the point where the thing happens, which is precisely why the second gets forgotten: nothing fails when it is missing. The subscriber sees no error, receives nothing, and concludes that the app is quiet. A declared event that never fires is worse than no event at all, because it advertises a channel that will silently never deliver.

Naming for the workflow rather than the capability. The fifth is the subject of Chapter 42 and is named here only so a first-time author meets it before writing the ids rather than after: names are the durable artifact, and they are the one thing in this tutorial that is expensive to change.

What the tutorial did not require

It is worth naming what did not appear in the four steps, because the absences are the architecture paying off. No new socket protocol was designed; the app inherited the frame codec. No serialization was hand-written; the bridge projects JSON to stryke values and back. No discovery service was registered; the bound socket file *is* the registration. No per-app tooling was built; the REPL, the completion source, and the generated catalog all read the new surface with no code specific to it. The new app did not integrate *with* the bus so much as *appear on* it, because the two lines it wrote were the only two lines that could have been app-specific,

and everything else was already shared. That is the difference between joining a platform and wiring up an integration, and it is the whole return on the architecture book's up-front discipline.

Chapter 20 – The Subscription and Event Model

A subscription's life, frame by frame

Chapter 4 introduced subscription as the streaming half of the one substrate and Chapter 15 governed when its callbacks run; this chapter follows a single subscription through its whole life, because the events surface has corners the earlier chapters passed over. A subscription begins with a `sub` frame and a local registration. When a script writes `$cite->on("itemAdded", $fn)`, two things happen at once: the `App` module stores the closure `$fn` in a table keyed by a fresh subscription id, entirely inside the VM, and it sends `{"t":"sub","id":4,"event":"itemAdded"}` down the wire. The app replies `{"t":"reply","id":4,"ok":true}` to confirm the registration, and from that moment the app's `emit("itemAdded", payload)` pushes an `{"t":"event","sub":4,"event":"itemAdded","payload":{...}}` frame for every occurrence. The `sub` field on each event frame is the subscription id, and it is how the `App` module finds the right closure to run: an event frame arrives, the module looks up `sub` in its table, and queues the payload for that closure's buffer. The closure never traveled; only its id did, and the id is the whole correlation.

Ending a subscription

A subscription that never ends is a leak, so the model has three ways to end one, and a careful watcher knows which it is using. The explicit end is `unsub`: the handle a `->on` call returns can be dropped or closed, which sends `{"t":"unsub","id":4}` and removes the closure from the VM table, after which the app stops pushing that stream. The implicit end is the handle's own death: when the `App::open` handle goes out of scope and its socket closes, every subscription on that connection ends at once, because the app sees the connection drop and tears down all per-connection state — the subscriptions, the reply slots, everything. The third end is the script's: when the `.stk` file finishes, the VM tears down, the socket closes, and the app cleans up on the disconnect exactly as it would for any dropped connection. The design leans on the second and third far more than the first, because a script that subscribes and then exits is the common case, and making disconnect a clean, total teardown means most scripts never write `unsub` at all — they just end, and the app notices.

Filtering at the source versus in the callback

An event stream can be narrowed in two places, and the choice has a cost the script author should make deliberately. The stream can be narrowed *in the callback*, which is what the download-watcher recipe of Chapter 16 did: it subscribed to every `downloadComplete` and tested `$dl->{path} =~ /\.pdf$/` inside the closure, letting the non-PDF events arrive and be ignored. That is simple and it keeps all the filtering logic in the script, but it pays to deliver events the script will discard. The stream can instead be narrowed *at the source*, if the app's event declaration supports a filter argument on `sub` — an app that emits a single `fileChanged` event for a whole tree can accept a path-prefix filter so a subscriber hears only its subtree. Source-side filtering delivers less over the wire but pushes the filter's definition into the app's contract; callback-side filtering delivers everything but keeps the script in sole control. Version one leans on callback-side filtering as the honest default — every event declared is delivered whole, and the script decides what it cares about — because it keeps the

event contract as small as the verb contract: a name and a payload, no per-subscription query language to design and version. Source-side filtering is the same kind of deferred sugar as the typed stubs of Chapter 18: real, useful, and left for an edition that has proven it needs it.

Many subscribers, one emit

An app's `emit` is a single call at the point the thing happens, and it fans out to every subscription registered on that event — across every connection, not merely within one. Two scripts watching `itemAdded`, plus a hook inside the app watching the same event, receive the same payload from the one `emit`, each delivered into its own queue, each drained by its own pump. The app is not aware of the multiplicity; it pushed one payload and the bridge handled the distribution, which is why an app author never writes fan-out logic and never has a reason to care how many watchers exist.

Two consequences of that shape are worth being explicit about, because both are places a reasonable assumption goes wrong.

The payload is delivered to every subscriber, and no subscriber's handling of it affects any other's. A callback that raises, a script that stops pumping, a connection that drops mid-delivery — none of these propagate. There is no acknowledgement flowing back from a subscriber to the app, so a script cannot signal “I have handled this,” cannot suppress delivery to another watcher, and cannot cause the app to retry. The event is a broadcast notification, and every property of it follows from that: no ordering between different subscribers, no coordination, no back-channel.

Nor is there deduplication or coalescing. If an app emits the same event a hundred times in a burst, a subscriber's queue receives a hundred entries and its next pump runs the callback a hundred times, subject only to the bound of the previous chapter's buffer. A watcher that wants to collapse a burst — to react once to “something changed” rather than once per change — collapses it in the script, by setting a flag in the callback and doing the work in the loop body rather than in the callback itself:

```
val $dirty = false
$cite->on("itemAdded", fn ($it) { $dirty = true })      # cheap: just mark

loop {
  App::pump()
  if $dirty {
    $dirty = false
    rebuild_index()      # once per pump, however many events arrived
  }
  App::sleep(1000)
}
```

That idiom is worth having in hand, because it is the correct shape for any watcher whose reaction is expensive and idempotent. The callback stays trivial, which keeps the pump's drain fast; the work happens once per pump regardless of burst size; and the pump's serialization guarantee means the flag needs no protection. It also reframes what the bounded queue's drop policy means for such a watcher: losing some events from a burst that all set the same flag costs nothing at all, which is a reason to prefer this shape wherever the reaction is “recompute” rather than “process this specific item.”

Replay, and why the bus does not offer it

A question every event system eventually faces is whether a late subscriber can catch up on events it missed — replay from a log. The bus deliberately does not offer this, and the refusal is of a piece with its whole posture. An event on this bus is a live notification, not a persisted record: it fires when the thing happens, it is delivered to whoever is subscribed at that moment, and it is gone. A script that needs the historical set does not subscribe and wait; it *calls a verb* that reads the current state — `library.search` for the items that exist, `history.list` for the requests already sent — because the state is the source of truth and the event is only

the poke that says the state changed. Folding replay into the event stream would mean the app persisting an event log, versioning it, bounding it, and answering “since when” queries against it — a whole durable subsystem — to provide what a state-reading verb already provides better. The division of labor is clean and worth stating as a rule: *verbs and state queries answer “what is true now”; events answer “something just changed”; and neither pretends to be the other.* A subscriber that missed an event reads the state; it does not ask the stream to remember.

Chapter 21 – Testing the Bus

The surface is testable without a desktop

A recurring fear about GUI automation is that testing it requires a running desktop, a window server, and a human-shaped click. The bus escapes that fear by construction, because almost everything worth testing about it is data, not pixels. A verb's contract is its parameter validation and its return shape; the transport's contract is that a frame in produces the right frame out; the codec's contract is that a stryke value projects to JSON and back without loss it did not warn about. None of those needs a screen. The bus is testable at three levels — the surface, the transport, and the end-to-end script — and only the third wants a running app, which is why the first two carry most of the test weight.

Level one: the surface, in isolation

A registered surface is a data structure with functions in it, and its validation behavior can be exercised directly. A surface test constructs a manifest, hands it a call frame, and asserts on the reply — no socket, no webview, no app. The cases that matter are the boundary cases the bridge is responsible for: a call missing a required parameter must be rejected before `run` is entered; a call with a parameter of the wrong JSON kind must be rejected with the verb's signature in the message; an unknown verb must produce `no such verb`; a well-formed call must reach `run` with exactly the declared arguments. Each of these is an assertion on a reply frame given a request frame, and each runs in microseconds with no I/O. The `run` bodies themselves are the app's own code, tested by the app's own suite; the surface test's job is the *boundary* — that the bridge admits the callers it should and rejects the ones it should — and that boundary is pure enough to test as a table of frames in and frames out.

Level two: the transport, over a pair of sockets

The transport's job is to carry frames faithfully, and it can be tested against itself with no app at all: bind a `UnixListener`, connect to it, and run the frame codec in both directions. A transport test asserts the properties the wire promises — that a frame written is a frame read, that newline framing survives a value containing escaped newlines, that two requests with distinct ids get replies matched to the right id even when the mock server answers them out of order, that a subscription's event frames carry the right `sub`, and that a mid-call disconnect surfaces as the broken-pipe death rather than a hang. The mock server on the far end is a few lines — accept, read a frame, write a canned reply — and it lets the test drive every failure mode of Chapter 14's taxonomy deterministically: to test the timeout death, the mock accepts the call and simply never replies; to test the broken-pipe death, it accepts and then closes; to test reordering, it holds two calls and replies to the second first. The transport's hardest behaviors are its failure behaviors, and a mock server makes each of them a reproducible test rather than a race that shows up once a month in the wild.

Level three: the end-to-end script, against a running app

The top level is the one that wants a real app, and it is the smallest, because the lower two have already proven the pieces. An end-to-end test launches the app, runs a `.stk` file against it over the socket, and asserts on the script's output — the same two-transport verification the rollout used, now as a repeatable test rather than a one-time check. This level is where a whole-app regression shows: a verb renamed without updating its registration, a socket line dropped from `main`, an event that stopped firing. It is deliberately thin because it is the expensive level — it needs the app up — so it tests the *wiring*, that the app appears on `App::List` and answers a representative call, and leaves the exhaustive per-verb behavior to the surface tests below it. The pyramid is the ordinary one: many fast surface tests, fewer transport tests, a thin cap of end-to-end tests, and the reason it works is that the bus was built in layers that each have a testable boundary, so a bug can be caught at the cheapest level that is responsible for it.

The golden transcript

There is a fourth artifact worth pinning, sitting between the surface tests and the transport tests: the exact byte sequence a known interaction produces on the wire. A golden-transcript test drives a fixed script against a mock server, records every frame in both directions, and compares the recording against a checked-in file. It is the cheapest possible guard on the one contract that must not drift silently — the frame shape of Chapter 28.

What it catches is a specific and easy-to-commit class of change: a field renamed in the codec, a field that started being emitted or stopped, an `id` that is minted differently, a subscription frame that gained a member. None of those necessarily fails a surface test or a transport test, because both of those assert on decoded values and a codec change that is symmetric in both directions passes them happily. It fails only when something on the other end reads the wire, which is exactly the situation the socket relay of Chapter 23 puts a human in and exactly the situation a second implementation of the protocol would be in. Pinning the transcript makes the wire's shape a tested artifact rather than an emergent one.

There is a discipline attached, and it matters more than the technique. When a golden transcript changes, the change is the finding — the correct response is to read the diff and decide whether the contract really moved, not to regenerate the file and commit the new bytes. A golden test that is routinely regenerated has been converted from a guard into a transcription service and asserts nothing at all. The versioning rules of Chapter 22 give the standard to judge a diff against: a frame that gained a field is an additive change old readers ignore, and the diff should show exactly that and nothing else. A field that disappeared or a `t` that changed meaning is a breaking change, and the transcript catching it before release is the entire reason the file exists.

Testing a script's logic without any app

There is a fourth kind of test that is easy to miss: the logic of an orchestration script, tested against a fake bus. Because a handle is reached only through the seven `App` methods, a test can substitute a fake `App` whose `call` and `get` return canned values and whose `on` feeds scripted events, and run the real orchestration against it. The reconciliation recipe of Chapter 16 — the two-way diff between the library and the server — is pure logic once its two `calls` are stubbed: feed it a fixed attachment list and a fixed remote listing, and assert it reports exactly the right difference. This is where the discipline that a handle is the only way to reach an app pays a testing dividend the security chapter did not foresee: a seam narrow enough to be a security boundary is also narrow enough to be a mock point, and the same seven methods that let the bus swap a socket for a callback let a test swap a real app for a fixture.

Chapter 22 – Versioning the Contract

What “the contract” actually is

A contract can only be versioned once it is named, so begin there. The bus's contract has three parts, and they age at three different rates. The *wire* is the frame format — the `t`, `id`, `verb`, `args`, `ok`, `value`, `error`, `sub`, `event`, `payload` fields — and it is the slowest-moving part, because every app and every script speaks it and a change breaks everyone at once. The *surface* is each app's set of verbs, state queries, and events, with their parameter and return types, and it is the fastest-moving part, because it grows every time an app adds a feature. Between them is the *module*, the seven `App` calls, which moves rarely because it is the language-facing API a hundred scripts import. Versioning the bus means versioning these three separately, because a policy that treated them as one thing would either freeze the fast-moving surface to protect the slow-moving wire or destabilize the wire every time a verb was added.

The surface grows without breaking

The surface's compatibility rule is the one a working author leans on daily, and it is the ordinary rule for any versioned API, stated for this one. *Adding* is always safe: a new verb, a new state query, a new event, or a new *optional* parameter on an existing verb breaks no script, because a script that did not use the new thing yesterday does not use it today. *Removing or renaming* is the breaking move: a verb that changes its id, a required parameter that appears where none was, a return type that drops a field a script read — each of these can break a script written against the old shape. The rule falls out of the same “id is the compatibility surface” principle the tutorial leaned on: the id is the promise, so keeping the id stable while widening what sits behind it is the whole discipline of a non-breaking change. An app that must rename a verb keeps the old id as an alias for a version, dispatching both to the same `run`, and drops the alias only after the scripts have moved — the same deprecation courtesy any long-lived API extends, made cheap here because an alias is one more entry in the registration.

The generated catalog is the changelog

The bus has a property most versioned APIs would envy: its surface is enumerable, so its changes are diffable without anyone writing them down. `GUI_SCRIPT_ACTIONS.md` is generated from the live surface, so two generations of it, diffed, *are* the changelog — every verb added shows as an added line, every rename as a delete plus an add, every dropped parameter as a changed signature. A release process that regenerates the catalog and diffs it against the last release sees exactly what changed in the contract, with no human enumerating it and therefore no human forgetting to. This is the “projection, not documentation” property of Chapter 17 spent on versioning: because the catalog is a measurement of the surface rather than a description of it, the measurement's diff is the change, and a breaking change cannot hide from a diff of a projection the way it can hide from prose a writer updated by hand and missed a line of.

The wire's forward door

The wire moves slowest, but it will move, and the frame format left itself one forward door for when it does: the frames are JSON objects keyed by field name, so a new field can be added to a frame and an old reader will

ignore it. A future pipelined mode that wanted a `priority` on a call frame, or a batched mode that wanted a `batch` field, adds the field and every existing reader — which looks up fields by name and ignores the rest — keeps working. The `t` field is the frame's discriminant and the one field that must never change meaning, because it is how a reader decides what kind of frame it holds; a genuinely new frame *kind* gets a new `t` value, which old readers reject as unknown rather than misinterpret. This is the standard tagged-union discipline, and it is why the wire can gain capabilities — pipelining, batching, source-side event filters — by addition rather than by a flag day. The contract was designed additive from version one, which is the same instinct that put the correlation `id` on every frame before anything needed it: build the extensible shape now, so the extension later is an addition and never a redesign.

Chapter 23 – Debugging and Tracing Requests

The wire is legible on purpose

When a call misbehaves, the first advantage the bus hands you is that its traffic is readable text. A newline of JSON per frame means the whole conversation between a script and an app can be watched live with a socket relay:

```
$ socat -v UNIX-CONNECT:$TMPDIR/zgui/zcite.sock -  
> {"t":"call","id":1,"verb":"library.search","args":{"q":"graphene"}}  
< {"t":"reply","id":1,"ok":true,"value":[{"id":42,"title":"..."}]}
```

That transcript is the ground truth. If a script reports an empty result, the transcript says whether the empty came back over the wire — an app-side issue — or whether the call never went out — a script-side issue. If a call hangs, the transcript shows whether the `call` frame was even written, and whether any `reply` came back. The legibility that Chapter 6 chose the newline format for is not an aesthetic preference; it is the single most useful debugging affordance the bus has, because it lets you see exactly what crossed the boundary rather than inferring it from behavior on either side.

Reading the three server-side logs

The app's own side of the conversation is traced too, and it goes where the whole suite's diagnostics go: the app's log, never the terminal. The bridge logs three things at the boundary, each at a level a script author can raise when hunting a problem. It logs *acceptance* — a new connection dialed the socket — which confirms `App::open` reached the app. It logs *dispatch* — a verb name arrived and routed to the engine or the frontend — which is where a `no such verb` shows its cause, because the log names the verb the handler failed to resolve. And it logs *reply* — the frame composed for a call, with its `ok` and, on failure, its error — which is the app-side twin of the `ok:false` the script received. Three log lines per call, correlated by the same `id` the wire uses, so a single request can be followed from the script's `call`, across the `socat` transcript, into the app's acceptance-dispatch-reply trace, and back to the script's returned value or death. The `id` is the thread through the logs exactly as it is the thread through the code.

The four failures, each with a signature

The failure taxonomy of Chapter 14 doubles as a debugging guide, because each of the four failures leaves a different fingerprint across the transcript and the logs. A *rejected* call shows a `call` frame out and an `ok:false` reply back, with the error naming the fault — the app answered, and the answer was no. A *broken* call shows a `call` frame out and then the connection closing with no reply — the transcript simply ends — and the app log shows the connection dropping mid-dispatch. A *timed-out* call shows a `call` frame out, no reply, and the connection *still open* — the distinguishing sign from a broken pipe — with the app log showing dispatch begun but no reply composed, which points at a `run` body that never resolved. An *undialed* call shows nothing on any socket, because `App::open` failed at `connect` before a frame was sent, and the only trace is the script's own death at the open. Learning to read which of the four fingerprints is present turns “the script is broken”

into “the verb accepted the call and never resolved, so the bug is in its `run` body” in the time it takes to glance at the transcript.

A tracing REPL

The REPL of Chapter 17 becomes a debugging instrument when it prints the wire alongside the result. A verbose mode that echoes the `call` frame it sends and the `reply` frame it receives turns the REPL into an interactive probe: type a verb and its arguments, watch the exact frame go out, watch the exact frame come back, and read the decoded value the `App` module produced from it. This is the fastest way to answer the question a misbehaving script raises — “is the app returning what I think it is?” — because it puts a human in the loop at the exact seam where the script’s assumption meets the app’s reality. The REPL costs nothing to build, per Chapter 17, and in verbose mode it costs nothing more to make it the primary tool for understanding a verb’s real behavior before a script is ever written against it.

Chapter 24 – Performance and Backpressure

Where the time goes

A performance claim is only honest if it names where the time actually goes, so account for it call by call. An in-process call — a palette step or a hook driving `App::here()` — is a host callback into the backend, a dispatch into the webview or a direct engine call, and a return; there is no socket, no serialization beyond the JSON the bridge already speaks, and no process boundary crossed. Its cost is dominated by the verb's own work, not by the bus. An out-of-process call adds exactly one Unix-socket round trip and two JSON serializations — request out, reply back — over the in-process cost. A Unix socket round trip on a loopback is measured in microseconds, not milliseconds, because it never touches a network card or a network stack; it is a memory copy between two processes mediated by the kernel. The structural claim of Chapter 11 is grounded here: there is no subprocess per call, no interpreter start-up, no HTTP framing, no TLS handshake — the costs that make per-platform scripting-runtime automation slow are absent by construction, and what remains is a socket write, a socket read, and the work the verb was asked to do.

The engine fast path

The hybrid handler's engine path is where the bus is fastest, and it is worth isolating why. When a call names an engine verb — one in the core's `COMMANDS` — the handler runs it in Rust, in the same `-core` engine the UI calls, and composes the reply without a single frame crossing into JavaScript. For an out-of-process caller that is one socket round trip and one Rust function call; for an in-process caller it is barely more than a Rust function call. The webview path — for frontend-registered verbs — adds the shim evaluation and the reply command, which is more, but it is paid only by verbs that genuinely live in the frontend. The design's performance shape is therefore that the common, hot verbs — the engine's own commands, which are most of the surface on a rich app — take the cheapest path, and only the frontend-specific verbs take the longer one. Speed is not uniform across the surface, and the design made sure the fast path is the one the dense part of the surface travels.

Backpressure on calls: the block is the brake

Backpressure on the call path is handled by the simplest possible mechanism: the synchronous `App::call` blocks until its reply, so a script cannot issue calls faster than the app answers them. There is no unbounded send queue to overflow because there is no send queue at all in today's straight-line mode — one call is outstanding at a time, and the script waits for each before issuing the next. This is backpressure by construction: the slowest component in the loop, whichever it is, sets the pace, and the faster components idle waiting for it rather than piling work on it. A future pipelined mode that let several calls be outstanding would have to add real backpressure — a bound on in-flight calls, a block when the bound is reached — but version one gets the property for free from the blocking call, and it is the right default, because an automation script almost always wants each step's result before deciding the next step anyway.

Backpressure on events: the bounded queue

The event path is where backpressure is a live design question, because events flow app-to-script on the app's schedule, not the script's, and Chapter 15 named the answer: a bounded per-subscription queue. An app emitting faster than a script pumps fills the queue; the next pump drains it; a queue that hits its bound drops its oldest entries and makes the drop observable. The stance is that a script watching a busy stream is responsible for pumping often enough to keep up, and the bounded queue is the honest failure mode when it does not — memory stays bounded, and the script can detect that it fell behind rather than silently missing events. The alternative designs are both worse: an unbounded queue trades a detectable drop for an undetectable memory leak, and a zero-length queue drops every event that lands between pumps, which is most of them. The bounded queue with an observable drop is the middle that keeps the common case correct — a script that pumps promptly loses nothing — while bounding the pathological case rather than hiding it.

Measuring instead of guessing

The preceding sections describe where time goes structurally — a socket round trip here, a webview hop there — and stop deliberately short of numbers. That restraint is the honest position: a latency figure belongs to a machine, a build, and a verb, and a number printed in a book is a measurement of somebody else's laptop on a day that has passed. What the book can supply instead is the method, because the interesting quantities here are all *differences*, and differences are measurable without anyone trusting a published absolute.

Three subtractions isolate the three costs the design cares about.

To isolate **the transport**, call the same verb twice: once in-process through `App:here()` and once out-of-process through `App:open()` against the same running app. The verb's own work is identical in both, so the difference between them is the socket round trip plus the two JSON serializations, and nothing else. That difference is the price of driving an app from the shell rather than from inside it, and it is the number to have before deciding whether a hot loop belongs in a palette step.

To isolate **the webview hop**, call an engine verb and a frontend verb of comparable triviality against the same app, both out-of-process. Both pay the same socket cost; only the second pays the shim evaluation and the reply-command return. The difference is what the hybrid handler's fast path is worth, and it is the number that tells an app author whether a frequently-called frontend verb is worth pushing down into the engine.

To isolate **the bus from the verb**, compare a verb whose body does nearly nothing against one that does real work. The first is essentially a measurement of the bus's floor; the second is a measurement of the verb. If the two are close, the verb is cheap and the bus is the cost; if they are far apart, the verb dominates and no amount of transport tuning will matter. That comparison is the one that most often ends a performance investigation early, because the common answer for a rich engine verb is that the bus is noise against the work.

The instrumentation for all three already exists and needs no new machinery. Chapter 23's three server-side log lines — acceptance, dispatch, reply — are correlated by the same `id` the wire carries, so the app's own log brackets the interval between a call arriving and its reply being composed. The script's side brackets the whole call. Subtract the first from the second and the remainder is the transport plus the module's own encode and decode. No profiler is required to attribute the cost of a bus call, because the correlation `id` that makes concurrency safe also makes the call traceable across both processes, which is a second dividend from a field that was put on every frame for an entirely different reason.

The cost of the bus at rest

A last accounting: what does an app pay for carrying the bus when no script is driving it? The answer is nearly nothing, and the nearly is worth naming. At rest, the bus is a bound `UnixListener` with no connections, a registered surface sitting as an in-memory data structure, and the `emit` calls the app makes at its own event

points, each a membership check against an empty subscription list. There is no polling loop, no background thread spinning, no periodic work; the listener blocks in the kernel waiting for a `connect` that may never come, and a blocked listener costs a file descriptor and nothing else. An app on the bus that no one is automating is, to a first approximation, an app not on the bus at all — which is the property that let the fan-out reach every app without anyone weighing whether a given app was “worth” the runtime cost. The cost at rest is a file descriptor and a data structure, and both are free.

Chapter 25 – A Per-App Orchestration Cookbook

How to read this cookbook

Chapter 16 justified the bus with orchestrations chosen to show a mechanism; this chapter is the working cookbook, organized by the app at the center of each recipe rather than by the mechanism it demonstrates. Each recipe is a whole `.stk` file solving a problem a person actually has, and each is grounded in verbs the census establishes. The verb ids are written in the live snake-case form the catalog reports — `collection.add_item`, `vpn.connect`, `calc.cells` — so a reader can lift a recipe and expect it to dial a real surface. Where a recipe needs a helper that is not an app verb, the helper is ordinary stryke, marked as such, to keep the line between “an app did this” and “the script did this” visible.

zcite: build a bibliography from a search

The reference manager’s most common automation is to turn a query into a formatted bibliography without touching the UI. Search, collect the hits into a named collection, and render the collection as a bibliography in a chosen style:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")

val @hits = @{ $cite->call("library.search", %{ q => "graphene oxide" }) }
$cite->call("collection.add", %{ name => "graphene-review" })
for val $it (@{ @hits }) {
  $cite->call("collection.add_item",
    %{ collection => "graphene-review", item => $it->{id} })
}
val $bib = $cite->call("bib.bibliography",
  %{ collection => "graphene-review", style => "ieee" })
p $bib->{text}
```

The recipe spends four namespaces — `library`, `collection`, `bib` — and every value that threads them is a return the old palette could not have given back: the hits from `library.search`, the item ids into `collection.add_item`, the rendered text out of `bib.bibliography`.

zreq: run a saved collection against a fresh environment

The API client’s daily automation is to run a whole saved collection against a chosen environment and summarize the results — a smoke test expressed as a script:

```
#!/usr/bin/env stryke
use App

val $req = App::open("zreq")

$req->call("environment.activate", %{ name => "staging" })
val @reqs = @{ $req->call("collection.list", %{ collection => "smoke" }) }
val $fail = 0
for val $r (@{ @reqs }) {
  val $res = try { $req->call("request.send", %{ id => $r->{id} }) }
              catch ($e) { $fail += 1; p "ERR ${ $r->{name} }: ${ $e }"; nil }
  if $res && $res->{status} >= 400 {
    $fail += 1
    p "FAIL ${ $r->{name} }: ${ $res->{status} }"
  }
}
p "smoke: ${\ scalar @reqs} requests, ${ $fail } failures"
if $fail > 0 { die "smoke suite failed" }
```

`environment.activate` swaps the variable bindings, `collection.list` enumerates the saved requests, and `request.send` runs each — the client's own features, driven in a loop the client's UI has no way to express, ending in an exit status a CI job can read.

zoffice: a calc sheet as a data source

The office engine's most useful automation is to treat a spreadsheet as the source of truth for a batch, reading a range as data and driving other apps from it. Here a sheet of email addresses drives a mail merge through the mail client:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")
val $mail = App::open("zemail")

val @rows = @{ $office->call("calc.cells",
  %{ sheet => "Recipients", range => "A2:B200" }) }

for val $row (@{ @rows }) {
  val $name = $row->{a}
  val $email = $row->{b}
  if !$email { next }
  $mail->call("message.send", %{
    to      => $email,
    subject => "Your invitation",
    body    => "Hello ${ $name }, ...",
  })
}
p "sent ${\ scalar @rows} invitations"
```

`calc.cells` returns the range as rows of data; the script walks them and fires `message.send` per row. The spreadsheet is the loop's data and the mail client is its effect, and the only place the two meet is the four lines of stryke between them.

zftp: mirror a local tree to a remote store with a dry run first

The transfer client's automation earns its keep on repeatable syncs, and the recipe that matters is the one that plans before it acts — compute the diff, show it, and only then transfer:

```
#!/usr/bin/env stryke
use App

val $ftp = App::open("zftp")

# plan: what would change, without changing anything
val @plan = @{ $ftp->call("sync.plan",
    %{ local => "~/site", remote => "/var/www/site" }) }

p "planned changes:"
for val $step (@{ @plan }) { p "  ${ $step->{op} } ${ $step->{path} }" }

# act: enqueue each planned transfer and watch them complete
for val $step (@{ @plan }) {
    if $step->{op} == "upload" {
        $ftp->call("transfer.add",
            %{ src => $step->{local}, dst => $step->{remote} })
    }
}
p "enqueued ${\ scalar @plan} transfers"
```

`sync.plan` returns the diff as data the script prints and then consumes, and `transfer.add` enqueues each upload. The plan-then-act shape is only expressible because `sync.plan` hands its answer back rather than acting on it — the return path turning a blind sync into a reviewable one.

ztunnel and zthrottle: a guarded, throttled backup

The last recipe composes three concerns — a tunnel, a throttle, and a transfer — into one guarded operation: bring up the VPN, throttle the backup process so it does not saturate the link, run the transfer, then restore both:

```

#!/usr/bin/env stryke
use App

val $vpn = App::open("ztunnel")
val $thr = App::open("zthrottle")
val $ftp = App::open("zftp")

# 1. bring the tunnel up and confirm it
$vpn->call("vpn.connect", %{ profile => "backup" })
loop {
  if $vpn->call("vpn.status", %{})->{state} == "connected" { break }
  App::sleep(500)
}

# 2. cap the transfer process's network so the link stays usable
val @procs = @{$thr->call("proc.snapshot", %{}) }
for val $p (@{ @procs }) {
  if $p->{name} =~ /zftp/ {
    $thr->call("proc.throttle", %{ pid => $p->{pid}, net_kbps => 4000 })
  }
}

# 3. run the backup, guaranteed tunneled and throttled
$ftp->call("transfer.add", %{ src => "~/archive", dst => "/backup/archive" })

# 4. restore
$vpn->call("vpn.disconnect", %{ profile => "backup" })
p "backup complete, tunnel down"

```

Three apps, three concerns, one file. `vpn.status` gates the work on a confirmed link; `proc.snapshot` and `proc.throttle` cap the transfer's bandwidth by pid; `transfer.add` does the copy. No single app could have expressed "back up this tree, but only while the tunnel is up and only at a bandwidth that leaves the link usable," because the sentence spans three apps' surfaces — and the script is the one place all three answer at once.

Chapter 26 – The Verb Reference

What this reference is, and how it relates to the catalog

This chapter is the printed companion to the generated catalog `GUI_SCRIPT_ACTIONS.md`. The catalog is the exhaustive, machine-generated census — read from each running app's live surface and re-taken on demand — and it is always the authority on the exact set of verbs a given build exposes. This reference is the human's map of that territory: a namespace-by-namespace walk of each app's surface, naming the namespaces and a representative set of the verbs in each, so a reader can learn the *shape* of an app's surface and know what to call `verbs()` for. Two conventions hold throughout. First, verb ids are written in the live snake-case form the catalog reports; where a chapter example earlier in the book used a lightly-cleaned illustrative name, the live id is the one here. Second, this reference names representative verbs per namespace, not every verb — the exhaustive list is the catalog's job, and the census in Chapter 8 gives the exact per- app counts. Read this to learn an app's vocabulary; read the catalog to enumerate it.

The shared shell: `appshell.*`

Every app inherits the shared shell verbs, because every app is a shell over `zgui-core`. A script that knows only these can open a terminal, browse files, flip a theme, or summon the palette in any app in the suite before learning a single app-specific verb.

Verb	Effect
<code>appshell.terminal.open / .close</code>	show or hide the embedded terminal
<code>appshell.files.open / .close</code>	show or hide the file browser
<code>appshell.palette</code>	summon the command palette
<code>appshell.settings</code>	open the settings panel
<code>appshell.shortcuts</code>	open the keyboard-shortcuts panel
<code>appshell.theme.light / .dark</code>	switch the color theme
<code>appshell.crt.on / .off / .toggle</code>	the CRT scanline effect
<code>appshell.neon</code>	the neon-glow visual toggle

zcite — the reference manager

Namespace	Representative verbs	Purpose
<code>library.*</code>	<code>library.search</code>	search and enumerate holdings

Namespace	Representative verbs	Purpose
<code>collection.*</code>	<code>collection.add</code> , <code>collection.rename</code> , <code>collection.add_item</code> , <code>collection.remove_item</code> , <code>collection.tree</code> , <code>collection.list</code> , <code>collection.diff</code> , <code>collection.merge</code>	folders and their membership
<code>item.*</code>	<code>item.add</code> , <code>item.add_attachment</code>	item lifecycle; <code>item.add</code> returns the created item
<code>bib.*</code>	<code>bib.bibliography</code> , <code>bib.citation</code> , <code>bib.csl</code> , <code>bib.styles</code> , <code>bib.cite_key</code> , <code>bib.sort_key</code> , <code>bib.disambiguate</code>	rendering citations and bibliographies
<code>export.*</code>	<code>export.bibtex</code> , <code>export.biblatex</code> , <code>export.ris</code> , <code>export.csl_json</code> , <code>export.marcxml</code> , <code>export.mods</code> , <code>export.endnote_xml</code> , <code>export.zotero_rdf</code> , <code>export.markdown</code> , <code>export.csv</code>	outbound format conversion
<code>import.*</code>	<code>import.bibtex</code> , <code>import.crossref_json</code> , <code>import.datacite_json</code> , <code>import.pubmed_xml</code> , <code>import.ris</code> , <code>import.endnote_xml</code> , <code>import.zotero_rdf</code>	inbound format conversion
<code>attachment.*</code>	<code>attachment.index</code> , <code>attachment.snapshot</code> , <code>attachment.store_file</code> , <code>attachment.sniff</code>	attachment handling

zreq – the API client

Namespace	Representative verbs	Purpose
<code>request.*</code>	<code>request.send</code> , <code>request.add</code> , <code>request.update</code> , <code>request.get</code> , <code>request.remove</code> , <code>request.fuzz</code>	the request lifecycle; <code>request.send</code> returns the response
<code>collection.*</code>	<code>collection.list</code>	grouping saved requests

Namespace	Representative verbs	Purpose
<code>environment.*</code>	<code>environment.activate</code> , <code>environment.add</code> , <code>environment.list</code> , <code>environment.remove</code>	variable-binding environments
<code>graphql.*</code>	<code>graphql.introspection_</code> <code>query</code> , <code>graphql.parse</code> , <code>graphql.schema.parse</code>	GraphQL introspection and parsing
<code>grpc.*</code>	<code>grpc.call</code>	gRPC method invocation
<code>codegen.*</code>	<code>codegen.generate</code>	turn a request into client code
<code>history.*</code>	<code>history.list</code> , <code>history.get</code> , <code>history.replay</code> , <code>history.clear</code>	past traffic

zoffice – the office engine

Namespace	Representative verbs	Purpose
<code>calc.*</code>	<code>calc.open</code> , <code>calc.cells</code> , <code>calc.edit_cell</code> , <code>calc.eval</code> , <code>calc.find</code> , <code>calc.replace</code> , <code>calc.sort</code> , <code>calc.named_</code> <code>ranges</code> , <code>calc.pivot_tables</code> , <code>calc.charts</code> , <code>calc.validations</code> , <code>calc.conditional_formats</code> , <code>calc.csv</code> , <code>calc.html</code> , <code>calc.markdown</code> , <code>calc.pdf</code> , <code>calc.render</code>	the spreadsheet engine
<code>writer.*</code>	<code>writer.open</code> , <code>writer.find</code> , <code>writer.replace</code> , <code>writer.render</code>	the prose engine
<code>impress.*</code>	<code>impress.charts_render</code> , <code>impress.charts_detail</code> , <code>impress.graphic_objects</code> , <code>impress.find</code>	the presentation engine
<code>draw.*</code>	<code>draw.connectors</code> , <code>draw.svg</code> , <code>draw.render</code> , <code>draw.find</code> , <code>draw.markdown</code>	vector documents

zemail – the mail client

Namespace	Representative verbs	Purpose
<code>account.*</code>	<code>account.add</code> , <code>account.autoconfig</code> , <code>account.list</code> , <code>account.update</code> , <code>account.remove</code>	account setup
<code>folder.*</code>	<code>folder.create</code> , <code>folder.delete</code> , <code>folder.rename</code> , <code>folder.list</code> , <code>folder.inbox_load</code> , <code>folder.digest</code>	mailbox folders
<code>message.*</code>	<code>message.send</code>	composition and sending
<code>contact.*</code>	<code>contact.add</code> , <code>contact.list</code> , <code>contact.merge</code> , <code>contact.import_vcard</code> , <code>contact.export_vcard</code> , <code>contact.find_duplicates</code>	the address book
<code>filter.*</code>	<code>filter.add</code> , <code>filter.apply</code> , <code>filter.run</code> , <code>filter.to_sieve</code> , <code>filter.match</code>	message filters, exportable to Sieve
<code>imap.*</code>	<code>imap.folders</code> , <code>imap.idle</code> , <code>imap.build_search</code> , <code>imap.parse_envelope</code>	protocol-level IMAP

The census line for `zemail` also names the PGP and S/MIME security surface; those verbs sign, encrypt, and verify messages, and a script drives them the same way it drives any other verb — the reference notes their presence without enumerating them here, deferring the exhaustive set to the catalog.

zwire — the browser

`zwire` is the app whose discovery and dispatch differ, per Chapter 7: its host lists a `SURFACE_VERBS` array for discovery but accepts any `browser.<verb>`, because the Chromium HUD's own executor runs it. The namespaces below are the shape of that surface.

Namespace	Representative area	Purpose
<code>browser.tab.*</code>	tab open, close, list, select	tab management
<code>browser.window.*</code>	window new, close, list	window management
<code>browser.tabgroup.*</code>	group create, add, list	tab groups
<code>browser.download.*</code>	download list, and the <code>downloadComplete</code> event	downloads
<code>browser.readingList.*</code>	reading-list read and add (the <code>readingList</code> state query)	reading list
<code>browser.power.*</code>	power and lifecycle controls	power

zftp – the transfer client

Namespace	Representative verbs	Purpose
<code>transfer.*</code>	<code>transfer.add</code> , <code>transfer.cancel</code> , <code>transfer.resume_check</code> , <code>transfer.list</code> , <code>transfer.clear</code> , <code>transfer.estimate</code> , <code>transfer.segments</code> , <code>transfer.multipart_plan</code>	the transfer queue
<code>sftp.*</code>	<code>sftp.build_init</code> , <code>sftp.parse_attrs</code> , <code>sftp.negotiate</code>	the SFTP wire
<code>ftp.*</code>	<code>ftp.parse_pasv</code> , <code>ftp.build_port</code> , <code>ftp.parse_mlsx</code>	the FTP wire
<code>s3.*</code>	<code>s3.list_objects</code> , <code>s3.presign</code> , <code>s3.sign</code> , <code>s3.complete_multipart</code>	S3 and cloud stores
<code>fs.*</code>	<code>fs.list</code> , <code>fs.mkdir</code> , <code>fs.delete</code> , <code>fs.rename</code> , <code>fs.chmod</code> , <code>fs.peek</code>	the remote filesystem
<code>sync.*</code>	<code>sync.compare</code> , <code>sync.plan</code> , <code>sync.resolve</code> , <code>sync.symlink_policy</code>	mirroring and reconciliation
<code>bookmark.*</code>	<code>bookmark.add</code> , <code>bookmark.list</code> , <code>bookmark.import</code>	saved connections

ztunnel – the VPN client

Namespace	Representative verbs	Purpose
<code>vpn.*</code>	<code>vpn.connect</code> , <code>vpn.disconnect</code> , <code>vpn.status</code> , <code>vpn.reconnect_schedule</code> , <code>vpn.killswitch_plan</code> , <code>vpn.autoconnect</code> , <code>vpn.logs</code> , <code>vpn.connections</code>	connection control and state
<code>wg.*</code>	<code>wg.genkey</code> , <code>wg.pubkey</code> , <code>wg.handshake_model</code> , <code>wg.allowed_ips</code>	WireGuard
<code>openvpn.*</code>	<code>openvpn.push_reply</code> , <code>openvpn.inline_blocks</code> , <code>openvpn.static_key_parse</code>	OpenVPN

Namespace	Representative verbs	Purpose
<code>dns.*</code>	<code>dns.leak_check</code> , <code>dns.split_horizon</code> , <code>dns.query_wire</code>	DNS handling
<code>profile.*</code>	<code>profile.save</code> , <code>profile.get</code> , <code>profile.export</code> , <code>profile.import</code> , <code>profile.encrypt</code> , <code>profile.decrypt</code>	profile management
<code>split.*</code>	<code>split.evaluate</code> , <code>split.route_plan</code> , <code>split.evaluate_app</code>	split-tunnel routing

zthrottle – system monitor and throttle

Namespace	Representative verbs	Purpose
<code>proc.*</code>	<code>proc.snapshot</code> , <code>proc.tree</code> , <code>proc.kill</code> , <code>proc.detail</code> , <code>proc.diff</code> , <code>proc.files</code> , <code>proc.history</code>	process inspection and control
<code>bench.*</code>	<code>bench.cpu</code> , <code>bench.mem</code> , <code>bench.disk</code> , <code>bench.net</code> , <code>bench.all</code> , <code>bench.contention</code>	benchmarks
<code>ioreg.*</code>	<code>ioreg.find</code> , <code>ioreg.node</code> , <code>ioreg.watch</code>	the I/O registry
<code>drives.*</code>	<code>drives.list</code> , <code>drives.set_target</code>	storage devices

zgo – the launcher

The launcher's surface is the thinnest of the census's named apps: system commands — sleep, lock, empty trash, and the rest — under `sys.*`, alongside the launcher's own workflow and snippet machinery. It is the app a script reaches for when it wants the *machine* to do something rather than an app: a run that ends by locking the screen, or a maintenance script that empties the trash, calls `zgo` for the one system verb it needs and no more.

The coreless four

`zphoto`, `zstation`, `ztranslator`, and `zcontainer` carry the bus but register no rich frontend catalog, so their enumerable surface is the shared `appshell.*` verbs plus their Tauri backend commands. A script drives them through the shared shell verbs above and whatever backend commands each exposes; the catalog reports exactly that surface for them and, honestly, no more. The reference names them here so the map is complete — every app that answers on a socket appears — while marking clearly that their scriptable depth is the shell's, not a domain catalog's, until they register one.

Using the reference

The reference is a map, not the territory. When a script needs the exact, current verb of an app, it does not consult this chapter or the catalog — it asks the running app with `verbs()`, and reads the answer the app will itself honor a moment later. This chapter teaches the shape so a reader knows which namespace to look in and roughly what to expect there; `verbs()` gives the precise, live, drift-free truth. That is the whole relationship the book has argued for from Chapter 3 onward: a surface that documents itself needs no second document to be correct, and this reference is offered in that spirit — a human's overview of a machine-readable truth, useful precisely because it never pretends to be the authority the live surface is.

Chapter 27 — State Queries in Depth

The third operation, reconsidered

Of the four operations, state reading is the one a new script author most often uses wrongly, not because it is hard but because its boundary with `call` is subtle. A verb *does* something and returns the result of doing it; a state query *reads* something that is already true and returns it without changing anything. The distinction is not enforced by the type system — both travel the same request/response substrate, both come back as a value — but it is real in the app's own contract, and honoring it is what keeps a script's behavior predictable. `library.search` is a verb: it runs a query and returns matches, and calling it twice runs the query twice. `selection` is a state query: it reports which items are selected right now, and reading it twice reads the same live fact twice with no side effect. When a script wants “what is true,” it reads state; when it wants “make this happen and tell me what happened,” it calls a verb. Getting this right is the difference between a script that observes an app and a script that keeps nudging it.

When to read state and when to call a verb

The decision rule is short and worth stating flatly. Read state when the answer is a property of the app's current condition that the app maintains anyway — the selection, the active collection, the connection status, the active environment. Call a verb when the answer requires the app to *compute* or *act* — to run a search, send a request, render a bibliography, open a document. The tell is idempotence: reading a state query twice in a row must return the same answer if nothing else changed, and if a script finds itself relying on a “read” that alters what the next read returns, it has mistaken a verb for a state query and the app's contract will eventually surprise it. State is the app's condition; verbs are the app's behavior; and a script that keeps the two straight reads cleanly and acts deliberately, never confusing the two.

The get path, mechanically

`get` runs the identical substrate as `call`, aimed at the state table instead of the verb table, which Chapter 4 established and this section makes concrete. A `get` frame — `{"t": "get", "id": 2, "state": "selection"}` — arrives, the handler resolves `selection` against the registered `state` entries rather than the verbs, runs the entry's `get` closure, and composes the same `{"t": "reply", "id": 2, "ok": true, "value": ...}` a verb would. There is no separate machinery: the state table is a second dictionary consulted by a second frame type, and everything downstream — correlation by `id`, projection of the result to a stryke value, the failure model — is shared with `call`. A state query that does not exist is rejected exactly as an unknown verb is, with `no such state: <name>`, because the allow-list discipline is the same. The economy is the point: an app that wired the one substrate got state reads for the cost of declaring a `state` array, and a script that learned `call` already knows `get` because they differ only in which table the name resolves against.

Reading state in a loop: the poll that knows

The tunnel recipe of Chapter 16 showed the archetypal use of state — a poll that reads a live condition until it reaches the desired value — and the pattern generalizes to every “wait until” a script needs. The shape is

always the same: read a state query, test it, and either proceed or sleep and read again, bounded by a try count so a condition that never arrives becomes a death rather than a spin.

```
# wait until an app's state reaches a condition, or die trying
fn wait_for($h, $state, $pred, $max) {
  val $tries = 0
  loop {
    if $pred($h->get($state)) { return true }
    if ($tries += 1) > $max { die "{$state} never satisfied" }
    App::sleep(250)
  }
}

val $vpn = App::open("ztunnel")
$vpn->call("vpn.connect", %{ profile => "work" })
wait_for($vpn, "status", fn ($s) { $s->{state} == "connected" }, 40)
```

The helper is ordinary stryke — no app knows it exists — and it turns any state query into a synchronization primitive. This is the return path of the bus doing what the old palette could not: a fire-and-forget action followed by a blind sleep only hopes; a poll against a live state query knows, and the difference is load-bearing wherever a later step depends on an earlier one having taken effect.

Designing a state query, from the app's side

The chapter so far reads state from the script's side. The app author's side has its own rules, and they are stricter than the verb author's, because a state query is subject to a usage pattern verbs are not: it gets polled. The `wait_for` helper above will call a `get` several times a second for as long as a condition takes to arrive, and every state entry should be written on the assumption that some script, somewhere, will do exactly that.

Three constraints follow. A state entry's `get` must be **cheap**, because it will be called in a loop. It must be **free of side effects**, because a script's model of it is that reading changes nothing, and a `get` that mutates will produce the class of bug where observing the system perturbs it and nobody suspects the observer. And it must be **available synchronously enough to answer promptly**, because a poll loop that blocks for a second per read is not a poll loop. Anything that fails one of the three is a verb, not a state query, and the surface should say so. If the answer requires reaching the network, opening a file, or running a query, declare it as a verb and let the caller pay a call's worth of ceremony for a call's worth of work.

The clean test the app author can apply is the same idempotence tell the script author uses, read in the other direction: if calling this thing twice in a row would be wasteful, surprising, or wrong, it is not a state query.

Granularity: one map or many scalars

The second design question is how much to put in one entry, and the shipped surfaces answer it both ways, which turns out to be consistent rather than confused. The tunnel exposes `status` as a map a script reads a `state` field out of; the reference manager exposes `activeCollection` as a bare value and `selection` as a list. The rule that reconciles them is about *change correlation*: fields that change together belong in one query, and fields that change independently belong in separate ones.

A connection's condition is one fact with several facets — whether it is up, since when, by which profile — and they all change at the same instant, when the link comes up or goes down. Splitting them into three queries would let a script read them at three different moments and assemble an inconsistent picture, and it would cost three round trips to learn one thing. One map, read atomically in one reply, is both cheaper and more correct.

The active collection and the current selection change for entirely unrelated reasons — one when the user navigates, the other when the user clicks — so bundling them would mean every poll of one carrying the

other's payload for nothing, and would leave a script unable to express which fact it was actually watching. Two queries.

The failure mode on each side is worth naming so an author can recognize it. Too coarse produces a single `state` entry returning the app's whole world, which every script must then index into, which makes the entry's payload a schema nobody declared and which grows without bound as the app does. Too fine produces a script issuing six `get` calls to assemble one coherent view, and getting a view that was never simultaneously true. The correlation rule threads between them and is the only rule needed, because it derives from what the script is actually trying to do: read a consistent answer to one question, once.

State versus event: two answers to “did it change?”

State queries and events are the two ways a script learns that something changed, and they answer the question from opposite ends. A state query answers by *pull*: the script asks, at a moment of its choosing, what is true now, and gets the current answer. An event answers by *push*: the app tells the script the instant a thing happens, and the script receives it at the next pump. A poll built on a state query is simple and needs no subscription, but it only sees the condition at the moments it happens to read, and it costs a call per read. An event sees every occurrence and costs nothing between them, but it needs a subscription and a pump. The choice is the ordinary polling-versus-eventing tradeoff, and the bus offers both because both are right sometimes: poll a state query when the script controls the cadence and wants a specific condition, subscribe to an event when the script must not miss an occurrence and does not know when it will come. The tunnel poll wants the former; the download watcher of Chapter 16 wants the latter; and a script author who knows which question they are asking reaches for the right one.

Chapter 28 – The Frame Reference

The wire, exhaustively

Chapter 6 showed the wire by example; this chapter is the authoritative catalog of every frame, because a reader debugging a transcript or implementing a client needs the complete set in one place. Every frame is a single line of JSON, an object with a `t` field naming its kind, and there are exactly seven kinds — five a client sends and two a server sends. Nothing else ever appears on the wire.

Client-to-server frames

A client — the `App` module, in-process or over a socket — sends five frame kinds: `call`, `get`, `verbs`, `sub`, and `unsub`. Each carries an `id` the client mints to correlate the reply.

`call` invokes a verb.

```
{"t":"call","id":1,"verb":"library.search","args":{"q":"graphene"}}
```

`verb` is the verb id; `args` is an object of the typed arguments. The server validates `args` against the verb's declared `params` before running it and replies with one `reply` frame carrying the value or an error.

`get` reads a state query.

```
{"t":"get","id":2,"state":"selection"}
```

`state` is the state-query id; there are no arguments. The server runs the query's `get` and replies with one `reply` frame.

`verbs` requests the introspection manifest.

```
{"t":"verbs","id":3}
```

No further fields. The server replies with one `reply` frame whose `value` is the stripped manifest — the verbs, state, and events with their types but no function bodies.

`sub` opens a subscription.

```
{"t":"sub","id":4,"event":"itemAdded"}
```

`event` is the event id. The server replies with one `reply` frame confirming the registration, after which it pushes an `event` frame per occurrence, each tagged with this frame's `id` as its `sub`. A matching `unsub` — `{"t":"unsub","id":4}` — ends the subscription and stops the pushes.

Server-to-client frames

A server — an app's bridge — sends two frame kinds.

`reply` answers exactly one request, correlated by `id`.

```
{"t":"reply","id":1,"ok":true,"value":[{"id":42,"title":"..."}]}
{"t":"reply","id":9,"ok":false,"error":"no such verb: foo.bar"}
```

`ok` is the fork: `true` carries a `value`, `false` carries an `error` string. Every `call`, `get`, `verbs`, and `sub` produces exactly one `reply`, and the `id` is the request's `id`.

`event` delivers one occurrence of a subscribed event.

```
{"t":"event","sub":4,"event":"itemAdded","payload":{"id":43,"title":"..."}}
```

`sub` is the subscription's `id` — the `id` from the `sub` frame that opened it — and it is how the `App` module routes the payload to the right closure. `payload` is the value the app passed to `emit`. Event frames carry no `id` of their own, because they are not replies to a request; they are pushes correlated to a subscription.

There is no third distinct server frame in version one; the taxonomy is `reply` and `event`, and the two failures the wire cannot phrase — a broken pipe and a timeout — produce no frame at all, which is precisely why the `App` module manufactures a death for each rather than waiting for a frame that will never come.

A full transcript, annotated

Reading the frames in sequence is the fastest way to internalize the protocol. Here is a whole session — enumerate, read, call, subscribe, receive, fail — as it crosses one socket:

```
-> {"t":"verbs","id":1}
<- {"t":"reply","id":1,"ok":true,"value":{"verbs":[...],"state":[...],"events":[...]}}

-> {"t":"get","id":2,"state":"activeCollection"}
<- {"t":"reply","id":2,"ok":true,"value":"graphene-review"}

-> {"t":"call","id":3,"verb":"item.add","args":{"doi":"10.1/x"}}
<- {"t":"reply","id":3,"ok":true,"value":{"id":51,"title":"..."}}

-> {"t":"sub","id":4,"event":"itemAdded"}
<- {"t":"reply","id":4,"ok":true}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"id":52,"title":"..."}}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"id":53,"title":"..."}}

-> {"t":"call","id":5,"verb":"item.frobnicate","args":{}}
<- {"t":"reply","id":5,"ok":false,"error":"no such verb: item.frobnicate"}

-> {"t":"unsub","id":4}
```

Every reply's `id` matches its request's; every event's `sub` matches the subscription's `id`; the one failure is an `ok:false` naming the fault. The whole protocol is visible in those fourteen lines, and a client that handles exactly these frames handles the entire bus.

Why the frame set is closed

It is deliberate that there are seven frame kinds and not eight, and the discipline is the same one the versioning chapter named. A new *capability* — pipelining, batching, a source-side event filter — is added by putting a new *field* on an existing frame, which old readers ignore, not by inventing a new frame kind, which old readers would reject. So the frame set is closed by intent: the wire grows by widening frames, never by multiplying them, and a reader written against these seven kinds keeps working across every additive change the contract makes. The closed set is not a limitation; it is the guarantee that lets the protocol evolve without a flag day.

What a reader does with a frame it cannot use

A closed frame set and an additive growth rule together imply a set of decoder behaviours, and stating them makes the guarantee checkable rather than merely asserted. There are three distinct things a reader can meet

that it does not understand, and they get three different answers.

An **unknown field** on a known frame is ignored. This is the whole mechanism by which the wire grows: a **call** frame that gained a field an older reader has never heard of is still a valid **call** frame with a **verb** and an **args**, and the older reader dispatches it exactly as it would have before the field existed. The tolerance mirrors the argument validator's tolerance of Chapter 3 — extra arguments are ignored so that the surface can grow additively — and it is the same reasoning applied one layer down. Rejecting unknown fields would convert every widening into a flag day, which is precisely the outcome the closed frame set exists to avoid.

An **unknown t** is a different matter, because **t** is the one field the transport does route on. A frame whose **t** names nothing the reader knows cannot be dispatched to anything, and the reader's only honest options are to ignore it or to refuse it. Ignoring it is right on the server side for a frame that carries an **id**, since the peer is waiting and an **ok:false** reply naming the unknown kind tells them far more than silence; refusing is right when there is no way to phrase a refusal. Either way the connection survives, and this is the property that matters: an unknown frame kind must not be a fatal protocol error, because a peer that tore down the connection on one would make the addition of any future frame kind catastrophic in exactly the way that adding a field is not.

A **malformed line** — one that is not valid JSON at all — is the third case and the only one where the reader has genuinely lost its place. Under newline framing, though, losing the place is cheap: the frame boundary is the newline, so a corrupt line is one discardable unit and the next line begins a fresh frame with no resynchronisation to perform. This is the quiet dividend of the framing choice Chapter 6 defended on legibility grounds. A length-prefixed protocol that mis-parses a header has lost the stream and must either resynchronise heuristically or drop the connection; a newline-framed protocol reads to the next newline and carries on. The debugging affordance and the error-recovery property come from the same decision.

The rule underneath all three is that the decoder should be **strict about what it acts on and lenient about what it ignores**. It acts only on fields it knows, in frames whose **t** it knows, which is what keeps behaviour predictable. It tolerates everything else, which is what keeps a new build and an old one able to hold a conversation. A protocol that inverted either half — acting on fields it does not fully understand, or rejecting frames it merely does not need — would be a protocol that could not be extended without coordinating every reader on the machine at once.

Chapter 29 – The Error Reference

Every way a call can fail, with its cause and its fix

The failure model of Chapter 14 gave the taxonomy; this chapter is the desk reference — every error a script can meet, what produced it, and what to do about it. The organizing fact is that all of them arrive as a death at the call site, so the message is the primary diagnostic, and knowing the message forms turns a stack trace into a diagnosis.

no such verb: <name>

Cause. A `call` named a verb in neither the engine's `COMMANDS` nor the frontend's registered set. Almost always a typo in the verb id, a verb that was renamed, or a verb the script assumed exists but the app never registered.

Fix. Read `verbs()` and check membership. The verb id you want is in the manifest or it is not; if it is not, either the name is wrong or the app does not expose it. A script that probes for optional verbs tests `verbs()` rather than calling and catching, because the manifest is the authority on what exists.

no such state: <name>

Cause. A `get` named a state query the app did not register. The state tables are smaller than the verb tables, so a mistaken state name is a common early error.

Fix. Read `verbs()->{state}` for the state queries the app actually exposes. If the value you want is not a state query, it may be the return of a verb instead — the selection is state, but the results of a search are a verb's return, and confusing the two produces this error.

missing required argument: <name> and the signature

Cause. A `call` omitted a parameter the verb declared `required`. The bridge rejects it before `run` is entered, and the error names the missing parameter and prints the verb's full signature.

Fix. Supply the argument. The signature in the message shows every parameter, its type, and which are required, so the fix is usually visible in the error itself. This is why the bridge validates against the manifest rather than letting the call reach `run` with a hole in it: the failure names exactly what is missing at the boundary, not three statements later inside the app.

<app> is not running (the undialed call)

Cause. `App::open("<app>")` failed at `connect` because no socket file for that app exists — the app is not up, or crashed without cleanup and has not relaunched. Per Chapter 10's second rule, the target must be running, and version one has no launch-on-demand.

Fix. Start the app, or gate the script on `App::list()` containing the name before opening it. A script that must tolerate an absent target checks the list first rather than opening and catching, because the list is the authority on what is dialable right now.

<app> vanished (the broken pipe)

Cause. An out-of-process target exited in the middle of a call; the pipe broke and no `reply` is possible. Per Chapter 14, the `App` module holds the connection and retries once on a broken pipe, so this death means the retry also failed — the target is genuinely down, not merely restarting.

Fix. This is usually a real condition to report, not a bug to work around: the app the script was driving died. A long-running script that expects targets to restart wraps the vulnerable calls in a `try` and re-opens the handle on failure, but a script that meets an unexpected `vanished` has learned something true about the world and should say so.

<verb> timed out

Cause. The target accepted the call and never replied under the deadline — a `run` body awaiting a promise that never settles, an engine call that wedged. Distinguished from a broken pipe by the connection staying open, per Chapter 23's fingerprints.

Fix. This points at the verb's implementation, not the script: a verb that reliably times out is a verb with a bug in its `run`. The script's only recourse is to catch the timeout and treat the verb as unavailable; the real fix is in the app, and the timeout death is the bus refusing to let the app's hang become the script's hang.

Sending a non-serializable value

Cause. A `call` tried to pass a stryke value with no JSON image — a closure, a handle, or a container holding one — as an argument. Per Chapter 13, these are refused at the boundary rather than serialized into something broken.

Fix. Do not pass behavior or connections as data. A callback belongs in `on`, which never serializes it; a handle stays VM-local and the script reads through it rather than handing it across. If a map carries a closure by accident, the fix is to send only its data fields.

The one error a script should never see

There is a failure the design works to make impossible rather than to report: a silent wrong answer. Every error above is loud — a death at the call site, with a message naming the fault — precisely so that the failure mode a script author most fears, a call that appears to succeed but returns the wrong thing, does not arise from the bus itself. The bus can deliver an app's own bug faithfully — a verb whose `run` returns the wrong value is the app's fault, surfaced when a script reads a field that is not there — but the bus's own layers do not silently corrupt: a call either returns the value the app produced, or it dies with a named cause. That guarantee is the sum of the whole failure model, and it is why the error reference is a list of loud, diagnosable deaths and not a warning about quiet wrong answers.

Chapter 30 – Cookbook II: Reporting, Maintenance, Migration

The scripts that run on a schedule

The first cookbook showed the bus moving values between apps in a single interactive run; this one collects the scripts that earn their place by running unattended — a report generated nightly, a maintenance pass run weekly, a migration run once. They lean harder on the failure model, because a script no one is watching must fail loudly and finish cleanly, and they lean on state queries and events to know when their work is done.

A cross-app status report

The most useful unattended script is often the plainest: gather one fact from each running app and print a report. It uses only `App::list` and `verbs/get`, touches every app generically, and never assumes a verb an app might not have:

```
#!/usr/bin/env stryke
use App

p "=== bus status ${\ now() } ==="
for val $name (@{ App::list() }) {
  val $h = App::open($name)
  val $surface = $h->verbs()
  val $nverbs = scalar @{$surface->{verbs} }
  val $nstate = scalar @{$surface->{state} }
  val $nevents = scalar @{$surface->{events} }
  p "${ $name }: ${ $nverbs } verbs, ${ $nstate } state, ${ $nevents } events"
}
```

The report is generic by construction: it drives every app through the shared seven calls and asks each only for its own manifest, so it works against an app it has never seen and grows automatically as apps join the bus. This is the census of Chapter 8 rendered live — not read from a generated file but asked of the running apps, one `verbs()` at a time.

A maintenance pass with per-app tolerance

A weekly maintenance run touches several apps and must not let one app's failure abort the rest. The pattern is a loop over the apps, each unit of work wrapped so a failure is recorded and the pass continues:

```
#!/usr/bin/env stryke
use App

val @report = @{}

fn step($label, $fn) {
  val $r = try { $fn(); "ok" } catch ($e) { "FAIL: ${ $e }" }
  push @report, "${ $label }": "${ $r }"
}

# each app's maintenance, isolated from the others
if grep(App::list(), "zemail") {
  val $mail = App::open("zemail")
  step("mail: expunge trash", fn () { $mail->call("folder.digest", %{ folder => "Trash"
  ~ }) })
}
if grep(App::list(), "zcite") {
  val $cite = App::open("zcite")
  step("cite: lint collections", fn () { $cite->call("collection.lint", %{ }) })
}
if grep(App::list(), "zthrottle") {
  val $thr = App::open("zthrottle")
  step("throttle: snapshot procs", fn () { $thr->call("proc.snapshot", %{ }) })
}

p "=== maintenance report ==="
for val $line (@{ @report }) { p $line }
if grep(@report, qr/FAIL/) { die "maintenance had failures" }
```

Three properties make this fit for unattended use. It checks `App::list()` before opening each app, so a target that happens to be down is skipped rather than aborting the run at `connect`. It isolates each unit of work in `step`, so one app's failure is a line in the report, not the end of the pass. And it ends in a non-zero exit when any step failed, so a scheduler that runs it sees the failure without a human reading the log. The bus's failure model — a death by default, a `try` where the author means to continue — is exactly the substrate this pattern needs.

A one-time migration between two apps

A migration run moves data from one app's world into another's, once, and it wants a dry run before it commits. Here reading-list entries from the browser become library items in the reference manager, with a preview pass that changes nothing:

```
#!/usr/bin/env stryke
use App

val $dry = App::env("DRY_RUN") == "1"
val $wire = App::open("zwire")
val $cite = App::open("zcite")

val @list = @{$wire->get("readinglist")}
p "reading list holds ${\ scalar @list} entries (dry_run=${ $dry })"

val $added = 0
for val $entry (@{ @list }) {
  val $doi = doi_from_url($entry->{url}) # ordinary stryke helper
  if !$doi { next }
  if $dry {
    p "would add: ${ $doi }"
  } else {
    val $item = $cite->call("item.add", %{ doi => $doi })
    $cite->call("collection.add_item",
      %{ collection => "migrated", item => $item->{id} })
    $added += 1
    p "added: ${ $item->{title} }"
  }
}
p "${ $dry ? "would add" : "added" } ${ $dry ? scalar @list : $added } items"
```

The dry-run switch is the migration's safety: run it once with `DRY_RUN=1` to see exactly what it would do, read the preview, then run it for real. The bus makes the preview cheap because reading is free of effect — `get("readinglist")` observes without changing, and `doi_from_url` is pure — so the dry run is the real run with its two mutating calls skipped. This is the plan-then-act shape of the transfer recipe generalized: a migration that can be inspected before it commits is a migration a person can trust to run once and be done.

A watcher that turns events into a log file

The last recipe is a long-running watcher that subscribes to several apps at once and writes every event to a single log — the bus as an observability tap across the suite:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $wire = App::open("zwire")

$cite->on("itemAdded", fn ($it) { logline("zcite", "itemAdded", $it->{title}) })
$wire->on("downloadComplete", fn ($dl) { logline("zwire", "downloadComplete", $dl->{path})
  ~ })

fn logline($app, $event, $detail) {
  append_file("~/bus-events.log", "${\ now() }\t${ $app }\t${ $event }\t${ $detail }\n")
}

# drain both streams forever, one pump serving every subscription
loop { App::pump(); App::sleep(500) }
```

One script, two apps, two event streams, one log. `App::pump()` drains every subscription the script holds, regardless of which app or which handle it belongs to, so a single pump loop serves the whole tap. The watcher never polls an app and never asks a verb; it sits on the events the apps already emit and folds them into a file, and it is the plainest demonstration that the bus's event half is not an afterthought but a peer of its call half — a

script can be built entirely out of watching, with not a single `call` in it, and still do real and useful work.

The through-line of both cookbooks

Across every recipe in both cookbooks, one property recurs and is worth naming as the book closes its practical half: the script is always the only place two apps' values meet. The bus never lets one app hold a connection to another; it makes each app a server to the user's scripts and nothing more, so a value that leaves `zcite` and enters `zreq` passes through a stryke variable in between, in a program the user wrote and can read top to bottom. That is the shape the opening sentence promised and the whole book has been building toward — one language driving every app by name, each app's answer landing in a variable the next call can use — and the cookbooks are simply that sentence, spent as many ways as a working day has problems.

Chapter 31 – Implementing the Handler

The shared crate, from the app's side

The tutorial of Chapter 19 gave the app author two lines and moved on; this chapter is for the reader who wants to know what those two lines rest on — the `zgui-bridge` crate, seen from inside. The crate exists so that no app writes socket code, frame code, or correlation code, and understanding its three pieces — `serve`, the `Bridge`, and the `Handler` — is understanding everything an app author does not have to build. The design goal is that the crate owns everything generic and the app owns only what is specific to it, which turns out to be a single decision per call: does this verb belong to the engine or to the frontend?

`serve`: the front door

`serve(app_name, handler)` is the whole of the socket lifecycle, and it does in order the boring, correct things Chapter 6 named. It computes the socket path from `app_name` and the runtime directory; it creates the `zgui/` parent if missing; it unlinks any stale socket file so a crashed predecessor's corpse does not block the bind; it binds a fresh `UnixListener` at mode `0600`; and it enters an accept loop, spawning a task per accepted connection. Each task owns one connection's state — its correlation table, its subscriptions — and reads frames in a loop, dispatching each through the `Bridge` to the app's `Handler` and writing back the reply. When the connection drops, the task tears down that connection's state and exits, which is the clean-disconnect teardown Chapter 20 leans on. The app author writes none of this; they pass a name and a handler and get a served, mode-restricted, crash-recoverable socket with per-connection isolation.

The `Handler`: one decision, two arms

The `Handler` is the one interface the app implements, and its core is the routing decision of Chapter 7 expressed as code the author fills in. Given a verb name and arguments, the handler answers whether the engine owns the verb and, if so, calls the engine directly; otherwise it forwards to the frontend. In shape:

```
impl Handler for MyApp {
    fn is_engine_cmd(&self, cmd: &str) -> bool {
        core::commands::COMMANDS.contains(&cmd)
    }

    async fn call_engine(&self, cmd: &str, args: Json) -> Reply {
        // route straight to the -core engine, in Rust, no webview
        core::dispatch(cmd, args).into()
    }

    async fn call_frontend(&self, cmd: &str, args: Json) -> Reply {
        // evaluate the dispatch shim in the webview and await the reply channel
        self.webview_dispatch(cmd, args).await
    }
}
```

The `Bridge` calls `is_engine_cmd` once per incoming `call` and dispatches to whichever arm it selects, composing the `reply` frame from the returned `Reply` and correlating it by the request's `id`. The author's

whole responsibility is those three methods, and two of them are nearly mechanical: `is_engine_cmd` is a set membership test against a list the core already exports, and `call_engine` is a call into the engine the UI already uses. Only `call_frontend` carries real machinery, and even that is the reply-channel pattern the crate provides a helper for.

The reply channel, from the backend

`call_frontend` is where the correlation of Chapter 4 lives in the app's own code, and it is worth seeing whole because it is the one genuinely asynchronous seam an app author touches. The backend cannot call into the webview and get a return value synchronously — the webview runs its verb asynchronously and reports back through a separate Tauri command — so the backend registers a one-shot channel keyed by the request `id`, evaluates the dispatch shim passing that `id`, and awaits the channel:

```
async fn webview_dispatch(&self, cmd: &str, args: Json) -> Reply {
    let id = self.next_id();
    let (tx, rx) = oneshot::channel();
    self.pending.lock().insert(id, tx);           // stash the reply slot
    self.window.eval(&format!(
        "window.__zguiBridgeDispatch({id}, 'verb', {cmd:?}, {args:?})");
    rx.await.unwrap_or(Reply::err("frontend dispatch dropped"))
}

// the Tauri command the shim calls back into, fulfilling the slot
#[tauri::command]
fn zgui_bridge_reply(state: State<MyApp>, id: u64, ok: bool, value: Json) {
    if let Some(tx) = state.pending.lock().remove(&id) {
        let _ = tx.send(if ok { Reply::ok(value) } else { Reply::err_json(value) });
    }
}
```

The `id` is the thread through the whole loop, exactly as Chapter 4 described from the outside: minted at the call, carried into the webview by the shim, carried back by `zgui_bridge_reply`, and used to find the one waiting slot. The `oneshot` channel is the concrete form of “a per-request channel that fulfills exactly once” — send once, receive once, and a dropped sender (the shim failed or the webview reloaded) surfaces as a clean error rather than a hang, because `rx.await` on a dropped `tx` resolves to the fallback, not to nothing.

What must never block, and why

The one operational rule an app author has to hold while implementing a `Handler` is about blocking, and it is short: a verb's execution must not stall anything except the caller waiting on it. The crate's structure is what makes that achievable, and understanding the structure is what keeps an author from accidentally defeating it.

`serve` spawns a task per accepted connection, so one script's long-running call cannot delay another script's calls — the two are on different tasks reading different streams. That isolation is free and it holds regardless of what the handler does. What it does *not* protect is the app itself. A `call_engine` implementation that blocks the executor rather than awaiting — a synchronous, long-running computation on an `async` task — can stall unrelated work in the process, and the symptom will not look like a bus problem. It will look like the app becoming unresponsive while a script runs, which is the single most alarming way for automation to fail, because it implicates the app in the eyes of the person sitting in front of it.

The same rule bounds the frontend path from the other direction. `call_frontend` awaits a reply channel that the webview fulfills, and the webview is also the thing rendering the user interface. A frontend verb whose `run` body does heavy synchronous work occupies the same place the UI is drawn from, so a script hammering such a verb will visibly slow the app. This is the strongest practical argument for the hybrid handler's engine path being the one the dense part of a surface travels: a verb in Rust competes with the app's other Rust work, which the runtime can schedule around, while a verb in the frontend competes with the app's ability to repaint.

Two implications for a handler author follow, and both are cheap to honor. First, real work belongs behind an `await` on something that yields — the engine's own async surface, a spawned blocking task, whatever the app already uses for its UI-initiated work — because a verb is not a special caller and should reuse the app's existing discipline for not freezing itself. The bus adds a second caller to code that already had one; if the UI's path through that code is non-blocking, the bus's should be the same path. Second, a verb that genuinely takes a long time is fine, and no part of the design objects to it — the caller blocks, which is the backpressure of Chapter 24 working as intended, and Chapter 14's deadline is there for the case where “long” becomes “never.” What is not fine is a long verb that takes the app down with it, and the difference between the two is entirely in how the handler waits.

What the crate refuses to own

A shared crate earns trust by the boundaries it declines to cross, and `zgui-bridge` draws one deliberately: it owns the transport and the correlation, and it owns nothing about what a verb *means*. It does not know that `library.search` searches a library or that `vpn.connect` raises a tunnel; it moves the frame to the handler and the reply back, and the semantics live entirely in the app. This is why one crate serves eleven apps of wildly different domains without a line of domain code in it: the crate is a pipe with a correlation table, and the meaning is always the app's. The same discipline that made the `App` module seven calls with the app's verbs on top makes the host one crate with the app's handler on top — a thin, generic substrate under a domain surface, top to bottom, which is the whole architecture in one sentence.

Chapter 32 – Cookbook III: Interactive and Composed Sessions

The bus at the prompt

The first two cookbooks wrote whole files; this one works at the prompt, because a large part of the bus's daily value is ad hoc — a question asked of a running app in the middle of doing something else. The REPL of Chapter 17 is the vehicle, and its idioms are worth collecting, because an interactive session composes the same seven calls into something closer to a conversation than a program.

Discovering an app you have never scripted

The first thing to do with an unfamiliar app is ask it what it can do, and the REPL turns that into a two-line habit. Open the app, print its verbs filtered to the namespace you suspect, and read the signatures:

```
val $h = App::open("zftp")
for val $v (@{ $h->verbs()->{verbs} }) {
  if $v->{id} =~ /^sync\.\/ {
    p "${ $v->{id} }({ $v->{params} } |map| "${ .{name} }": "${ .{type} }")
  }
}
```

The output is the exact `sync.*` surface the app will answer to, with types, generated from the live manifest. A script author who does this before writing a line of automation starts from what the app actually exposes rather than from a guess, and the habit is cheap enough — two lines at a REPL — to make it the default first move against any app.

Building a call incrementally

An interactive session lets a call be built up rather than written whole, testing each piece against the real app. Read the state first, inspect it, then use it:

```
val $req = App::open("zreq")
val $env = $req->get("activeEnvironment")      # what is active?
p $env                                         # inspect it
val @hist = @{ $req->call("history.list", %{ limit => 5 }) }
for val $h (@{ @hist }) { p "${ $h->{method} } ${ $h->{url} } -> ${ $h->{status} }" }
val $again = $req->call("history.replay", %{ id => @hist->[0]->{id} }) # re-run the last
p "replayed: ${ $again->{status} }"
```

Each line is checked against the live app before the next is written, so a session grows into a script by accretion — the parts that worked, kept; the parts that did not, discarded — and what remains is a correct automation assembled from verified pieces rather than a hopeful one written all at once and debugged after.

Composing two apps at the prompt

Even cross-app composition works interactively, because a handle held at the REPL is a live connection for as long as the session lasts. Open two apps, read from one, and feed the other, watching each result:

```
val $cite = App::open("zcite")
val $req  = App::open("zreq")
val @sel  = @{$cite->get("selection")} # what is selected in zcite?
p "selected ${\ scalar @sel} items"
val $it   = @sel->[0]
val $res  = $req->call("request.send",
  %{ url => "https://api.crossref.org/works/${ $it->{doi} }" })
p "crossref says: ${ $res->{status} }"
```

This is the invoice pipeline of Chapter 16 discovered rather than written — each seam tried by hand, each value inspected before it is passed on — and it is the way most real orchestrations are found: interactively, one verified handoff at a time, then saved to a `.stk` file once the whole chain works. The REPL is not a lesser mode of the bus; it is where scripts are born.

A composable helper library

Interactive discovery eventually crystallizes into helpers a script author reuses, and the bus's uniformity makes them app-agnostic. A handful of small functions, written once against the seven calls, work against every app:

```
# does an app expose this verb?
fn has_verb($h, $id) {
  for val $v (@{ $h->verbs()->{verbs} }) { if $v->{id} == $id { return true } }
  false
}

# call a verb only if it exists, else nil - tolerate surface differences across builds
fn call_if($h, $id, %args) {
  if has_verb($h, $id) { $h->call($id, %args) } else { nil }
}

# every running app that exposes a given verb
fn apps_with($id) {
  grep(map(App::list(), fn ($n) { %{ name => $n, has => has_verb(App::open($n), $id) }
    ~  })),
    fn ($x) { $x->{has} })
}
```

None of these knows a single app-specific fact; they are built from `verbs`, `call`, and `list`, and they work against `zcite`, `zreq`, or an app written next year, because the uniformity of the surface is what lets a helper be generic. This is the platform property of Chapter 18 spent at the script level: because every app answers the same seven calls, a utility written against those calls is a utility for the whole suite, and a script author's personal library of helpers grows to cover apps it was written before they existed.

Chapter 33 – The Bus and the Connector Family

App among its siblings

The `App` module did not arrive alone; it joined a family, and seeing it beside its siblings is the last thing needed to place it. `stryke`'s connector packages — `stryke-gui`, `stryke-aws`, and now `stryke-app` — share one shape: a `cdylib` with C-ABI exports and a `stryke`-facing module on top, a foreign capability made native to the language. The family resemblance is not incidental. It is the discipline that let `stryke-app` be built quickly and be trusted immediately, because it is the third instance of a pattern already proven twice, and a reader who knows one connector knows how to read the others.

`stryke-gui`: the physical sibling

`stryke-gui` is the connector the `App` module is most usefully contrasted with, because the two are complementary rather than competing, and the contrast sharpens what the bus is. `stryke-gui` is *physical*: it moves the mouse, presses keys, drives the screen — it automates a GUI the way a human's hands would, from the outside, against pixels and input events. `App` is *semantic*: it calls named verbs and reads typed state — it automates an app the way the app's own code would, from the inside, against functions and values. Where `stryke-gui` clicks the button labeled "Search," `App` calls `library.search` and gets the rows. The two share the same connector shape and the same performance discipline — `stryke-gui` holds one persistent input handle rather than spawning a tool per keystroke, exactly as the bus holds one socket rather than forking per call — but they reach an app at two different layers. `stryke-gui` works against any app, including ones that never joined the bus, at the cost of driving a surface meant for human eyes; `App` works only against apps on the bus, but reaches their actual functions and returns. A script that needs both — drive an un-bussed app's UI with `stryke-gui`, then hand its result to a bussed app with `App` — uses them together, two connectors of one family solving two halves of one automation.

`stryke-aws`: the remote sibling

`stryke-aws` is the sibling that shows the family is not about locality. It reaches a remote cloud API — authenticated, over the network, against someone else's service — and it wears the same connector shape as the two local siblings. Placing it beside `App` draws the boundary the bus draws for itself in Chapter 18: `stryke-aws` is a remote connector by design, with the network's threat model — credentials, transport security, capability scoping — baked into what it is, while `App` is a local connector by design, refusing the network precisely because it has no threat model for it and does not pretend to. The family holds both a remote member and a local member without contradiction, because each was built for its own boundary, and the bus's refusal to grow a network bind is not a limitation of the connector pattern but a choice about which member of the family `App` is. Cross-machine automation, if it ever ships, would be a different connector with `stryke-aws`'s posture, not `App` with a wider bind.

The precedent from `zterminal`

One more relative sits outside the connector family but inside the same discipline: `zterminal`, which the transport chapter named as the socket precedent. `zterminal` speaks a raw wire protocol — the `tmux` `imsg` protocol — straight to a socket, with no subprocess between, and it did so before the bus existed. Its relevance is that it proved the exact discipline the bus's out-of-process transport needed: that a MenkeTechnologies app can hold a persistent socket and speak a framed protocol over it natively, without shelling out to a tool per operation. The bus did not invent that capability; it pointed a second protocol — newline JSON instead of `imsg` frames — down a pipe of the same kind `zterminal` had already dug. Naming the precedent is the book's consistent method: nothing here is invented where something could be reused, and the socket transport is reused discipline, not new discipline.

Where the family points

Seen whole, the connector family sketches the shape of stryke's reach: physical automation of any GUI (`stryke-gui`), semantic automation of the bussed suite (`stryke-app`), remote automation of cloud services (`stryke-aws`), each a `cdylib-plus-module` built to one shape, each native to the language, none a subprocess spawned per call. The bus is the family member that turns the author's own suite into a programmable surface, and it is the one that could only exist because the suite was built the way it was — thin shells over shared cores, each already exposing a command surface to its own frontend. The other connectors reach outward, to foreign GUIs and foreign clouds; `App` reaches inward, to the functions the author already wrote, and hands them a caller. That inward reach is the whole book: not a new capability bolted onto the apps, but the recognition that a suite of shared-core apps is already a single programmable surface, wanting only the small set of pieces — a typed manifest, a correlated bridge, a stryke module, a newline of JSON — that let one language call every function in it, by name, and get the answer back.

Chapter 34 — Patterns and Anti-Patterns

The idioms worth naming

A capability accretes idioms — recurring shapes that experienced users reach for without thinking — and a book that has shown the bus in use can now name them, because naming an idiom is what lets a reader recognize it in their own code and reach for it deliberately. The patterns below are all present in the recipes of the earlier chapters; this chapter lifts them out and states each as a shape, with the anti-pattern it replaces, so that a script author meets each once as a named thing rather than rediscovering it under deadline.

The star topology

The foundational pattern is the one Chapter 13 proved falls out of the handle rule: the script is always the hub, and every app is a spoke. A script that drives four apps holds four handles and passes every cross-app value through its own variables; no app ever holds a connection to another. The shape is a star with the script at the center, and it is worth adopting deliberately rather than merely inheriting, because it is what makes an orchestration readable — every place two apps' data meets is a line in the script, visible, inspectable, and yours. The anti-pattern it replaces is the one the handle rule makes impossible: handing app A a handle to app B and stepping back, hoping the two apps talk among themselves. They cannot, and the reason is a feature — a script you can read top to bottom is worth more than a mesh you cannot.

Poll-until for state, subscribe-for-events

The second pattern is the poll-until of Chapter 27: read a state query in a bounded loop until it reaches the desired value, then proceed. Its partner is the subscribe-and-pump of Chapter 15: register a callback and drain it at a controlled cadence. The pattern is choosing correctly between them — poll a state query when the script controls the cadence and waits for one condition; subscribe to an event when the script must not miss an occurrence. The anti-pattern both replace is the *blind sleep*: firing an action and then `App::sleep`-ing a guessed interval in the hope that the action finished. A blind sleep is wrong in both directions at once — too short and the next step runs before the action took effect, too long and the script wastes the difference on every run — and the return path the bus adds exists precisely to replace hoping with knowing. Any `App::sleep` not inside a poll loop bounded by a state read is a smell worth a second look.

Try-per-item for batches

The third pattern is the try-per-item of Chapters 14 and 16: in a loop over a batch, wrap each unit of work so a failure becomes a recorded outcome rather than the end of the run. The shape is a `try/catch` inside the loop body, pushing successes to one list and failures to another, and finishing with a report. The anti-pattern it replaces is the bare loop that dies on the first bad item, losing every item after it and telling the user nothing about which one failed or why. A batch script that must process every item it can and report the rest is the common case for unattended work, and the try-per-item is its shape. The discipline is to make the default — a death that stops the script — opt out of, per item, only where the author genuinely means to continue.

Discover-before-call

The fourth pattern is the discover-before-call of Chapters 17 and 32: ask `verbs()` what an app exposes before writing calls against it, and, in code that must tolerate surface differences across builds, test membership before calling. The `call_if` helper of Chapter 32 is this pattern crystallized — call a verb only if it exists, else nil. The anti-pattern it replaces is *fishing by calling*: probing for a verb's existence by calling it and catching the `no such verb` death. Fishing works but it is backwards — it uses the failure path as a control-flow branch, which is exactly the confusion Chapter 14 warned against — and `verbs()` gives the answer directly, cheaply, and without a death to catch. Probe the manifest; do not fish the dispatcher.

The pure-helper seam

The fifth pattern is subtle and Chapter 16 named it in passing: ordinary stryke computation sits in the same straight-line program as app calls, and keeping the two visibly separate is what makes a script readable. `doi_from_url` is a pure helper; `item.add` is an app call; and a script reads cleanly when the reader can see at a glance which lines touch an app and which are local computation. The anti-pattern is pushing pure computation into app verbs that do not need to be app verbs — asking an app to parse a string a stryke function could parse — which spends a socket round trip on work the VM could do for free and blurs the line between what the script computes and what the apps provide. The rule is: compute in stryke what stryke can compute, and call an app only for what only the app knows.

The single-pump loop

The sixth pattern is the single-pump loop of Chapter 30's event tap: one `App::pump()` in one loop drains every subscription the script holds, across every app, because pump is global to the VM's subscriptions, not per-handle. A watcher on three apps' events needs one pump loop, not three. The anti-pattern is a per-handle pump — imagining that each app's events need their own drain — which does not match the model and, worse, tempts a script into nested loops that starve one stream while draining another. One pump, one loop, all subscriptions: the model is simpler than the mistake.

The idempotent read

The last pattern is the discipline of Chapter 27: treat state reads as idempotent observations and verbs as effects, and never rely on a read to change what the next read returns. The anti-pattern is mistaking a verb for a state query — calling a search verb where a `get` of live state was meant, and then being surprised that “reading” it twice ran the query twice. The tell is the same one Chapter 27 gave: if reading a thing twice in a row does not return the same answer with nothing else changed, it is a verb, not a state query, and the script's model of the app is off by exactly that confusion.

Chapter 35 — Cookbook IV: Domain Recipes

Recipes that live inside one app

The earlier cookbooks favored cross-app orchestrations, because those are the recipes only the bus makes possible; this last cookbook returns to single-app automations, because much daily work is one app driven hard, and the bus makes that expressible too. Each recipe here spends one app's surface deeply rather than crossing between apps, and each is grounded in the namespaces the verb reference names.

Bulk-tagging a search result in the reference manager

A librarian's common chore is to run a search and apply a treatment to every hit — here, adding each result to a collection and re-exporting the whole collection in a citation style, all against `zcite` alone:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $query = "machine learning AND materials"

val @hits = @{ $cite->call("library.search", %{ q => $query }) }
p "search matched ${\ scalar @hits} items"

$cite->call("collection.add", %{ name => "ml-materials" })
for val $it (@{ @hits }) {
  $cite->call("collection.add_item",
    %{ collection => "ml-materials", item => $it->{id} })
}

# render the whole collection as a bibliography and write it to disk
val $bib = $cite->call("bib.bibliography",
  %{ collection => "ml-materials", style => "apa" })
write_file("~/ml-materials.txt", $bib->{text}) # ordinary stryke I/O
p "wrote ${\ scalar @hits} references"
```

The recipe touches three namespaces and never leaves the app, but it is still automation the old palette could not express, because each step consumes the previous step's return: the hits feed the collection, and the collection feeds the bibliography.

Regression-testing an API collection with assertions

A developer's chore is to run a saved request collection and assert on the responses — turning `zreq` into a test runner driven from a script rather than clicked through by hand:

```
#!/usr/bin/env stryke
use App

val $req = App::open("zreq")
$req->call("environment.activate", %{ name => "ci" })

val @cases = @{
  %{ id => "health", want => 200 },
  %{ id => "auth", want => 401 },
  %{ id => "listing", want => 200 },
}
val $fail = 0
for val $c (@{ @cases }) {
  val $res = try { $req->call("request.send", %{ id => $c->{id} }) }
    catch ($e) { $fail += 1; p "ERR ${ $c->{id} }: ${ $e }"; nil }
  if $res && $res->{status} != $c->{want} {
    $fail += 1
    p "FAIL ${ $c->{id} }: want ${ $c->{want} }, got ${ $res->{status} }"
  } elsif $res {
    p "ok ${ $c->{id} }: ${ $res->{status} }"
  }
}
if $fail > 0 { die "${ $fail } assertion(s) failed" }
p "all ${\ scalar @cases} cases passed"
```

The saved requests, the environment, and the send are all **zreq**'s own features; the assertions and the exit status are the script's. The bus is what lets the two meet — the response landing in **\$res** where the script can assert on its status — and the result is a test suite whose cases live in the API client and whose logic lives in a file a CI job runs.

Auditing a remote store against a policy

An operator's chore is to walk a remote tree and flag what violates a policy — here, files that are world-readable or larger than a bound, using **zftp**'s filesystem namespace:

```
#!/usr/bin/env stryke
use App

val $ftp = App::open("zftp")
val @flagged = @{}

fn walk($path) {
  for val $e (@{ $ftp->call("fs.list", %{ path => $path }) }) {
    if $e->{type} == "dir" {
      walk("${ $path }/${ $e->{name} }")
    } else {
      if $e->{size} > 100_000_000 {
        push @flagged, "${ $path }/${ $e->{name} }: large (${ $e->{size} })"
      }
      if $e->{mode} & 0o004 {
        push @flagged, "${ $path }/${ $e->{name} }: world-readable"
      }
    }
  }
}

walk("/srv/data")
p "flagged ${\ scalar @flagged} files"
for val $f (@{ @flagged }) { p "  ${ $f }" }
if scalar @flagged > 0 { die "policy violations found" }
```

`fs.list` returns each entry as data with a name, type, size, and mode, and the recursion, the policy, and the report are ordinary stryke. The app knows how to list a remote directory; the script knows the policy; and the audit is the two composed. Nothing here crossed apps, yet nothing here was possible before the bus, because before it there was no way to read a remote listing into a program and reason over it.

Driving the office engine as a document factory

The last recipe treats `zoffice` as a batch document factory: for each row of a data range, fill a template and render it, producing a stack of finished documents from one sheet and one template:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")

# the data rows that drive the batch
val @rows = @{ $office->call("calc.cells",
  %{ sheet => "Certificates", range => "A2:C100" }) }

for val $row (@{ @rows }) {
  val $name = $row->{a}
  if !$name { next }
  # open the template, substitute, render to PDF
  $office->call("writer.open", %{ path => "~/templates/cert.odt" })
  $office->call("writer.replace", %{ find => "{{name}}", with => $name })
  $office->call("writer.replace", %{ find => "{{course}}", with => $row->{b} })
  $office->call("writer.replace", %{ find => "{{date}}", with => $row->{c} })
  val $out = $office->call("writer.render",
    %{ format => "pdf", path => "~/out/cert-${ $name }.pdf" })
  p "rendered ${ $out->{path} }"
}
p "produced ${\ scalar @rows} certificates"
```

`calc.cells` supplies the data, `writer.open/writer.replace/writer.render` drive the template, and the loop is the script's. The office engine has every one of these features for a human clicking through them one document at a time; the bus is what lets a script drive them a hundred documents at a time, each value from the sheet flowing into a substitution and out as a rendered file. It is the plainest statement of what the whole book has argued: the apps were always sets of functions waiting to be called, and the bus is the caller — for one app driven a hundred times, exactly as much as for a dozen apps driven once.

Chapter 36 – A Design FAQ

The questions the chapters raise but do not stop for

A book that argues a design linearly leaves questions in its wake — the “but what about” that a reader holds while the argument moves on. This chapter is for those, gathered and answered directly. Each answer is grounded in a decision an earlier chapter made; the FAQ form is a second index into the design, organized by the question a skeptical reader asks rather than by the layer the answer lives in.

Why not HTTP, or gRPC, or D-Bus?

Because the bus is a single-user, single-machine local facility, and each of those brings machinery it does not need and a threat surface it refuses. HTTP brings a network stack, request framing, and status semantics for a conversation that never leaves the machine; gRPC brings a schema compiler and a code-generation step for a wire that is already a newline of JSON a human can read; D-Bus brings a system bus, a broker process, and an introspection language for what a directory of socket files provides directly. The bus chose a Unix socket and newline JSON because they are the smallest thing that carries a correlated request/response with an attached stream, and because they keep the security model to one sentence — a file at mode `0600`, no network code, ever. Every heavier transport would add capability the bus does not want and a bind the bus refuses.

Why synchronous calls instead of async everywhere?

Because an automation script is almost always a sequence of steps where each depends on the last, and for that shape a synchronous call is both the simplest to write and the correct default for backpressure. `App::call` blocks until its reply, so the script cannot outrun the app, and a straight-line orchestration reads top to bottom as it executes. The asynchronous machine is still there underneath — a timer against a correlation id, per Chapter 14 — but it surfaces as a function that returns a value or dies, which is the API an automation author wants. A future pipelined mode can let a script fire a batch and gather the answers, and the correlation `id` on every frame was put there from version one precisely so that mode is an addition rather than a redesign; but the default is synchronous because the default use is sequential.

Why does a failed call die instead of returning nil?

Because a mistyped verb or a dead target is a bug in the script, not a branch the script meant to take, and returning nil would let the mistake propagate silently until some later line dereferenced the nil and blamed the wrong statement. Dying at the call site, with the fault named, puts the error where the fault is. A script that genuinely wants to survive a failure opts in with a `try`, per Chapter 14 — but the default is a loud death, so the fast path stays free of defensive ceremony and the slow path fails honestly. The single worst outcome for an automation script is a hang, which produces no signal at all; the second worst is a silent wrong answer; and the failure model spends its complexity to make both impossible, leaving only the loud, diagnosable death.

Why can a handle not be passed to another app?

Because a handle is a live connection, not a value, and there is nothing on the far side to receive a connection. Serializing a handle would mean shipping a socket, which means nothing to the app that received it. The rule that a handle is VM-local is what produces the star topology of Chapter 13 — the script stays the hub through which every value passes, and no app ever holds a connection to another — and that topology is also exactly what the security model wants, since an app is never a client of another app, only ever a server to the user's scripts. One decision, two chapters' worth of consequence, no coordination needed between them.

Why is there no “run arbitrary code” verb?

Because such a verb would be one entry in the allow-list that made every other entry meaningless. A caller with a generic eval verb could do anything the app's process could, and the careful enumeration of typed verbs would be decoration over a hole. The apps that legitimately run arbitrary code do it only through the palette's `js` and `bash` steps, which are not bus verbs — they are steps in a command the user typed into their own local palette, never reachable through `call`. The distinction is exact and load-bearing: an escape hatch the user reaches for deliberately in a local editor is a different thing from a capability a remote caller can invoke, and the security model protects precisely that line.

What happens if two copies of an app launch at once?

The second to bind wins the socket name, and the first is left serving a socket no dialer will find. Per Chapter 6, this is treated as correct rather than papered over, because the suite's apps are single-instance by intent — one running `zcite`, one command palette, one front door — so “the last to bind owns the name” is the single-instance rule expressing itself at the socket layer. A future launch-on-demand is where this would be tightened; version one lets the last writer win, and the unlink-before-bind step is what makes a crashed app's stale socket recoverable on the next launch.

Does carrying the bus slow an app down when nothing is driving it?

No, to a first approximation. At rest the bus is a bound `UnixListener` with no connections, a registered surface sitting as an in-memory data structure, and the `emit` calls the app makes at its own event points — each a membership check against an empty subscription list. There is no polling loop and no background thread; the listener blocks in the kernel waiting for a `connect` that may never come, which costs a file descriptor and nothing else. An app on the bus that no one is automating is, per Chapter 24, an app not on the bus at all, which is the property that let the fan-out reach every app without weighing the runtime cost of each.

Can a script drive an app it has never seen?

Yes, and the generic status report of Chapter 30 is the proof. A script can `App::list()` the running apps, `App::open` each, and `verbs()` its surface, learning what the app exposes at run time rather than from a hardcoded list. Helpers like `has_verb` and `call_if` from Chapter 32 are built entirely from the seven calls and work against any app, including one written after the helper. This is the platform property of Chapter 18 spent at the script level: because every app answers the same seven calls, a utility written against those calls is a utility for the whole suite.

Why version the wire, the surface, and the module separately?

Because they age at three different rates, per Chapter 22. The surface grows every time an app adds a feature; the module moves rarely because a hundred scripts import it; the wire moves slowest of all because everyone speaks it. A policy that treated them as one thing would either freeze the fast-moving surface to protect the slow-moving wire or destabilize the wire every time a verb was added. Versioning them separately lets the surface grow additively — new verbs, new optional parameters, all non-breaking — while the wire changes only by adding fields old readers ignore, which is why the frame set is closed and grows by widening frames rather than multiplying them.

When will cross-machine automation ship, and how?

If it ships, it ships as its own thing with its own threat model — authentication, transport security, capability scoping — not as this socket with a wider bind. Chapter 18 is firm on this: the bus's whole security posture rests on the socket being a single-user, single-machine artifact at mode `0600`, and the moment a frame can arrive from another host every verb becomes a remote procedure call the allow-list was never designed to distrust. Cross-machine is a real and different problem, and folding it into the local bus by loosening one bind would dissolve a clean model. The connector family of Chapter 33 already shows the shape a remote member would take: `stryke-aws`'s posture, not `App`'s.

Chapter 37 — Glossary

The terms below are the book's load-bearing vocabulary, each defined as this book uses it. Where a term has a broader meaning elsewhere, the definition here is the bus's.

allow-list. The dispatch discipline by which only verbs explicitly registered — in the engine's `COMMANDS` or through `ZGui.automation` — are callable, and every unrecognized verb is rejected with `no such verb`. The default answer to an unknown verb is `no`.

appShell. The shared frontend every app is built over, contributing the fifteen shared `appshell.*` verbs — `terminal`, `files`, `palette`, `settings`, `shortcuts`, `theme`, `CRT`, `neon` — that every app inherits without declaring them.

audit trail. A script-side record of a run — one line per call, with the app, verb, outcome, and duration — written by a wrapper around the handle rather than by the bus, because only the script knows which of its arguments are safe to write down. Distinct from a checkpoint, which exists to make the next run correct rather than to let a human read the last one.

bridge. Layer 2: the correlated request/response substrate, with an attached event stream, that lets an in-flight script call back into an app and get the value into the VM. One substrate, used three ways — `call`, `get`, and `on`.

bus name. The short name that identifies an app on the bus — `zcite`, `zreq` — used by `App::open`, returned by `App::list`, and forming the socket filename `<app>.sock`.

call. The operation, and the frame kind, that invokes a verb with typed arguments and returns its value. The central operation of the bus; everything else supports it.

catalog. `GUI_SCRIPT_ACTIONS.md`, the machine-generated census of every app's live surface, re-taken on demand from the running apps. The authority on the exact verb set; a projection of the code, not prose about it.

core. The pure-Rust `*-core` engine each app is a thin shell over. Owns the app's verbs; its `COMMANDS` list is the authority on which verbs route straight to Rust.

event. An app-to-script notification — a named channel the app pushes down with `emit` and a script receives by subscribing with `on`. Answers “something just changed,” where a state query answers “what is true now.”

frame. One line of newline-delimited JSON on the wire. Seven kinds: `call`, `get`, `verbs`, `sub`, `unsub` from the client; `reply`, `event` from the server.

get. The operation, and the frame kind, that reads a state query — a read-only observation of the app's current condition, with no side effect.

handle. A live binding to one app: in-process, the host callback into the local surface; out-of-process, an open socket to one app's `.sock` file. Minted by `App::here` or `App::open`, VM-local, never a value that can cross to another app.

hybrid handler. The per-app `bus.rs` that routes each incoming call either to the engine in Rust (if the verb is in `COMMANDS`) or to the frontend via the dispatch shim (otherwise). The one app-specific decision the shared host delegates.

id. The correlation token on every request frame, matched by the reply's `id` and the event's `sub`. A conversation token, not a clock; what makes concurrency and event delivery safe.

incidentally visible verb. A verb whose purpose is to change durable state but which also moves what the user sees, because the app is a live view over the state it changed. The class that catches unattended runs, since nothing in the surface marks it.

in-process. The transport for a script running inside the app — a palette step or a hook — binding `App:here()` directly to the local surface through the host callback, with no socket.

job shape. The surface convention for work too long to block on: a starting verb that returns an identifier immediately, progress delivered as events, status readable as a state query, and cancellation offered as a further verb keyed by the identifier. The bus has no cancel frame, so a long verb is cancellable only if its author wrote the verb.

manifest. The typed surface an app registers: its verbs, state queries, and events, with their parameter and return types. Read by a script as `verbs()`, stripped of function bodies; read by the palette's own picker; the single source both readers share.

out-of-process. The transport for a standalone `.stk` file driving a separately running app over its Unix socket. Adds one socket round trip and two JSON serializations over the in-process path.

pump. The explicit point — `App:pump()` — at which queued event callbacks run. Events never interrupt a running script; they queue and drain only where the script pumps, so the author always knows the exact set of places a callback can run.

star topology. The shape every multi-app run takes: the script at the center, each app a spoke, every cross-app value passing through the script's own variables. A consequence of the rule that a handle is not a value.

state query. A read-only named observation of an app's current condition — the selection, the active collection, the connection status — read with `get` and idempotent by contract.

surface. Layer 1: an app's typed manifest of verbs, state, and events, owned by the core and registered once on boot. Also, loosely, the whole set of what an app exposes to the bus.

verb. A named, typed, callable action an app declares — its id, parameters, return type, and runnable body. The unit of everything the bus does; an app's surface is mostly verbs.

wire. The newline-delimited JSON protocol carried by both transports. Deliberately small, legible, and closed: it grows by adding fields old readers ignore, never by adding frame kinds they would reject.

Chapter 38 – Tooling: The REPL and Completion in Practice

The reflective surface, spent as tools

Chapter 17 named two tools that fall out of an enumerable surface — a REPL and a completion source — and called them the surface's own reflectiveness, spent twice. This chapter builds both far enough to use, because a reader who has followed the design deserves the tools that make it livable, and because the way each is built is the last and clearest demonstration that everything the bus offers is a projection of one registration read many ways.

The REPL, built from two calls

A REPL for the bus is a loop over `verbs()` for discovery and `call` for execution, and its whole implementation is short because the surface does the work. It reads an app name, opens a handle, and then reads verb-and-argument lines, dispatching each and printing the decoded reply:

```
#!/usr/bin/env stryke
use App

val $name = prompt("app> ")
val $h = App::open($name)
p "connected to ${ $name } - ${\ scalar @ { $h->verbs()->{verbs} }} verbs. ? to list, q to
  ~ quit."

loop {
  val $line = prompt("${ $name }> ")
  if $line == "q" { break }
  if $line == "?" {
    for val $v (@ { $h->verbs()->{verbs} }) { p "  ${ $v->{id} }" }
    next
  }
  # a line is a verb id then a JSON argument object: request.send {"url":"..."}
  val ($verb, $json) = split_first($line, " ")
  val $res = try { $h->call($verb, parse_json($json ?? "{}")) }
    catch ($e) { p "! ${ $e }"; nil }
  if $res != nil { p pretty_json($res) }
}
```

Nothing in that loop is app-specific. It works against `zcite`, `zreq`, or an app added next year, because it drives every app through the same two calls, and its discovery — the `?` command — reads the live manifest, so the list of verbs it offers is always exactly what the connected app will answer to. The REPL is a database shell for a scriptable app, and it cost a loop.

A tracing REPL, for debugging

Chapter 23 named the verbose variant, and it is one added line: echo the frame you send and the frame you receive. In a REPL that already sends a `call` and receives a `reply`, a trace mode prints both around the decoded value:

```
if $trace {
  p "-> {"t\":"call\","verb\":"${ json_str($verb) }","args\":"${ $json ?? "{}" }}"
}
val $res = $h->call($verb, parse_json($json ?? "{}"))
if $trace { p "<- {" compact_json($res) }}" }
p pretty_json($res)
```

Now the REPL is a probe: type a verb, watch the exact frame go out, watch the exact frame come back, read the value the module decoded from it. It is the fastest way to answer “is the app returning what I think it is?” because it puts a human at the exact seam where the script’s assumption meets the app’s reality — which is where Chapter 23 said most misbehaving-script questions are actually answered.

Shell completion, from `list` and `verbs`

The completion source is the second tool, and it is where the surface’s reflectiveness touches the shell the whole suite lives in. A completion function offers, after `App::open()`, the live set of running app names from `App::list()`, and after a handle’s `->call()`, the live set of verb ids from that app’s `verbs()`. The shape, as a completion callback:

```
# after App::open(" - complete the running app names
fn complete_app_name($prefix) {
  grep(App::list(), fn ($n) { starts_with($n, $prefix) })
}

# after $h->call(" - complete the verb ids of the app $h is bound to
fn complete_verb($h, $prefix) {
  map(grep(@{ $h->verbs()->{verbs} }, fn ($v) { starts_with($v->{id}, $prefix) })),
    fn ($v) { $v->{id} })
}
```

The completion is live in the strongest sense: it offers exactly the apps that are running and exactly the verbs they expose *at that moment*, not a static list baked into the completion at install time that drifts as apps start, stop, and gain verbs. A script author tab-completing against `App::open("z` sees only the apps actually up; tab-completing a verb after opening `zreq` sees only `zreq`’s current verbs. The completion cannot go stale because it is not a copy of the surface — it is the surface, queried at the moment of the tab.

Why neither tool is a feature of the bus

It matters that neither the REPL nor the completion is a capability the bus added; both are the surface’s own enumerability, spent once each. The bus did not ship a REPL and a completion engine; it shipped an enumerable surface, and a REPL and a completion are what an enumerable surface makes almost free. This is the deepest form of the “one declaration, many readers” property the book has traced from Chapter 3: the same registration that makes an app scriptable makes its palette picker richer, makes its verbs discoverable, makes a catalog generatable, makes a REPL possible, and makes a completion live — six readers of one declaration, none of which the app author wrote, all of which the app author gets for declaring the surface once. A capability whose tooling falls out of its data model rather than being built beside it is a capability that was designed at the right layer, and the REPL and the completion are the proof, because they exist without a line of code that knows any app’s name.

The tools compound

A last observation closes the practical arc. The tools do not merely coexist; they compound, because each is built from the same seven calls and so each works inside the others. The completion completes the verbs the REPL calls; the REPL, in trace mode, shows the frames the debugging chapter reads; the catalog the versioning chapter diffs is the same enumeration the completion offers live; and a helper library written against the seven calls, per Chapter 32, is callable from a script, a REPL session, or a maintenance run without change. The uniformity that let one crate host eleven apps and one module drive them all is the same uniformity that lets the tools built on top interlock — one surface, one wire, one vocabulary, and every tool a different reading of it. That is the whole system in its final shape: not a stack of features, but one programmable surface and the many ways a script, a shell, a REPL, and a generator each read it — every one of them handed, at last, a caller.

Chapter 39 – The Step Chain, Reconsidered

What the grammar is for, now that one step can do everything

Chapter 1 walked the palette’s six step kinds — `url`, `js`, `scheme`, `event`, `stryke`, `bash` — and found every one of them unable to carry the job the bus now carries. Chapter 9 then handed the `stryke` step a binding to `App::here()` and made it, alone among the six, capable of reading an app, branching on the answer, and calling a second verb. That leaves an obvious question the earlier chapters set down and walked past: if one step can now do everything, what are the other five for?

The answer is not “nothing,” and the temptation to conclude otherwise is the first mistake a palette author makes after the bus lands. The grammar was not deprecated by the bus; it was re-sorted. Each of the other five steps keeps a job the `stryke` step does *worse* — not because the `stryke` step lacks the reach, but because reach is not the only cost a step has. A step that starts a JIT-compiled VM, binds a handle, and dispatches a verb to do what a one-word step already does is paying for capability it does not spend. The re-sorting is worth stating explicitly, because a palette that gets it right stays fast and readable, and a palette that gets it wrong turns every doorbell into an orchestration.

The five steps and their remaining jobs

The `event` step is the one most obviously in the `stryke` step’s shadow, and it is the one that survives most clearly. It fires exactly one app action by id and returns nothing, which is precisely right when the command *is* one action and nothing follows it. A palette entry that toggles the CRT effect or opens the settings panel wants `event`, not a script: there is no value to read, no branch to take, and no second app to reach. The bus did not make `event` obsolete; it made it honest about its scope. It is the doorbell, and most palette commands genuinely are doorbells.

The `url` step navigates. Its target is a location, not an app verb, and the thing it opens frequently belongs to no app on the bus at all. A palette entry that opens a documentation page is not automation and should not become one; routing it through a verb call would mean an app declaring a verb whose entire body is “open this location,” which is a verb that exists only to be the wrong tool for the job.

The `scheme` step fires a custom-scheme URL — `zcite://item/42` — and remains one-way by construction. Its property is that it addresses an app by URL rather than by socket, which means it is the step that does not care whether a socket exists, whether the target registered a surface, or whether the target is on the bus at all. That is a narrow but real job. A chain that wants to poke an app with an address and learn nothing back has no reason to open a connection, dial a verb, and discard a reply; the scheme step is the shape of “address it and move on,” and the bus’s `call` is the shape of “ask it and wait.”

The `js` and `bash` steps are the two Chapter 10 pinned in place as escape hatches, and their remaining job is exactly the one that chapter defined for them: reaching what the allow-list deliberately does not expose. `js` evaluates against the front-most webview’s own internals, which is the right and only tool when the thing you need is genuinely private to the frontend and has not been — and perhaps should not be — promoted to a verb. `bash` reaches the rest of the machine. Both remain user-authored steps in a local palette and never

remotely callable verbs, and the bus's arrival strengthens rather than weakens that line: now that the typed surface exists, the *reason* to reach for `js` is narrower and more deliberate, which makes each remaining use of it a considered one rather than a shrug.

What the bus did not fix: step-to-step data flow

There is a defect in the original grammar that the bus pointedly does not repair, and being clear about it prevents a class of disappointed expectation. Chapter 1's diagnosis was that “not one of them could read a value out of an app and hand it to the next step.” That is still true. A `url` step followed by an `event` step still passes nothing between them. There is no step-chain variable, no output binding, no `$1` from the previous step. The grammar remains a sequence of independent effects.

What changed is not that the chain learned to carry values but that the chain no longer *needs* to, because the whole of a value-passing workflow now collapses into a single `stryke` step where the values live in ordinary variables. The migration is therefore not “wire the steps together”; it is “replace the multi-step chain with one step.” A chain that read as three `event` steps with a `bash` step in the middle to sleep between them becomes one `stryke` step that calls three verbs and reads a state query between them, and the intermediate values that the chain could never hold are simply local bindings:

```
# what a four-step chain becomes, once one step can hold values
val $me = App::here()
val $sel = $me->get("selection")
if scalar @{ $sel } == 0 { return }
val $c = $me->call("collection.add", %{ name => "trriage" })
for val $it (@{ $sel }) {
  $me->call("collection.add_item",
    %{ collection => $c->{name}, item => $it->{id} })
}
```

Every value that crosses a line there — `$sel`, `$c->{name}`, `$it->{id}` — is a value the step chain had no way to represent. The chain did not gain the ability to pass them; the step gained the ability to make the chain unnecessary.

The collapse rule, and its limit

A useful rule falls out. Collapse a chain into one `stryke` step exactly when the chain contains a data dependency or a decision — when a later step needs a value an earlier step produced, or when whether a later step runs at all depends on how an earlier one turned out. Keep the chain when its steps are genuinely independent effects that happen to be bundled under one keystroke: open a location, and also toggle a theme, and also fire an action. Those have no dependency, and expressing them as three steps is clearer than expressing them as three lines inside a script, because the step chain *shows* the independence at a glance while a script hides it in sequence.

The limit of the rule is worth naming too. A chain that mixes kinds — a `stryke` step that computes something and a `bash` step that acts on it — still cannot pass the value, so the mixed chain is the one shape that remains genuinely awkward. The resolution is not a new step-chain feature but the observation that `stryke` can do the `bash` step's job inside the script, keeping the value in a variable rather than trying to hand it across a step boundary that was never built to carry one. Where the chain wants to pass a value, the answer is always to stop having a chain.

The palette entry as a dispatcher

The last shift the bus produces in palette authoring is one of role rather than mechanism. Before, a palette entry was a bundle of effects; the interesting logic, if any, lived somewhere else or nowhere. After, a palette entry is

frequently a *dispatcher* — a keystroke that hands control to a script that then does the real work against the app, possibly against several apps, possibly ending in a mailed summary or a written file. The entry becomes thin and the script becomes the artifact, which is the right distribution of weight: a keystroke should be a name for a behavior, and the behavior should live somewhere a person can read, version, and test.

This also closes a gap that had no good answer before. A palette command and a standalone `.stk` file used to be different kinds of thing, expressed in different grammars, with the palette's version strictly weaker. Now they are the same thing with two different triggers: the palette step binds `App::here()` and the shell file binds `App::open(name)`, and everything above that line is identical code. A workflow can be developed at the prompt as a `.stk` file, per the interactive session idioms of Chapter 32, and then bound to a keystroke by pasting it into a `stryke` step with `here()` substituted for `open()` — one substitution, and a script becomes a command. That portability between the trigger surfaces is the practical dividend of Layer 5's "one module placed in three scopes," and it is the reason a palette author and a script author are now, for the first time, the same person writing the same language against the same surface.

Chapter 40 — Concurrency: Many Scripts, Many Apps

Two axes, and only one of them is the script's

Every earlier chapter has treated a run as if the script were alone with the apps. It is not, and the ways it is not fall on two axes that behave very differently. The first axis is *one script driving many apps*, which Chapter 13 settled: the star topology, the script as hub, every cross-app value passing through a stryke variable. That axis is fully under the script's control, because the script issues every call and sequences every step. The second axis is *many writers driving one app* — a second `.stk` file, a palette step the user just invoked, a hook the app fired at itself, and, most importantly, the human sitting at the keyboard with the GUI open in front of them. That axis is not under the script's control at all, and the bus does nothing to pretend otherwise.

The second axis is the one this chapter is about, because it is the one the design deliberately leaves to the app and to the script author rather than solving in the transport. Chapter 6 established the mechanism: one listener, many connections, each with its own `id` space, its own subscriptions, its own reply slots. That gives isolation of *bookkeeping* between concurrent dialers. It gives no isolation of *effect*. Two scripts that both call `collection.add_item` against the same collection are two callers mutating one app's state, and the bus's per-connection isolation has nothing to say about the outcome.

What is ordered, and what is not

Three ordering statements are true, and the fourth thing a reader wants to be true is not.

Within one connection, request and reply are correlated by `id` and nothing more; the `App` module assumes no arrival order, per Chapter 15. In today's straight-line mode a script has one call outstanding at a time, so its own calls against one app execute in the order it issued them — not because the wire guarantees ordering, but because the script never has two in flight.

Across two apps there is no ordering relationship whatsoever, and there could not be. `zcite` and `zreq` are separate processes that have never heard of each other; the only thing sequencing a call to one against a call to the other is the script's own straight-line execution. A script that calls `zoffice` and then `zpdf` has ordered them because it blocked on the first before issuing the second. Remove the blocking — a future pipelined mode — and the ordering is the script's to re-establish, not the bus's to preserve.

Within one app, across two connections, there is no ordering at all. If two scripts each issue a call, the app serves each on its own task, and which verb body runs first, or whether they interleave inside the app, is the app's business and its concurrency model's — not the bus's. The bus delivers frames; the app decides what running two verbs at once means.

And the thing a reader wants: there is no transaction. The bus has no `begin`, no `commit`, no verb-sequence atomicity. A script that calls three verbs to accomplish one logical change has performed three separate changes, and anything else touching the app may observe or interleave with the intermediate states. This is stated flatly rather than hedged because a script author who believes otherwise will eventually write a sequence that is correct only when nothing else is running, and that is exactly the bug that surfaces once and never reproduces.

The human is a concurrent writer

The concurrency case that catches people is not two scripts; it is one script and one person. The apps on this bus are not headless services. They are GUI applications with a user in front of them, and while a script drives an app the user may be clicking in it. The selection a script read a moment ago is a selection the user can change by moving the mouse. The document a script opened is a document the user can close. The collection a script is filling is a collection the user can rename.

This has a direct consequence for the read-modify-write shape, which is the most common thing an orchestration does. Consider the plainest possible sequence: read `selection`, loop over it, act on each item. Between the `get` and the last iteration of the loop, an arbitrary amount of wall-clock time passes — each iteration may be a network request — and the live selection may have changed several times. The script is not iterating the selection; it is iterating a *snapshot of the selection taken once, at the top*. That is almost always what the author wanted, and it is worth understanding as a deliberate property rather than discovering it as a surprise.

The design guidance falls straight out. Capture the collection you intend to act on once, at the start, into a local variable, and drive the loop from the local copy — which is what every recipe in the cookbooks does, and now the reason is explicit. Act on *identities*, never on positions: `$it->{id}` remains meaningful when the selection changes underneath, while “the third selected item” does not. And never re-read a live state query mid-loop expecting it to still describe the world the loop started in; if a script genuinely needs the current truth at each step, it should read it at each step and handle the change, not read it once and assume it froze.

Building exclusion out of what exists

Since the bus offers no locking, a script that truly must not interleave with another writer has three options, and they are worth ranking honestly.

The first and best is *be the only writer*. One orchestrating script, running while the user is not driving the same app by hand, is the shape almost every real automation takes, and it dissolves the problem rather than managing it. The scheduled maintenance runs of Chapter 30 are exactly this: they run when nobody is watching, and the reason they are safe is not a lock but an absence of competitors.

The second is *push the atomicity into the app*. If a logical change must be all-or-nothing, the right place for it is one verb whose `run` body does the whole thing, because a single verb's execution is the only unit the app can make atomic. A script that finds itself wanting a transaction across three verbs has found a missing fourth verb. This is not a workaround; it is the ownership rule of Chapter 2 applied to concurrency — the engine knows how to make its own state changes coherent, and the bus's job is to call the engine, not to invent coherence above it.

The third and weakest is a *lease built from a state query and a verb*, where an app exposes something a script can claim and release. This works only if the app was designed to support it, is invisible to the human at the keyboard, and buys less than it appears to, since nothing prevents a second script from ignoring the lease entirely. It is mentioned for completeness and recommended for nothing. The honest summary is that the bus is a calling convention, not a concurrency-control system, and a script that needs concurrency control should get it from the app's own semantics or from not having competitors.

The one concurrency guarantee the bus does make

Against all that, there is a real guarantee, and it is the pump's. A script's own event callbacks never race its own main line, because callbacks run only at `App : pump()` and nowhere else, per Chapter 15. A watcher that maintains a shared array across its callback and its loop body cannot observe that array half-updated, cannot have a callback fire inside an expression, and does not need a mutex around its own data — because the VM never runs two of the script's own frames at once. That property is why the event tap of Chapter 30 can push

to a list from two different subscriptions' callbacks and read the list in the loop with no synchronization: the three sites are serialized by the pump, strictly.

It is a narrow guarantee, and its narrowness is the chapter's summary. Inside one script's VM, the bus's execution model gives strict serialization for free. Outside it — between scripts, between a script and the user, between a script and the app's own internal work — the bus gives frame delivery and nothing more, and every coordination question there belongs to the app's semantics or to the author's discipline about who is allowed to be driving what, when.

Chapter 41 – Restarts, Partial Runs, and Resumability

Failure above the call

Chapter 14 gave the failure model for one call and Chapter 29 catalogued each error a call can produce. Both operate at the granularity of a single request. A long unattended run fails at a different granularity: not “this call failed” but “this run died two thirds of the way through, and something must be done about the third that did not happen and the two thirds that did.” That is a distinct problem with distinct answers, and the bus’s own machinery deliberately solves none of it — which makes it exactly the problem a book about the bus owes a chapter to.

Three shapes recur. A target app restarts mid-run, from a crash or from the user quitting it. The script itself dies, from an uncaught failure or from the machine sleeping under it. Or the run finishes but a subset of its items failed, per the try-per-item pattern, and the failures need retrying later. Each wants a different treatment, and all three rest on one uncomfortable fact this chapter has to establish first.

The effect of a broken call is unknown

Consider a `call` that dies with a broken pipe, per Chapter 14: the frame went out, the peer vanished, and no reply is possible. The script knows the call did not *complete*. It does not know whether the call *happened*. The frame may have been read, dispatched, and executed to completion by the app, with only the reply lost when the process went down — or it may have been sitting unread in a socket buffer when the process died. Both produce exactly the same observation on the script’s side: a write that succeeded and a read that failed.

There is no fix for this in the transport, and it would be dishonest to imply one. It is the ordinary and unavoidable consequence of a request/response protocol without a durable log on either side, and the bus is explicitly not building a durable log — Chapter 20 already refused to persist an event stream for the same family of reasons. What the bus guarantees is that the *reply* is delivered at most once and correlated correctly. What it cannot guarantee is that the *effect* happened exactly once, because the effect is the app’s and the failure was the channel’s.

The consequence for a script is a discipline rather than a mechanism: after a broken call, do not assume, and do not blindly retry. Read the state. A script that just failed to add an item asks the app what items exist; a script that just failed to send a request consults the history the app maintains. This is the state-versus-event division of Chapter 20 spent on recovery — the state is the source of truth, and after any uncertainty the correct move is to go and observe the truth rather than to reason about what probably happened. That the bus makes state cheap to read is exactly what makes this discipline practical, which is a small vindication of `get` being a first-class operation rather than a convenience.

Re-opening a handle, and the subscription that does not come back

When a target restarts, the handle bound to it is dead: the socket the handle wraps is a connection to a process that no longer exists, and the single bounded retry of Chapter 14 covers only the case where the target came back fast enough to answer the immediate redial. A run that expects restarts must re-open the handle explicitly, which is simple — and must re-subscribe, which is the part that gets forgotten.

The reason is a consequence Chapter 20 established but did not spell out. Subscriptions are per-connection: when the connection drops, the app tears down all of that connection's state, subscriptions included. A newly opened handle is therefore a handle with no subscriptions, even if the script's own closures are still sitting in the VM from the previous connection. A watcher that reconnects and does not re-on will sit in its pump loop forever, perfectly healthy, receiving nothing, which is the quietest possible failure and therefore the worst.

The pattern that survives this is to make connection and subscription one operation, so that they cannot get out of step:

```
#!/usr/bin/env stryke
use App

# connect + subscribe as one unit, so a reconnect can never lose the stream
fn attach($name) {
  val $h = App::open($name)
  $h->on("downloadComplete", fn ($dl) { handle_download($dl) })
  $h
}

val $wire = attach("zwire")

loop {
  val $ok = try { App::pump(); true } catch ($e) { p "lost zwire: ${ $e }"; false }
  if !$ok {
    App::sleep(2000)
    $wire = try { attach("zwire") } catch ($e) { nil }
  }
  App::sleep(500)
}
```

The helper is ordinary stryke, and its whole content is the discipline: there is no path in the program that produces a handle without a subscription on it. The bus does not offer a durable subscription that survives a reconnect, and this is the reason it does not need to — the reconnection logic belongs in the script, where the script's own knowledge of what it wanted to watch already lives.

Checkpointing a batch

The second shape — the script itself dying — has an answer as old as batch processing, and the bus neither helps nor hinders it. A run over many items writes down which items it has finished, and on restart it skips those. The bus's only contribution is that the unit of work is a verb call whose outcome the script knows precisely, which makes “finished” an easy predicate to evaluate honestly.

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val %done = read_checkpoint("~/.cache/file-dois.done") # ordinary stryke I/O

for val $doi (@{ lines(read_file("dois.txt")) }) {
  if %done{ $doi } { next } # already filed on a past run
  val $item = try { $cite->call("item.add", %{ doi => $doi }) }
               catch ($e) { p "deferred ${ $doi }: ${ $e }"; nil }
  if $item {
    %done{ $doi } = true
    append_file("~/.cache/file-dois.done", "${ $doi }\n")
  }
}
p "filed ${\ scalar keys %done} of the input set"
```

Two properties make that correct rather than merely plausible. The checkpoint is appended *after* the call returned a value, so an item is recorded as done only once the app confirmed it — the return path of the bus is what makes the checkpoint trustworthy, since a fire-and-forget action could only have recorded that the attempt was made. And an item whose call failed is simply not recorded, so the next run retries it; the failure path needs no separate bookkeeping because absence from the checkpoint already means “not done.”

The uncomfortable case from earlier survives here in exactly one place: an item whose call broke mid-flight may have taken effect in the app without being recorded, and the next run will call it again. Whether that is harmless depends entirely on the app's own semantics for that verb, not on the bus, which brings the chapter to its last question.

Idempotence belongs to the app

A script cannot make a verb idempotent. If `item.add` called twice with the same DOI produces two items, it produces two items, and no amount of care in the script's retry logic changes that — the bus will faithfully deliver both calls, because faithful delivery is the whole of its contract. Idempotence is a property of a verb's implementation and therefore a property the app author chose, deliberately or by accident, when they wrote the `run` body.

This is worth naming as guidance in both directions. For an app author: a verb that is naturally idempotent — one that recognizes an existing thing and returns it rather than creating a duplicate — is enormously more valuable to unattended automation than one that is not, and the cost of making it so is usually a single lookup at the top of the `run` body. That is a design decision worth making at surface-declaration time rather than discovering after the first retry storm. For a script author: do not assume idempotence you have not verified, and where you cannot verify it, prefer the reconciliation shape over the retry shape — read what exists, compute the difference, and act only on the difference.

That reconciliation shape is the resumability pattern the bus is best at, and it is not a coincidence that Chapter 16 already showed it as a recipe in its own right. A script that computes “what is present” against “what should be present” and acts on the gap is inherently resumable, inherently safe to run twice, and needs no checkpoint file at all, because the state of the world is its checkpoint. Where a workflow can be expressed that way it should be, and the bus makes it expressible for the first time, since computing the gap requires reading both sides — which is precisely the return path the whole book has been about.

Chapter 42 – Naming a Surface

The id is the most expensive thing an app author types

Chapter 22 established that the verb id is the compatibility surface: a script that calls `library.search` is coupled to that string and to nothing else, and an app that renames it breaks every script written against it. That makes the id the single most durable artifact in the whole design — more durable than the `run` body it dispatches to, which can be rewritten freely, and far more durable than the `label`, which is a human-facing string that may change with the wind. An engine can be replaced under a verb and no script notices. Rename the verb and every script notices at once.

A surface's names are therefore not a matter of taste, and the shipped census of Chapter 8 turns out to encode a consistent set of conventions worth extracting, because eleven apps naming things the same way is what lets a script author who learned one app guess correctly at the next. This chapter reads those conventions off the surface and states them as rules.

Namespace is the noun, leaf is the action

The dominant shape across every app is `noun.action`, where the namespace names a thing the app's domain has and the leaf names something done to it. `collection.add`, `collection.rename`, `collection.addItem` — the namespace is the domain object, and the leaves are its lifecycle. `transfer.add`, `transfer.cancel`, `transfer.resume_check`. The same shape at `account.add`, `folder.create`, `profile.save`, `request.send`.

Reading a namespace is therefore reading the product's domain model, as Chapter 8 observed, and the rule that produces that property is: *name the namespace after the noun the user would name*. `zcite` shelves things in collections, so the namespace is `collection`, not `folder` or `group` or `container`. `zreq` sends requests, so it is `request.send`, not `http.execute`. The test is whether a competent user of the app, asked what the thing is called, would say the namespace's word. When they would not, the namespace is describing the implementation rather than the domain, and every script author will have to translate between what they mean and what the surface calls it, forever.

The conventions the census already keeps

Several finer conventions run consistently across the shipped surfaces, and adopting them in a new app is free interoperability.

Symmetric operations get symmetric names. `vpn.connect` / `vpn.disconnect`. `appshell.terminal.open` / `.close`. `export.*` / `import.*`. `profile.encrypt` / `profile.decrypt`. When an operation has an inverse, the inverse should be findable by guessing, and a surface that offers `connect` and `teardown` has spent a script author's attention for nothing.

The list of a noun is the noun's list. `collection.list`, `transfer.list`, `history.list`, `account.list`, `contact.list`, `environment.list`, `drives.list`. A script author who learns the convention in one app knows how to enumerate in all of them, and a new app that names its enumeration `all` or `enumerate` or `fetch_many` has broken a transfer of knowledge that cost nothing to keep.

Formats become verbs, not parameters. `export.bibtex`, `export.ris`, `export.marxml`; `calc.csv`, `calc.html`, `calc.markdown`, `calc.pdf`. The alternative — one `export` verb with a `format` parameter — would be a smaller surface and a worse one, and it is worth understanding why the shipped surfaces went the other way. A verb per format is enumerable: `verbs()` reports exactly which formats this build actually supports, the completion of Chapter 38 offers them at the prompt, and the catalog of Chapter 17 documents them without anyone writing a list of legal `format` values. A `format` parameter hides all of that inside a string whose legal values live only in the `run` body and in prose that will drift. It also lets each format carry its own parameters — a format that needs a style and one that needs a delimiter are different signatures, and one verb with a union of both parameters is a signature that lies about half its arguments. The cost is verb count, and verb count is cheap; the benefit is that the surface documents its own capabilities, and that is the property the whole design is built to protect.

Planning verbs are named for planning. `sync.plan`, `transfer.multipart_plan`, `vpn.killswitch_plan`, `split.route_plan`. The suffix is a shipped convention with real semantic content: a `_plan` verb computes and returns a description of what would happen and changes nothing. That convention is what makes the plan-then-act shape of Chapter 25's mirror recipe and the dry run of Chapter 30's migration expressible at all, and it is worth adopting deliberately in a new app, because a surface where “compute the change” and “make the change” are separate verbs is a surface a cautious operator can trust. A verb that plans should never act, and a verb that acts should not be the only way to find out what it would do.

Events are past-tense. `itemAdded`, `downloadComplete`, `requestSent`, `noteCreated`. An event names something that already happened, which is exactly what distinguishes it from a verb, and the tense carries that distinction into the name so that a reader of a subscription list never has to check which kind of thing they are looking at.

The two casings, and where they come from

An honest reading of the surface finds an inconsistency worth explaining rather than smoothing over: verb ids are dotted and snake-cased at the leaf — `collection.add_item`, `imap.build_search`, `s3.list_objects` — while state query and event ids are bare camelCase — `activeCollection`, `readinglist`, `activeEnvironment`, `itemAdded`.

The split is not a designed rule; it is a fingerprint of where each name is born. Verb ids are dominated by the engine's `COMMANDS` lists — Rust-side identifiers, snake-cased by the convention of the language they live in, and namespaced because a flat list of several hundred commands needs the structure. State and event ids are registered in the frontend's `ZGui.automation.register` call, in JavaScript, where camelCase is the language's own convention and where the sets are small enough that no namespace has been needed. Two registries, two host languages, two casings.

The practical guidance is to follow the local convention rather than to unify, because consistency *with the neighbors a name will actually sit beside* is what a script author's memory keys on. A new engine verb should be snake-cased and namespaced like the engine verbs around it; a new state query should be a camelCase noun like the state queries around it. Naming a state query `active_environment` because verbs use underscores would make it the only one of its kind, which is a worse outcome than the inconsistency it was trying to fix. Where a genuine unification is wanted it belongs in a versioned surface change with aliases, per Chapter 22, not in a single app quietly diverging.

Four ways to name a surface badly

The anti-patterns are as instructive as the rules, and each has a tell.

Naming after the UI. A verb called `sidebar.refresh` or `dialog.confirm_import` is named after a widget, and widgets are the least stable thing in an application. The frontend gets redesigned and the verb is either renamed — breaking scripts — or kept, now describing a control that no longer exists. Name the capability, not the control that currently invokes it.

Encoding a workflow. A verb called `import_and_file_and_notify` bundles three decisions the script should be making. It is un reusable by construction: any script that wants two of the three has to either accept the third or not use the verb. The surface should expose capabilities and let scripts compose workflows, which is the entire premise of the bus — composition is the caller's job, and a verb that pre-composes has taken the job away.

The mode parameter. A verb whose behavior forks on a string argument — `run` with a `mode` of `"plan"` or `"apply"` — collapses two verbs into one and hides the more important of the two from `verbs()`, from the completion, and from the catalog. The enumerable surface is the documentation; a parameter that changes what a verb *is* rather than what it *operates on* is a capability hidden inside a string.

Renaming for elegance. The most tempting and the most expensive. A verb whose name is merely imperfect costs a script author a moment of confusion once; a verb that is renamed costs every existing script a break. Per Chapter 22, the deprecation path exists — keep the old id as an alias dispatching to the same `run`, and drop it after the scripts have moved — but it is a cost, and the cheapest time to pay for a good name is before the first script is written against it. Which is the chapter's whole argument compressed: the ids are the contract, the contract is the durable artifact, and the twenty minutes spent choosing them is the highest-leverage twenty minutes in putting an app on the bus.

Chapter 43 – What the Bus Replaces

The comparison a practitioner actually makes

Chapter 11 compared the bus against its prior art to answer a question about novelty, and answered it conservatively. This chapter asks a different and more useful question: for a given automation you want to write today, what would you have written instead, and what would it have cost? That is the comparison a working engineer makes, and it is not the same as the novelty comparison, because the alternatives a person actually reaches for are mostly not the historical systems Chapter 11 weighed. They are five much more ordinary things, and each is worth walking with the specific automation of Chapter 16's invoice run in mind — export a billing range, stamp it in a PDF engine, mail it — since that run is concrete enough to price.

Against driving the pixels

The first alternative is to automate the GUI as a human would: move the pointer, click the menu, type into the dialog. This is what `stryke-gui` does, and Chapter 33 was careful to call it complementary rather than inferior, which remains true — it is the only option for an app that is not on the bus. As the way to write the invoice run, though, it is worse in four specific ways, and they are worth enumerating because they are the four things the bus exists to fix.

It has no return value. A click produces a screen, not a result, so the script cannot learn the path of the file the export produced; it must guess, or watch a directory, or read the screen. It has no completion signal, so every step is followed by a wait that is either too short or too long — the blind sleep that Chapter 34 named as an anti-pattern, here forced by the medium rather than chosen. It is coupled to layout: the automation encodes where things are, and a redesign that moves a menu item breaks a script that was otherwise correct. And it is coupled to focus and to the machine's state; a script driving the pointer cannot share the machine with the person who owns it.

The bus's answer to all four is one answer: the verb returns. `$out->{path}` is the export's own account of what it produced, delivered when the export actually finished, expressed in terms of the app's domain rather than its layout, over a socket that does not care which window has focus. Every one of the four defects is the missing return path in a different costume.

Against a command-line surface per app

The second alternative is the one the `bash` step implies: give each app a CLI and shell out to it. Chapter 1 recorded why that step was a dead end — “a bash step cannot call an app verb because there is no command-line surface for the app's verbs to call” — but the deeper point is what it would have cost to build one, per app, and what it still would not have bought.

Each app would need its own argument parser, its own output format, its own error conventions, and its own decision about how a command-line invocation reaches the running instance that holds the live state. That last problem is the one that makes the approach collapse: a CLI is a new process, and the thing the script wants — the current selection, the open document, the active environment — lives in the running GUI process, not in the new one. So either the CLI is useless for anything stateful, or it needs an IPC channel to the running app, at which point it has become a worse client of the bus that would then have had to exist anyway. Building eleven

CLIs is building eleven half-buses, each with its own wire, plus eleven argument parsers, plus a subprocess spawn per call.

Against that, the bus is one wire, one codec, one module, no argument parsing at any layer because arguments are typed JSON, and no process spawned per call — the discipline Chapter 11 named as the speed leg. And the reach is strictly larger, because the enumerable surface is the whole app's registered verb set rather than whatever subset an author remembered to wire into a `main`.

Against reading the app's data behind its back

The third alternative is the one that tempts every experienced engineer and is the worst of the five: skip the app and read its store. The reference manager keeps its library somewhere; open that file, parse it, and answer the question directly. The reconciliation recipe of Chapter 16 — what is on the server that the library does not track — looks especially inviting this way, since both sides are ultimately files.

Three things are wrong with it and only one is obvious. The obvious one is that the store's schema is not a contract: it is a private implementation detail that the app is free to migrate on its next release, and a script coupled to it breaks on an upgrade that broke nothing else. The second is that a running app holds state that the store does not yet reflect — the current selection is not in any file, and neither is anything the user has changed but not saved — so the file answers a question about a slightly different world than the one the user is looking at. The third and most dangerous is writing: a script that modifies the store behind a running app's back is racing the app's own writes and defeating its in-memory caches, and the failure mode is silent corruption discovered later.

The bus's alternative is to ask the process that knows. `fs.list` and `library.search` are answered by the code that owns the data, against its live state, through a contract that is versioned and enumerable. This is the same reasoning that made the manifest the source of truth for documentation in Chapter 17, applied one layer down: do not build a second representation of a thing that can be asked directly, because a second representation is a thing that can be wrong.

Against clipboard and temp-file choreography

The fourth alternative is the one most people have actually done: run the steps by hand and move the values between apps with the clipboard and the filesystem. Export to a file you choose, open the file in the next app, save it somewhere you will remember, attach it in the third app. The invoice run written this way is not automation at all; it is a checklist.

It is worth listing because it is the honest baseline for a great deal of real work, and because it exposes what the bus is really replacing. The clipboard and a temp directory are an integration bus — an untyped, unlogged, single-item one whose correlation mechanism is the operator's memory. The bus replaces the operator's memory with a variable. `$out->{path}` leaving the office suite and entering the PDF engine is the file handoff that used to be a person choosing a folder, and `$final->{path}` becoming a mail attachment is the drag that used to follow it. Nothing about the apps changed; the choreography moved from a person's hands into a file that can be read, reviewed, versioned, and run at three in the morning.

Against putting an HTTP API on every app

The fifth alternative is the modern instinct, and the design FAQ of Chapter 36 already declined it on transport grounds. From the practitioner's side the accounting is a little different and sharper. An HTTP surface per app means a port per app, and ports mean an allocation scheme, a conflict when two apps want the same one, and a discovery mechanism to tell a client which app got which — all problems the socket directory of Chapter 6 does not have, because the filename is the name. It means a bind that is reachable from off the machine unless carefully constrained, which is a security posture to maintain rather than a property to have. It means

mapping app-level failures onto status codes and back, where the bus carries `ok:false` with the app's own message. And it means a client library per language, where the bus has one module in the one language it is for.

What the HTTP version would buy is reach the bus explicitly refuses — other machines, other languages, other runtimes. That is not an accident of the design and it is not free to want: Chapter 18 argued that a cross-machine surface is a different product with a different threat model, and taking on that threat model to automate one's own desktop is paying a real price for capability the use case does not need.

What the bus does not replace

An honest comparison names the cases where the alternatives still win, and there are three. An app that is not on the bus cannot be reached by the bus, and `stryke-gui` remains the only tool for it — physical automation is the universal solvent precisely because it requires nothing of its target. Work that is genuinely exploratory is still faster in the GUI: the bus is for repeating a decision, not for making one the first time. And the escape hatches remain escape hatches — when the thing needed is not a verb and should not become one, the `js` and `bash` steps of Chapter 10 are the sanctioned answer, and their existence is what lets the allow-list stay strict everywhere else.

The pattern across all five comparisons is one property with five faces. Pixel automation lacks a return value; a per-app CLI lacks access to live state; reading the store lacks a contract; the clipboard lacks correlation; HTTP has all three and charges a threat model for them. The bus is the smallest thing that supplies a typed return, from the live process, under a versioned contract, correlated to the call, without leaving the machine — and every alternative in this chapter is missing exactly one of those and paying for the absence somewhere else.

Chapter 44 — Long Work: Jobs, Progress, and the Cancellation Gap

The one assumption the call model makes

Every chapter so far has treated a verb as something that finishes. `App:call` blocks, the verb runs, a reply comes back, the script continues — and the whole failure model of Chapter 14 is built on the premise that the reply arrives promptly or the deadline fires. That premise is correct for the overwhelming majority of the census. A search returns rows, a state read returns a map, an export writes a file and hands back a path, and every one of them is measured in the milliseconds or the low seconds where a blocking call is not merely acceptable but exactly what the script author wanted.

It is not correct for everything. Some work in this suite is genuinely long. A multipart upload of a large archive to an object store, an OCR pass over a several-hundred-page document, a full-text reindex of a large library, a sync comparison across a remote tree with tens of thousands of entries — these are minutes, not milliseconds, and a script that blocks on them for the whole duration is holding a connection open across a span in which the app may be restarted, the network may drop, and the user may reasonably want to know what is happening. The call model does not break for these; it simply stops being the right shape. This chapter is about the shape that replaces it, and about the one thing the bus deliberately does not give you to build it with.

What the frame set does not contain

Read the frame reference of Chapter 28 looking specifically for a way to stop something and the absence is immediate: there is no cancel frame. A client can `call`, `get`, `verbs`, `sub`, and `unsub`. It cannot reach into a call already in flight and revoke it. Once a `call` frame has been accepted and dispatched, the only things that end it are the verb resolving, the verb rejecting, the deadline firing, or the connection breaking — and of those four, only the last is under the caller's control, and it is a blunt instrument that takes every other outstanding call and every subscription on that connection down with it.

The absence is deliberate, and it follows from the layer prohibitions of Chapter 2 rather than from anyone deciding cancellation was unimportant. Consider what a generic cancel frame would have to do. It arrives naming an `id`; the transport must route it not to a handler but to a *running verb body*, which means the transport would have to know that a verb body is a thing that exists and can be interrupted — a violation of the rule that Layer 4 must not interpret a frame beyond its `t`. And on the far side, the verb's `run` would have to be handed something it could check — a cancellation token, a flag, a signal — which means Layer 1's registration would have to grow a parameter that exists only because a transport exists, violating the rule that the surface must not know a transport is there at all. A generic cancel cannot be added without dissolving two of the five prohibitions that make the stack what it is.

There is a second reason, and it is the more practical one. Cancelling generic work is not generically meaningful. Stopping an upload halfway leaves a partial object; stopping a reindex halfway leaves an index in an unknown state; stopping an OCR pass halfway may or may not have written pages to disk. What “cancelled” means — what is rolled back, what is kept, what is left for a resume — is domain knowledge that only the engine implementing the work has. A transport-level cancel would deliver a uniform *request* and produce eleven different and undocumented *outcomes*, which is worse than not offering it, because a script author would reasonably assume the uniform request implied a uniform result.

Why cancellation is a verb

So cancellation, where it exists, is a verb — and the census already shows it, because the app authors reached the same conclusion from the other side. `zftp` declares `transfer.cancel` alongside `transfer.add`; `ztunnel` declares `vpn.disconnect` alongside `vpn.connect`. Those are not gaps the bus failed to fill; they are the correct filling. `transfer.cancel` is a verb whose implementation knows what a half-finished transfer is, whether the partial file is kept for a `transfer.resume_check` or discarded, and what the transfer's status becomes. It returns a typed value saying so. It is documented in the surface, discoverable through `verbs()`, and validated at the boundary like every other call.

Read that way, the “cancellation gap” is not a gap in the bus at all — it is a statement about where the responsibility sits. The bus's contribution is that `transfer.cancel` is callable from a script *at all*, which before the bus it was not. What the bus declines to do is invent a cancel for verbs whose authors did not write one, which it could only do by guessing at semantics it has no access to.

The job shape, stated

Once cancellation is a verb, the rest of the long-work design follows, and it is a shape any app author can implement out of the three entry kinds Chapter 3 already provides. The shape is this: *a long verb does not do the work; it starts the work and returns immediately with an identifier*. Progress is an event. Status is a state query. Cancellation and completion are further verbs, keyed by the identifier.

Each of those pieces earns its place. The starting verb returns fast, so the blocking call model is honoured rather than fought: the script's `call` completes in the milliseconds it takes the engine to enqueue the work, the deadline of Chapter 14 never comes near firing, and the connection is not held open across a span where anything can happen to it. The identifier is what makes every subsequent operation addressable — without it there is no way for a second call to name the first call's work, since the wire's own `id` is a correlation token for one request and dies with its reply. The event stream is what makes progress observable without polling, and it costs nothing when nobody subscribes, per the empty-subscription accounting of Chapter 3. And the state query is what makes progress observable *for a script that started after the work did*, or a script that reconnected after a restart, because a state read is answerable at any moment while an event is only delivered to whoever was listening when it fired.

A job-shaped verb, worked

The transfer client is the clearest case in the census, because the transfer namespace is already built around an addressable unit of work — `transfer.add` enqueues one, `transfer.cancel` stops one by id, `transfer.resume_check` asks about one. A script that mirrors a directory to a remote store enqueues each transfer, watches whatever progress and completion events that build declares, and reports — and never blocks on the bytes.

```
#!/usr/bin/env stryke
use App

val $ftp = App::open("zftp")
val %live = %{} # id -> the path it is moving
val $open = 0

$ftp->on("transferProgress", fn ($t) {
  if %live{ $t->{id} } {
    p "  ${ %live{ $t->{id} } }: ${ $t->{percent} }%"
  }
})

$ftp->on("transferDone", fn ($t) {
  p "done ${ %live{ $t->{id} } } (${ $t->{status} })"
  $open -= 1
})

for val $f (@{ lines(read_file("upload.list")) }) {
  val $job = $ftp->call("transfer.add", %{ path => trim($f) })
  %live{ $job->{id} } = trim($f)
  $open += 1
}

# the enqueue loop is over in milliseconds; the work is not
loop {
  App::pump()
  if $open == 0 { last }
  App::sleep(500)
}
```

The two event ids in that script are placeholders for whatever the app's own surface declares — the point is the shape, and a script author reads the real names out of `verbs()` before writing the first `on`.

Three properties of that script are worth stating because none of them is available in the blocking shape. The enqueue loop finishes almost immediately, so a hundred files are queued before the first one has finished moving, and the app schedules them according to its own concurrency policy rather than the script's serial pacing. The script's own progress reporting is live, arriving on the app's schedule and drained at the pump the script controls, exactly per Chapter 15 — the pump discipline is not a special accommodation for jobs, it is the same discipline every watcher uses. And the termination condition is a property of the script's own bookkeeping rather than of any single call's return, which means the script can decide to stop waiting, report what is still outstanding, and exit — leaving the transfers running, because they were never the script's to hold.

That last point is the philosophical difference between the two shapes and the reason the job shape scales. In the blocking shape, the script's lifetime and the work's lifetime are the same lifetime; kill the script and you have an undefined amount of completed work and no record of it. In the job shape they are separate: the work belongs to the app, the script is an observer that happens to have started it, and the script dying is an event of no consequence to the work at all. Chapter 41's whole discussion of restart and resumption becomes far easier under this shape, because the question "what did my last run actually finish?" has an answer the app can give — read the status of each job id — rather than an inference the script has to reconstruct.

Cancelling, and what a cancelled job leaves behind

With jobs addressable, cancellation is an ordinary call, and the interesting part is not the call but the aftermath.

```
for val $id (@{ keys %live }) {
  try { $ftp->call("transfer.cancel", %{ id => $id }) }
  catch ($e) { p "could not cancel ${ $id }: ${ $e }" }
}
```

The `try` is there because cancelling is racy by nature: the job may have completed between the script deciding to cancel it and the frame arriving, in which case the app's honest answer is a rejection naming an id it no longer considers live. That is not a failure to handle so much as a fact to record, which is why the catch prints rather than dies.

What the cancel *leaves* is the app's business, and a script must not assume. A transfer client that supports resume will likely keep a partial file and mark the transfer resumable, which is exactly what `transfer.resume_check` exists to interrogate. A different app might discard everything and return to a clean state. The bus carries the request and the reply faithfully and takes no position, and the guidance for a script author is the guidance of Chapter 41 restated for this case: do not assume a cancelled job left nothing behind; read the state that describes it, and act on what you read.

The three questions an app author must answer

An author adding a long-running capability to a surface has a design decision to make before writing a line of the `run` body, and three questions settle it.

Is this long enough to be a job? The threshold is not a number, it is a relationship: the work is job-shaped when its duration is long enough that a caller would reasonably want to observe it, survive a restart during it, or stop it. Work that finishes faster than a user would notice should stay a plain blocking verb, because the job shape costs the caller three more concepts and buys nothing when the work takes a moment.

What is the identifier, and how long does it live? An id that is valid only while the job runs makes post-hoc inspection impossible; an id that persists after completion lets a script started an hour later ask how the run went. The second is more useful and costs a retained record. Whichever is chosen must be documented in the surface, because a script author reasoning about resumability is reasoning about exactly this lifetime.

Is the progress event or the status query the primary channel? Both should exist, because they answer different questions — the event says “something just changed”, the query says “here is the current truth” — but one of them will be the one scripts actually use, and it should be the state query. An event stream is the optimization for a script that is already watching; a state query is the thing that works for every script, including one that started late, reconnected, or simply prefers a poll. An app that ships only the event stream has built a job surface that a restarted script cannot rejoin.

Where the job shape does not apply

Two cases resist it and are worth naming so nobody contorts a surface to fit.

The first is work that is long only because it is *waiting*, not because it is computing — a verb that blocks until a condition holds. That shape should not exist as a verb at all. The condition belongs in a state query and the waiting belongs in the script, which is precisely the `wait_for` helper of Chapter 27: the script decides the interval, the ceiling, and what to do when the ceiling is reached, and no app has to guess any of the three on the script's behalf. A verb that waits has taken a policy decision away from the only party qualified to make it.

The second is work whose duration is long but whose result is a single value the script cannot proceed without. Here the job shape adds ceremony for nothing: the script starts the job, then immediately polls until it finishes, then reads the result — which is a blocking call written out in four steps. If the caller genuinely cannot continue until the answer arrives, blocking is not a compromise, it is an accurate expression of the dependency, and the correct response to a long wait is a generous deadline rather than a restructured surface.

The honest gap that remains

Stating all of that leaves one real limitation standing, and it should be named rather than argued away. A verb whose author did not write a cancel cannot be cancelled by a script, at any granularity, short of breaking the connection — which does not stop the work, it only stops the script from hearing about it. If an app ships a long verb with no job shape and no cancel, a script that starts it has committed to it. The bus offers no recourse, and the reason it offers none is the reason this chapter opened with: building a recourse would require the transport to reach into a running verb, which is the one thing the layering forbids it to do.

That limitation is a real cost of the design, and the design accepts it on the grounds that the alternative is worse. A generic cancel that worked unevenly across eleven apps and left each one in a state its own author had not thought about would be a facility scripts could not trust, and an untrustworthy facility in an automation stack is worse than a missing one, because a missing facility forces the script author to solve the problem where the knowledge is. The gap is a pointer at an app author's to-do list, not a hole in the wire.

Chapter 45 – The Filesystem as the Second Wire

Two carriers, one orchestration

Chapter 13 ended its accounting of large values with a rule stated in one line: pass paths, not contents. That rule is correct and this chapter takes it seriously enough to work out what it actually commits a script author to, because “pass a path” is a sentence with an enormous amount of operational discipline hiding inside it. The moment two apps exchange a filename, a second carrier has entered the orchestration — the filesystem — and unlike the socket it has no framing, no correlation, no reply, and no lifetime management whatsoever. It is a shared mutable namespace that both apps and the script can write to, that outlives all three, and that nothing cleans up.

The bus is deliberately silent about it. There is no bus-owned temp directory, no handle type representing a file, no verb the bus itself contributes for transferring bytes. The design's position is that the filesystem is already a perfectly good mechanism for moving bulk between processes on one machine, that reinventing it inside a newline-JSON protocol would produce a worse version of it, and that the right amount of bus machinery for file exchange is none. That position is defensible, and it is also an assignment of work to the script author, which is what this chapter is about.

Who owns the file

Every file that crosses between two apps has exactly one owner at any moment, and most of the confusion in file-passing orchestrations comes from that ownership being implicit. Make it explicit and the discipline follows.

The producing verb owns the file until it returns. A render verb such as `calc.pdf`, given an output path, is until its reply arrives, the only party entitled to touch that path — it may write it incrementally, it may write to a neighbouring temp name and rename at the end, it may truncate and rewrite on a retry. A script that reads the path while the call is outstanding is reading a file mid-write, and the fact that it can is a consequence of the filesystem's lack of framing, not permission to do so.

Ownership transfers to the script at the reply. Once the producing call has returned, the script holds the file: it may read it, hand its path to another app, move it, or delete it. This is the moment the value model of Chapter 13 has actually delivered something — not the bytes, but the right to name them.

Ownership transfers again, temporarily, when the script hands the path to a consuming verb, and here the important question is whether the consumer *mutates in place*. A verb that reads a file and produces a new one leaves the original untouched and returns ownership at its reply. A verb that edits the file at its given path has consumed the original, and the script's copy of the path now names something different from what it named a moment ago. The surface should say which, and where it does not, a script should assume mutation and keep its own copy if it needs the original.

The rule that falls out is short: *the script is the owner between calls, and never during one*. Every read, every move, every deletion happens in the gap between a reply and the next call, which is exactly where a straight-line stryke script naturally does its work anyway.

Where the file goes

A script choosing an output path is making a decision with three constraints, and the constraints conflict often enough to be worth setting out.

The path must be writable by the producing app, which on this suite is trivially true — every app runs as the same user as the script, per the trust boundary of Chapter 10, so there is no sandbox to negotiate and no permission to arrange. This is one of the quiet benefits of the security model being a single-user model: file exchange needs no brokering because there is only one uid in the picture.

The path should be predictable enough to clean up and unique enough not to collide. A run that writes to a fixed name has an obvious collision problem the moment two runs overlap, and a run that writes to a fully random name in the system temp directory leaves litter that nobody can attribute later. The shape that satisfies both is a per-run directory under a known root — a directory whose name encodes the script and the run, containing every file that run produced. Cleanup becomes one removal of one directory, and forensic inspection after a failed run becomes possible because the files are still there and still identifiable.

And the path should be on the same filesystem as the file's eventual destination if the orchestration ends in a move, because a rename within a filesystem is atomic and cheap while a rename across one is a copy with a window in the middle where the destination exists but is incomplete. This is ordinary systems knowledge, but it becomes a bus concern the moment a script's last step is “put the finished artifact where the user expects it”, which is the last step of a great many reporting runs.

Atomicity belongs to the producing verb

A script cannot make a verb's file output atomic, in exactly the sense that Chapter 41 established that a script cannot make a verb idempotent. If `calc.pdf` writes its output progressively to the path it was given, then anything watching that path sees a partially-written PDF for the duration, and no discipline on the script's side changes that — the script is blocked on the call and has no opportunity to intervene.

So atomicity is an app-author property, and it is worth stating as surface design guidance in the same breath as idempotence. A verb that produces a file should write to a temporary name in the destination directory and rename onto the target as its last act before returning, so that the target path never exists in a partial state and any observer sees either the old file or the complete new one. The cost is one rename. The benefit is that a second script watching that directory, a backup running on a timer, or the user opening the file the moment it appears all see a coherent artifact.

For a script author, the practical consequence is a rule about what to trust. The *reply* is the completion signal, not the file's appearance. A script that waits for a path to exist rather than for the call to return has substituted a filesystem event for the return path the entire book is about, and it will observe partial files on exactly the runs where it matters. The file appearing means the producer started; the call returning means the producer finished.

Cleanup, and the run that dies

Here is the part that is genuinely awkward, and pretending otherwise would be dishonest. A script that dies mid-run — because a call failed and the failure model of Chapter 14 turned it into a death — leaves behind every intermediate file it had produced up to that point. There is no destructor that fires, no scope guard that runs, and no bus machinery that knows the files existed at all, because the files were never the bus's to know about.

Three responses are available and they rank clearly.

The best is the per-run directory of the previous section, combined with a sweeper that runs on a schedule and removes run directories older than some age. This does not prevent the litter; it bounds it, and it does

so without requiring the failing run to have done anything correct on its way down, which is the property that matters, because a run that died is precisely a run whose cleanup code did not execute.

The second is an explicit cleanup path in the script, wrapping the fallible section in a `try` and removing the intermediates in the catch before re-raising. This works and it is worth doing for a script whose intermediates are large enough to matter, but it is worth being clear-eyed that it handles only the failures that arrive as catchable errors. A script killed at the terminal does not run its catch.

The third, and worst, is to write intermediates into the system temp directory and rely on the operating system's own eventual cleanup. This is not wrong so much as it is unattributable: it produces files nobody can trace to a run, and it makes debugging a failed orchestration harder at the exact moment the intermediates would have been the most useful evidence available.

The through-line is that the failure of a file-passing orchestration is a *messier* failure than the failure of a pure-socket one, and that this is a real cost of using the second carrier. A run that exchanged only JSON leaves nothing behind when it dies. A run that exchanged files leaves exactly as much as it had produced, and the design work is in making that residue bounded and identifiable rather than in pretending it will not occur.

A path is a machine-local value

One more property of the second carrier deserves its own statement, because it connects to a refusal made much earlier in the book. A path is meaningful only on the machine whose filesystem it names. A socket frame carrying `{"path": "/tmp/run-17/bill.pdf"}` is a perfectly valid frame that would be perfectly meaningless if the two ends of the socket were on different machines.

That is not a limitation the bus is working around; it is one more reason the remote temptation of Chapter 18 is refused. A cross-machine bus would have to answer, for every verb in the census that takes or returns a path, what a path means when the caller and the callee do not share a filesystem — and the answers are all bad. Either paths become opaque handles the remote side resolves, which means inventing a distributed file namespace; or bulk starts crossing the wire after all, which reintroduces exactly the problem path-passing was adopted to avoid; or the surface splits into local-only and remote-safe verbs, which fragments the one contract the whole design exists to keep uniform. The single-machine constraint is what makes the filesystem usable as a second carrier, and the filesystem being usable as a second carrier is what keeps the wire small and legible. The two decisions hold each other up.

The invoice run, re-read as ownership

Chapter 16's three-app invoice pipeline is the canonical file-passing orchestration in this book, and it repays a second reading with ownership in mind rather than data flow.

The office engine renders a billing range and returns a path. At that moment the office engine has finished with the file and the script owns it; nothing in the office engine will touch that path again unless the script calls it again. The script then hands the path to the PDF engine to flatten, watermark, and protect. During that call the PDF engine owns the file, and whether the script's original path still names the unstamped document afterwards depends entirely on whether that verb mutates in place or produces a new artifact — which is the single most important thing the surface can tell the script author about that verb. The script then hands a path to the mail client as an attachment, and the mail client reads it during the call and takes no ownership beyond that, because reading is not owning.

Three apps, three ownership windows, and between them three moments where the script alone holds the file and could inspect it, copy it, or abandon it. Read that way the orchestration is not “three verbs that pass a filename around” but a sequence of custody transfers, each bracketed by a call, each with a defined owner. That framing is what makes the run debuggable: when the output is wrong, the question is which custody window it went wrong in, and the intermediates are still on disk to answer it — provided the script wrote them somewhere it can find.

The rule, stated

The division of labour between the two carriers is simple enough to state in one sentence and worth holding as a design rule for both surface authors and script authors: *the wire carries what the script reasons about; the filesystem carries what the script only moves*. An identifier, a status, a count, a row, a decision — those cross the socket, because the script is going to branch on them. A rendered document, an exported archive, a captured image — those cross the filesystem, because the script's entire relationship with them is to name them to the next app. When a script finds itself decoding a large value only to hand it straight to another verb unexamined, it has put bulk on the control plane, and the fix is a verb that takes a path. When a script finds itself parsing a file off disk to make a decision, the app should probably have returned that decision as a value, and the fix is a verb that returns one.

Chapter 46 – The Catalog: Generation, Diffing, and the Gate

What generation buys that registration cannot

Chapter 17 established that `GUI_SCRIPT_ACTIONS.md` is generated from source rather than written by hand, and gave the reason in one sentence: prose that claims a count is a snapshot that starts drifting the instant it is written. That is the argument for generation over authorship. There is a second argument, less obvious and more important, and it is the argument for generation over *introspection* — for reading the source rather than asking a running app.

The bus already has a perfectly good runtime answer to “what verbs does this app have”: `verbs()`, which returns the live manifest the app booted with, stripped of bodies. A catalog could be built by launching all eleven apps, dialling each socket, calling `verbs()`, and writing the union to a file. That would be more accurate than reading source, since it reports what the app will actually answer to rather than what its source appears to declare.

It is also unbuildable in the place a catalog most needs to be buildable. A generated artifact that requires a running desktop requires a display, a window server, eleven launched applications, and every one of them healthy at the same moment. It cannot run on a machine with no graphical session. It cannot run as part of an ordinary build. It cannot run at all if one of the eleven fails to start, and a catalog that silently omits an app because that app crashed on launch is worse than no catalog. The source-reading generator has none of those dependencies: it reads files, it produces a document, and it does so identically on a developer’s machine and on a headless box. That is the same instinct as the first two levels of Chapter 21’s test pyramid, which deliberately test the surface and the transport without a desktop, for exactly the same reason.

The trade is stated plainly: the catalog is a projection of the *source of truth as written*, not a recording of *the process as running*, and the next sections are about what that trade costs and how the cost is paid.

Three shapes of app, three extractions

The generator cannot have one extraction strategy because the apps do not have one shape, and Chapter 8’s census already sorts them into the three the generator must handle.

The first shape is the **engine-backed app**, which is most of them. Its verb list is a Rust constant — `commands::COMMANDS` in the `*-core` crate — and extracting it means reading a source-level list of string literals. This is the easiest of the three and the most reliable, because the same constant is the one the hybrid handler’s `is_engine_cmd` tests against at run time, so what the generator reads and what the app dispatches on are literally the same array. Divergence is not merely unlikely here; it is structurally impossible while the constant has one definition.

The second shape is the **host-declared surface**, where an app’s `bus.rs` carries its own array of verb ids for discovery. `zwire` is the example Chapter 7 already named, because it is also the case where discovery and dispatch deliberately differ: the host lists a `SURFACE_VERBS` array, and the comment on that array records that `call` additionally accepts any `browser.<verb>`, since the Chromium HUD’s own executor runs it. The generator counts what the array lists. It does not and cannot count the wildcard, because a wildcard

namespace has no enumeration — and this is the first place the catalog's number is provably a floor rather than a measurement.

The third shape is the **frontend catalog**, and traderview is the case that forced the generator to learn it. That app has no `-core` engine at all; its whole surface is registered by its webview, and what it registers is the `%%K` palette catalog — every `view:` tile and every `action:` shortcut. Extracting that means reading the tile table and the shortcut list out of the frontend source, which is enumerable, and it means knowing which entries to leave out: the recents and the favourites are ephemeral palette entries reflecting what the user did lately, not durable verbs, and a generator that counted them would produce a number that changed with usage rather than with code. Filtering them is a judgement the generator makes on purpose, and it is the second place the catalog's honesty is a deliberate act rather than an automatic property.

On top of all three, the shared `appshell.*` verbs are added once per app rather than extracted per app, because every app inherits them from `zgui-core` without declaring them. That is a correctness requirement, not a shortcut: a generator that looked for those verbs in each app's own source would find them nowhere and report every app as missing the surface it definitely has.

What the generator refuses

The most important behaviour in the generator is a refusal, and Chapter 8's closing section named it: for the coreless apps that register no comparable frontend catalog, the generator reports the shared shell surface plus their backend commands and stops. It does not estimate. It does not extrapolate from how large the app looks. It does not mark them as “surface unknown, presumably large.”

That refusal is what makes every other number in the catalog worth reading. A generator that estimated once would have to be trusted about which numbers were estimates and which were counts, and the only way to maintain that trust is to inspect the generator — at which point the catalog has stopped being a source of truth and become a claim requiring verification. A generator that never estimates needs no such inspection: every figure it prints is a count of things it found, and a small figure means a small enumerable surface rather than a small app. The distinction is stated in the catalog itself so a reader is not left to infer it.

There is a general principle here that applies well beyond this generator. An automatically produced number is only as valuable as the reader's confidence about what it counts, and the fastest way to destroy that confidence is to make the counting rule conditional. One rule, applied everywhere, producing a floor, is worth more than a cleverer rule that produces a closer estimate and requires a paragraph of caveats to interpret.

The divergence the generator cannot see

Reading source rather than a running process buys headless reproducibility and costs one specific kind of accuracy, and the honest thing is to name exactly what kind.

The generator reports what a source file declares. The app reports what it registered at boot. For the engine-backed shape those are the same array and cannot differ. For the other two they can, in a handful of realistic ways. A frontend verb registered conditionally — inside a branch on a feature flag, a platform check, or a plugin's presence — is declared in source and may not exist at run time. A verb registered dynamically, from data rather than from a literal, exists at run time and may be invisible to a generator reading literals. A host array that has drifted from what the handler actually routes is declaration and dispatch disagreeing, which is the case Chapter 7's discovery-versus-dispatch asymmetry describes as normal for `zwire` and would describe as a bug anywhere else.

None of those is hypothetical enough to ignore, and the design has an answer that costs almost nothing, because both halves already exist. The catalog is a list of ids. A running app will hand any script its list of ids. Comparing them is a script:

```
#!/usr/bin/env stryke
use App

val %declared = %{}
for val $l (@{ lines(read_file("GUI_SCRIPT_ACTIONS.md")) }) {
  val $id = catalog_id($l)      # pull the verb id out of a catalog row
  if $id { %declared{ $id } = true }
}

for val $name (@{ App::list() }) {
  val $h = App::open($name)
  val %live = %{}
  for val $v (@{ $h->verbs()->{verbs} }) { %live{ $v->{id} } = true }

  for val $id (@{ keys %live }) {
    if !%declared{ $id } { p "${ $name } : live but uncatalogued: ${ $id }" }
  }
  for val $id (@{ keys %declared }) {
    if !%live{ $id } { p "${ $name } : catalogued but absent: ${ $id }" }
  }
}
}
```

That script is the conformance check the generator cannot perform on itself, and its shape is worth noticing: it is a bus script, driving the bus, to audit a document about the bus. It needs the running desktop that the generator was designed not to need — which is precisely why it is a separate thing run occasionally by a human rather than a step in the build. The generator runs everywhere and reports the declaration; the conformance check runs where a desktop exists and reports the divergence; and between them the question “is the catalog true?” has an answer that does not depend on anyone’s memory.

The two directions of the comparison mean different things and should be read differently. *Live but uncatalogued* is usually a generator gap — a registration shape it does not know how to read — and is mostly harmless, since the catalog was always a floor. *Catalogued but absent* is the serious one: it means a script author reading the catalog will write a call the app rejects, which is the catalog actively lying rather than merely under-reporting.

Diffing two catalogs

Because the catalog is generated, two generations of it are comparable, and the comparison is the mechanism Chapter 22’s versioning rules were waiting for. Chapter 22 states the policy — adding is safe, removing and renaming break — but a policy without a mechanism is a thing people intend rather than a thing that happens. The mechanism is a diff of two id sets, and it classifies into exactly three buckets.

Ids present in the new catalog and absent from the old are additions. Every one of them is safe by the rule, because a script that did not call a verb yesterday does not call it today. No review is required, and the count of them is the most useful single number for a release note: it is how much the surface grew.

Ids present in both are unchanged at the id level, which is the level the diff can see. This is where the mechanism’s limit sits and it must not be glossed: two catalogs agreeing that `library.search` exists says nothing about whether its parameter list gained a required argument or its return type dropped a field, both of which are breaking changes the rule of Chapter 22 explicitly names. If the catalog records signatures — the parameter names and types the manifest carries — then the diff can classify those too, and it should, because a required parameter appearing where none was is the breaking change most likely to be made accidentally. If it records only ids, the diff is a partial gate and should be described as one.

Ids present in the old catalog and absent from the new are removals, and they are the only bucket that needs a human. Each one is either a genuine deletion, which breaks every script that called it, or a rename, which breaks them in a way that looks like a deletion plus an addition. Chapter 22’s guidance for the rename case — keep the old id answering alongside the new one for a deprecation period — is only actionable if

someone sees the removal before the release goes out, and this is the diff bucket that makes them see it.

The gate, and what it should not block

Wiring the diff into a build is a short step from having it, and it is worth being careful about what the gate does, because an over-eager gate is abandoned and an abandoned gate is worse than none.

The gate should fail on the removal bucket, loudly, naming every removed id. That is the one case where the cost of not noticing is borne by someone else — the author of a script written months ago, who finds out at run time — and the cost of noticing is one deliberate acknowledgement by the person making the change.

The gate should not fail on additions, ever. Surface growth is the normal state of a healthy suite, and a gate that requires ceremony for every new verb taxes exactly the activity the whole design exists to make cheap.

The gate should not be satisfiable by regenerating the baseline. This is the same discipline Chapter 21 states about the golden transcript, and it is the failure mode every snapshot-comparison guard eventually faces: when the check fails, the tempting fix is to update the file it compares against, at which point the guard has been converted into a transcription service and asserts nothing. The removal bucket is a claim about compatibility, and the only correct responses to it are to restore the id, to alias it, or to accept the break explicitly and say so in the release.

And the gate should be reported per app, not in aggregate. A suite-wide count of removals tells nobody which script is about to break. The eleven surfaces age independently, on eleven different release cadences, and a diff that respects that is a diff someone can act on.

The catalog is not the documentation

One clarification closes the chapter, because the catalog is easy to mistake for something it is not. It is an inventory, not a manual. It answers “does this exist and what is it called”, which is the question a script author has at the start, and it does not answer “what does this do”, which is the question they have next. That second question is answered by the signature projection of Chapter 17 — the parameter names, types, and return type printed from the live manifest — and by the app’s own domain conventions, which Chapter 42 reads off the surface as naming rules.

The division is the same one that runs through the whole design. The catalog is generated from source and is therefore always buildable and never quite current. The manifest is read from a running app and is therefore always exactly current and requires the app to be running. Neither is a substitute for the other and neither is a document anybody maintains, which is the property they share and the reason both can be trusted: they are projections of the code taken from two different angles, and a projection cannot drift from the thing it projects.

Chapter 47 – The Audit Trail

The question the return path makes askable

An automation that fires and forgets leaves nothing to review. That is not a virtue of discretion; it is a consequence of there being nothing to record. The old palette's `event` step, dispatching one action by id and learning nothing, could have written down that it dispatched — and that is the entire content of the record it could produce. Whether the action succeeded, what it produced, how long it took, and what the app's state was afterwards were all unavailable, because the return path did not exist.

The bus makes all four available, and the moment a script can know them it can write them down. This chapter is about doing that deliberately: what a record of a run should contain, where the seam to produce it is, why the bus does not produce it for you, and what such a record is and is not evidence of. It is not about checkpointing — Chapter 41 covers writing down what has been finished so a restart can skip it, and the two artifacts look similar and answer completely different questions. A checkpoint exists so the *next* run behaves correctly. An audit trail exists so a *human* can answer questions about a run that already happened.

Why the bus does not log calls itself

The obvious place to put call logging is the bus itself. Every call passes through the `App` module on its way out and back; a flag on the module could write every verb, every argument map, and every reply to a file with no work from any script author. The design does not do this, and the reason is worth stating because it is not laziness.

The first reason is that the bus cannot know what is sensitive. An argument map is arbitrary JSON supplied by the script, and across this census it will contain, at various times, mail account credentials on the way to `account.add`, a WireGuard private key on the way through the `wg.*` namespace, an authorization header on the way to `request.send`, and the contents of a document on the way to an import verb. A blanket logger writes all of it to a file in plaintext. A selective logger requires the bus to hold a policy about which parameters of which verbs of which apps are secret, which is per-app domain knowledge in the one layer that Chapter 2 forbids to have any — the module must not know what a verb means.

The second reason is that the useful unit of a record is not a call. It is a *run*: a named operation, with a beginning, an outcome, and a set of calls that belong to it. A bus-level logger produces an undifferentiated stream of calls from every script on the machine, interleaved, with no notion of which script or which run they came from. A script-level recorder produces one file per run with the run's own name on it, which is what a human actually wants to read.

The third reason is the general one this design keeps returning to: the seam already exists. A script holds handles and makes calls through them, so the script can wrap the call. Adding machinery to the bus to do what one helper function does in the caller is adding a permanent shared cost to serve a case the caller can serve better.

The wrapper, since the handle is the seam

The whole mechanism is one function that calls through to a handle and writes down what happened on the way.

```
#!/usr/bin/env stryke
use App

val $log = "~/local/share/bus-audit/${\ run_id() }.tsv"

fn audited($h, $app, $verb, %args) {
  val $t0 = now()
  val $out = try { $h->call($verb, %args) }
    catch ($e) {
      record($app, $verb, "err", "${ $e }", now() - $t0)
      die $e # the failure still propagates
    }
  record($app, $verb, "ok", "", now() - $t0)
  $out
}

fn record($app, $verb, $status, $detail, $ms) {
  append_file($log,
    "${\ now() }\t${ $app }\t${ $verb }\t${ $status }\t${ $ms }\t${ $detail }\n")
}
```

Two properties of that helper are the whole design. It **re-raises**, which means the failure model of Chapter 14 is completely unchanged: a script wrapped in `audited` dies at exactly the same call, with exactly the same message, as the unwrapped version. An audit wrapper that swallowed errors would have converted a recording device into a behaviour change, which is the one thing an audit layer must never be. And it records the failure *before* re-raising, so the death that ends the run is itself the last line of the run's record, which is precisely the line a human reading the file afterwards is looking for.

What it deliberately does not record is the argument map, and that is the same reasoning the bus itself used one section ago, applied at the layer that can actually make the decision. A script author knows whether the run's arguments are DOIs or passwords. A script whose arguments are safe can widen `record` to include them and gain enormously in usefulness; a script whose arguments are secret leaves them out and still gets a complete record of what was attempted and what happened. The decision sits with the only party holding the knowledge, which is the same placement principle as the hybrid handler's one deliberate leak in Chapter 2.

What an entry has to contain

Working backwards from the questions a person asks a record produces a short and non-negotiable list.

What was called — the app and the verb id, separately, because a verb id is only unique within an app and a run that drove four apps needs to be filterable by which. *When* — absolute, so the entry can be correlated with everything else on the machine that timestamps itself. *How long* — because “which step was slow” is a question asked of every run that felt slow, and the measurement is free once the wrapper already brackets the call. *Whether it succeeded* — as a discrete status field rather than as an inference from the presence of an error message, so that filtering the file for failures is a field match rather than a guess. And *what the failure was* when it failed, verbatim, because the four-way taxonomy of Chapter 14 distinguishes a rejected call from a broken pipe from a timeout from an undialed target, and those four demand different responses from whoever reads the file.

The format above is tab-separated because that is what the event tap of Chapter 30 already uses and because a record whose fields are positional is trivially filterable with the ordinary tools of the shell the script runs in. One JSON object per line would be the more faithful echo of the wire's own discipline, and it is the better choice for a run whose detail field can contain arbitrary text, since a tab-separated file with an embedded tab in its last column is a file that has quietly stopped parsing. Both are defensible; what is not defensible is a multi-line human-readable format, because a record nobody can filter is a record nobody consults.

Dry run, from the same seam

Once every call goes through one function, that function can decline to make it, and dry-run mode falls out of the audit wrapper for almost no additional code:

```
val $dry = dry_run_requested()      # an argument, a flag file, however the
                                   # script prefers to be told

fn audited($h, $app, $verb, %args) {
  if $dry && !is_read($verb) {
    record($app, $verb, "skip", "dry run", 0)
    return nil
  }
  # ... the real call, as above
}
```

The interesting part is `is_read`, and it is interesting because it cannot be answered generically. The bus does not mark verbs as read-only; the surface has verbs, state queries, and events, and while a *state query* is idempotent by contract — Chapter 27 says so, and Chapter 34 names the idempotent read as an idiom — a *verb* carries no such declaration. So `is_read` is a predicate the script author writes, usually as a namespace or prefix test against the conventions Chapter 42 documents, and it is only as good as those conventions are kept.

That is a real limitation and it bounds what dry run can promise. It is a useful development aid — run the orchestration, see the sequence of calls it would make, catch the loop that would have fired four hundred requests instead of four — and it is not a safety mechanism. A script author who mis-classifies one verb has a dry run that performs a real mutation, and no layer beneath them will catch it. The honest framing is that dry run shows you the plan, which is exactly what the `sync.plan`-then-act shape of Chapter 25 does with app support and far more authority, because there the app itself computes what would change.

Correlating with the app's side

A script-side record answers what the script attempted. It does not answer what happened inside the app, and for the failures that matter most — a verb that rejected for a reason the error string compresses, a call that took ten seconds inside an engine — the app's own logs are where the answer is.

Chapter 23 describes the three server-side log lines the host writes for each request: acceptance, dispatch, and reply. Chapter 24 already notes that those lines carry the wire's correlation `id`, and that bracketing the interval between the first and the last attributes the cost. The same property makes cross-side correlation possible: the script's record says a call to `transfer.add` at a given time took some duration and failed, and the app's log around that timestamp says which `id` it was, when the handler dispatched it, and when it composed the reply. The join is by time rather than by `id`, because the script never sees the wire's `id` — `App::call` hides it, correctly, since a correlation token is transport bookkeeping and not a value the caller has any use for.

That is a slightly weaker join than a shared identifier would give, and it is the right trade. Exposing the wire `id` to the script to make audit correlation exact would mean putting a transport detail into the language-facing API to serve a diagnostic use case, and Chapter 2's third prohibition — the module must not know which transport it is on — is worth more than the convenience. Timestamps join well enough for a single-user machine where the two logs are minutes of traffic apart, not milliseconds.

What the trail is evidence of, and what it is not

Three uses justify keeping the file, and one common expectation of it is false.

It is evidence for **review**: what did last night's run actually do, and did it do the same thing as the night before.

This is the use that pays most often and requires nothing but the file existing.

It is evidence for **reproduction**: a run that failed left a record of the exact sequence of calls up to the failure, and that sequence is a script — a shorter, more focused script than the original — that reproduces the problem. This is the use that turns a vague report into a bug.

It is evidence for **attribution**: when an app's state is wrong and several things could have made it wrong, the record says whether this automation touched it and when. On a single-user machine with the human as a concurrent writer per Chapter 40, ruling the script out is often the fastest way to find the actual cause.

What it is not is a **transaction log**. It records that a call was made and that a reply arrived, and Chapter 41 has already established the one place that falls short: a call that broke mid-flight has an unknown effect, and the record faithfully says “err” for something that may have completed entirely inside the app. A record can never be more certain than the protocol it records, and the protocol's honest position is that a broken call's effect is unknown. The trail narrows the question from “what happened?” to “did this one call take effect?”, which is a much better question to be left with, and it does not answer it. Only the app's own state does, which is why the reconciliation shape of Chapter 41 — read what is, compute the gap, act on the gap — remains the strongest answer available and an audit trail remains a complement to it rather than a replacement.

Chapter 48 – Driving an App a Human Is Using

The apps are not headless

Every automation stack the bus is compared against in Chapter 43 drives something that was built to be driven: a headless browser, a server process, a command-line tool. The bus drives none of those. It drives the same running instance of the same graphical application that a person may be looking at, clicking in, and typing into at the moment the frame arrives. There is no automation instance, no `--headless` mode, no second copy of the app started for the script's benefit. `App::open("zcite")` dials the reference manager the user has open.

Chapter 40 covered one half of what that implies — the human as a concurrent writer, and the read-modify-write discipline that follows from a selection that can change under a loop. This chapter is the other half, and it is about *effects rather than data*: what a script's calls do to the screen, to the user's attention, and to the app's own responsiveness, and which of the bus's mechanisms are affected by a human being present at all.

Three classes of verb, and the census contains all of them

Sort the census by whether calling a verb changes what the user sees and it splits cleanly, and the split is not correlated with anything else about the verbs.

Some verbs are **invisible**. `library.search` computes a result and returns rows; nothing in the window moves. `fs.list` enumerates a remote directory. `bib.bibliography` renders citation text into a returned value. `vpn.status` reads a condition. A script can issue thousands of these against an app somebody is working in and the only observable consequence is whatever the machine's fans do.

Some verbs are **visible by construction**, because being visible is the entire point of them. `appshell.terminal.open` opens a terminal panel. `appshell.files.open` opens the file browser. The palette verb summons the palette. The theme, CRT, and neon toggles Chapter 8 lists among the shared fifteen exist to change appearance and nothing else. A script calling one of these has reached into the user's window and moved something.

And some verbs are **incidentally visible**, which is the class that surprises people. A verb that changes durable state may also update a view that is currently displaying that state — adding an item to a collection the user has open, renaming something visible in a sidebar, completing a transfer whose row is on screen. The visibility is not the verb's purpose; it is a consequence of the app being a live view over the state the verb changed, which is exactly what a good GUI application is.

Nothing in the surface marks which class a verb is in, and nothing should: `returns` and `params` describe the conversation, not the pixels. So the classification is a judgement a script author makes, and it is worth making before writing an unattended run, because the three classes have genuinely different etiquette.

Modality, and the one place a human can block a call

There is a mechanism-level consequence of the human's presence, and it follows directly from the hybrid handler of Chapter 7 rather than from any new machinery.

An engine verb never enters the webview. It is dispatched in Rust, against the same `*-core` engine the UI calls, and its reply is composed without a frame crossing into JavaScript. Nothing the user is doing in the interface stands between that call and its answer. The engine path is, for this reason, not merely the fast path but the *human-independent* path.

A frontend verb takes the other arm: the backend evaluates the dispatch shim in the webview, the shim runs the registered verb, and the reply comes back through the reply command. Every step of that depends on the webview being in a state where it will run the shim and the registered verb will do its work. A modal dialog awaiting a decision, a long-running frontend operation the user started, a view mid-transition — any of these can delay the shim, and if the delay outlasts the bridge's deadline the script gets a timed-out call, which per Chapter 14 is a death naming the verb.

This is not a defect in the bridge; it is the accurate behaviour. The frontend verb's implementation *lives in the frontend*, so a frontend that is busy is a verb that is busy, and there is no honest way for the bridge to produce an answer that the thing implementing the verb has not produced. What it does mean is that the reliability of a call under unattended conditions is partly a function of which arm of the handler it takes, and a script author who cannot tolerate a modal dialog costing them a run should prefer engine verbs where both exist. Chapter 24 already gives the measurement that identifies which arm a verb takes — the engine-versus-frontend subtraction — and here that same measurement answers a robustness question rather than a performance one.

The attention rule

For the visible classes, the guidance reduces to one rule, and it is a rule about intent rather than about mechanism: *an automation should not move the user's attention unless moving the user's attention is what the automation is for*.

A scheduled maintenance run of the kind Chapter 30 describes has no business opening a terminal panel, flipping a theme, or summoning a palette, because none of those advance the maintenance and all of them will be discovered by a person who did not ask for them and cannot tell what caused them. This is the worst property of a misplaced visible call: it is *unattributable*. A window that opens itself while a person is working looks like a bug in the app, not like the consequence of a timer that fired, and the user will spend their investigation in the wrong place.

The converse case is real and legitimate. A palette command's `stryke` step — Chapter 9's first front-end — was invoked by the user a moment ago and is entitled to change what they see, because they asked. A hook that reacts to something the user just did in the app is in the same position. The distinction is not “visible verbs are bad”; it is that a visible effect must be traceable by the user to something they initiated, and the three front-ends of Layer 5 differ enormously in whether that is true. The palette step: always. The hook: usually, since the event it fired on was usually caused by the user. The scheduled `.stk` file: never.

That gives a per-surface default worth adopting. Scripts written for the palette may be as visible as they like. Scripts written for a scheduler should call only invisible verbs, and where they cannot, they should confine the visible ones to apps the user is demonstrably not in front of — which, on a single-user machine at four in the morning, is all of them.

The app is its own progress display

The chapter has been treating visibility as a hazard, and there is a case where it is the opposite, which the job shape of Chapter 44 makes reachable for the first time.

When a person deliberately runs a long orchestration and watches it, the fact that it drives the real app means the real app is showing them what is happening. Transfers appear in the transfer client's own list, with its own progress rendering, because the script enqueued them through `transfer.add` and the app is displaying its own queue. Items appear in the reference manager's library view as `item.add` returns each one. Nothing built the progress display; the app already had one, and the automation is populating it as a side effect of doing the work.

Compare that to the alternative the bus replaced. A script driving the pixels shows you the pixels being driven, which is a display of the automation rather than of the work. A script reading and writing an app's data files behind its back — Chapter 43's third comparison — shows you nothing at all, and worse, the app's own view is now stale relative to the data underneath it. The bus's approach is the only one of the three where the app's display stays truthful during the run, because the run went through the app.

The practical consequence is that a script driving a watched run can and should keep its own printed output to the level the app's display does not cover — the cross-app sequencing, the decisions the script made, the items it skipped — and leave the per-item progress to the apps, which already render it better than a terminal will. A script that prints a line per transfer alongside an app that is drawing a progress bar for the same transfer has duplicated a display that was already there and better.

The verb that should not exist

One design conclusion for app authors falls out of everything above, and it is a prohibition sharp enough to state as a rule: *a verb must not depend on view state*.

A verb whose behaviour differs based on which tab is focused, which pane is open, or what the window's current mode is has made the automation's result a function of what the user was doing, which means the same call at two different moments does two different things for reasons no script can see or control. Worse, it makes the verb untestable at Chapter 21's first level — the surface in isolation, with no desktop — because there is no view for the verb to consult.

The exception proves the shape. A verb *may* legitimately take view state as an explicit *input*: a state query named `selection` exists precisely so a script can read what the user has highlighted and pass those identities to a verb. That is view state crossing the boundary as data, visible in the script, captured at a moment the script chose, and per Chapter 40 held as a snapshot the script controls. What must not happen is the verb reaching for the view itself, invisibly, at a moment nobody chose.

An app author who finds a verb wanting to know the current view has almost always found a missing parameter. The fix is to add it, let the caller supply it, and let the app's own UI supply it too when the palette invokes the same verb — which is the same registration serving two readers, exactly as Chapter 3 described for the manifest.

The rule, stated

Three sentences carry the chapter. A script drives the app the user is in, so every call is either invisible to them or is something they will see happen. Which of the two it is depends on the verb, is not marked anywhere in the surface, and is therefore the script author's judgement to make before the run rather than the user's surprise to discover during it. And the mechanism follows the same split as everything else in this design: the engine path is independent of the human, the frontend path is not, and knowing which arm a verb takes tells you both what it costs and how reliably it will answer when somebody else is using the app at the same time.

Chapter 49 – A Complete Session, Narrated

From cold shell to finished run

The book has shown the bus in pieces — a call here, a recipe there, a tool built to illustrate a property. This closing chapter runs one session whole, from a cold terminal to a finished cross-app result, narrating each step in the vocabulary the earlier chapters built, so that a reader sees the mechanisms not as separate chapters but as the single continuous experience they add up to in use. The task is concrete and ordinary: gather the current selection from the reference manager, look each item up against a live API, file the ones that resolve into a new collection, and mail a summary — four apps, one file, one run.

Discovery: what is even running?

The session begins the way most do, by asking the machine what it is working with. At a cold prompt, the first move is `App::list()`, and its answer is the directory scan of Chapter 6 rendered as a fact:

```
val @up = @{ App::list() }
p "running: ${\ join(" ", @up) }"
# running: zcite, zreq, zemail, zwire, zoffice
```

Four of the apps the task needs are up; the fifth it will not need. This is the undialed-call guard of Chapter 29 used prophylactically — check the list before opening, so a target that happens to be down is a skipped step rather than a death at `connect`. The script has learned, in one call, exactly which apps it can drive, from the live socket directory and nothing else.

Introspection: does the app still call it that?

The second move is the discover-before-call pattern of Chapter 34. Rather than trust its memory of a verb id, the script asks the app, reading the manifest of Chapter 3 through the `verbs()` of Chapter 5:

```
val $cite = App::open("zcite")
if !has_verb($cite, "library.search") { die "zcite surface changed" }
```

The `has_verb` helper of Chapter 32 is built from `verbs()` alone, works against any app, and turns “I hope the verb is still named that” into “the manifest says it is, or the script stops now with a clear reason.” This is the surface documenting itself, per Chapter 17: the script consults the running app, not a wiki, and the answer cannot be stale because it is the same registration the app will honor a moment later.

The read: state, not a verb

Now the work. The script reads the current selection — a *state query* of Chapter 27, not a verb, because the selection is a live condition the app maintains, and reading it twice would return the same answer with no side effect:

```
val @sel = @{ $cite->get("selection") }
p "selected ${\ scalar @sel} items"
```

`get("selection")` crosses the socket as the `get` frame of Chapter 28 and comes back as a stryke array of maps, projected from JSON by the value model of Chapter 13. The selection lived behind a window the old palette could not read; now it is a variable the script walks.

The cross-app loop, with failure tolerance

The center of the run is a loop that leaves one app and enters another once per item — the star topology of Chapter 13, every value passing through the script — with the try-per-item tolerance of Chapter 34 so one bad lookup does not end the run:

```
val $req = App::open("zreq")
val $cite2 = $cite # same handle; the hub stays the script
val @filed = @{}

$cite->call("collection.add", %{ name => "verified" })

for val $it (@{ @sel }) {
  val $doi = $it->{doi}
  if !$doi { next }
  val $res = try { $req->call("request.send",
    %{ url => "https://api.crossref.org/works/${ $doi }" }) }
    catch ($e) { p "lookup failed for ${ $doi }: ${ $e }"; nil }
  if $res && $res->{status} == 200 {
    $cite->call("collection.add_item", %{ collection => "verified", item => $it->{id}
      ~ })
    push @filed, $it->{title}
  }
}
p "verified and filed ${\ scalar @filed} items"
```

Read the seams the way Chapter 16 taught. `$it->{doi}` leaves `zcite`'s selection; the crossref URL built from it enters `zreq`'s `request.send`; the response's status decides whether `$it->{id}` goes back into `zcite`'s `collection.add_item`. Two apps, and the only place their data meets is the loop body — a program the user wrote and can read top to bottom. The `try` turns a failed lookup into a logged skip, per the failure model of Chapter 14, so a network hiccup on one DOI costs one item, not the run.

The finish: a summary, mailed

The run ends by crossing to a third app, `zemail`, to send the summary — the value that was computed in the loop leaving the script and becoming a message body:

```
val $mail = App::open("zemail")
$mail->call("message.send", %{
  to      => "me@example.com",
  subject => "verified ${\ scalar @filed} references",
  body    => "Filed into 'verified':\n" + join("\n", map(@filed, fn ($t) { "  - ${ $t
    ~ }" })),
})
p "summary sent; run complete"
```

`message.send` is `zemail`'s own verb, and the body is a string the script built from the loop's results. No app knew about the others; the reference manager never heard of the API client, which never heard of the mail client. Each answered the script, and the script was the one place all three met.

What just happened, in the book's terms

Step back and name the mechanisms the session spent, because their sum is the whole book. `App::list` read the socket directory (Chapter 6). `App::open` dialed three sockets, each a handle that is a connection and not a value (Chapters 5, 13). `verbs()` and `has_verb` introspected the live manifest (Chapters 3, 17, 32). `get("selection")` read state, not a verb (Chapter 27). The loop stayed a star with the script at the hub (Chapter 13). `request.send`, `collection.add_item`, and `message.send` were called with typed arguments and returned values into variables (Chapters 2, 4). A `try` made a failure a recorded skip rather than a death (Chapter 14). Every value that crossed between apps crossed through the script, projected from JSON and back at each boundary (Chapter 13). And underneath all of it, the same newline of JSON carried every frame, correlated by the same `id`, over the same shared crate serving each app's one socket (Chapters 6, 7, 31).

That is the sentence the book opened on, executed once: one language drove every app the task needed, by name, and got each answer back into a variable the next call could use. The apps were always sets of functions waiting to be called; the session is what it looks like when they are finally called — from one file, one terminal, one run, by a script the bus handed a caller.

Chapter 50 – Text: Encoding, Normalization, and the Bytes in a String

The one type the wire has too much of

JSON has six types, and four of them cross without ambiguity: `null`, a boolean, an array, an object — each decodes to exactly one thing on either side. Two do not. Chapter 13 spent a section on the number, which loses its integer/float distinction in transit. It did not spend one on the string, and the string is the type that carries almost everything: every verb id, every state name, every error message, every path, every title, every DOI, every folder name, every account address in this census crosses the wire as a JSON string.

A string is not simple. It is a sequence of Unicode scalar values with at least three separate ways to disagree with an apparently identical string: a different encoding, a different normalization, and a different case. None of the three is visible when the two are printed side by side, and all three matter the moment a script uses a string from one app as a key against strings from another — which is the central move of every cross-app orchestration in this book.

UTF-8 is the wire's assumption, and it holds on one side by construction

JSON interchanged between systems is UTF-8, and the bus never has to negotiate an encoding because there is no configuration point at which a different one could be chosen. That much is settled. What is worth being precise about is which side of the bus that guarantee is structural and which side it is merely expected.

The app side is structural for the Rust half. A Rust `String` is UTF-8 by invariant; a value that reached a `String` is already valid UTF-8, and an engine verb returning one cannot produce an ill-formed JSON string no matter what it read. That is a real guarantee and it covers every engine command that routes straight to the core, per the hybrid handler's first arm.

It is not structural for everything an app can produce. A Unix path is a sequence of bytes with no encoding guarantee — the kernel enforces only that it contains no NUL and no slash inside a component — so a file whose name is not valid UTF-8 is a legal file that has no JSON string. An app whose `fs.list` walks a directory containing such a name has three options and only three: skip the entry, replace the invalid sequences with the replacement character and return a name that no longer opens the file, or fail the call. All three are bad; the third is the least bad, because the other two produce a value a script will act on.

This is not an edge case invented to be tidy about. The census contains a remote filesystem namespace, a transfer client, and a mirror recipe that diffs a local tree against a remote store. A tree mirrored from a system with a different filename encoding is exactly where undecodable names live, and the diff is exactly the operation that will silently treat a skipped entry as an absence and propose deleting the remote copy. An app author writing a directory-listing verb should decide, deliberately and in writing, which of the three it does, and a script author should know that a listing verb answers a question about *representable* names.

What the framing depends on, stated exactly

Chapter 6 justified newline framing on the grounds that JSON escapes an embedded newline, so an encoded value is always one physical line. That is true and it is worth stating the general form of it, because the guarantee is broader than the newline and a reader implementing a client should know its full shape.

JSON requires every code point below U+0020 to appear in a string as an escape. That is the whole control range, not the newline alone — the carriage return, the tab, the vertical tab, the NUL, the escape character, all of them. So a non-pretty-printed JSON value is guaranteed to contain no byte below 0x20 anywhere, which is a stronger property than “no newline” and is what actually makes the framing safe. A reader can split the stream on 0x0A with no state machine, no quote tracking, and no awareness of JSON at all, and it will never split a frame in the middle, because the only 0x0A on the wire is the one the writer appended.

The property fails exactly once: if a writer pretty-prints. A JSON encoder set to indent output emits newlines *between* tokens, outside strings, and those newlines are not escaped because they are not in a string. Nothing else about the frame changes — it is still valid JSON, it still has the same fields, a parser handed the whole thing would accept it — and the transport shatters, because the reader now sees a dozen malformed lines where one frame was. This is worth naming as the single configuration mistake that breaks the wire, since it is the one an app author can make while making the output *more* readable.

Normalization: two strings that print the same and are not equal

Unicode allows the same visible text to be encoded more than one way. A Latin letter with an accent can be one composed code point, or the bare letter followed by a combining mark. Both render identically. Neither is wrong. They are different sequences of code points, therefore different JSON strings, therefore different map keys, therefore unequal under every comparison a script performs.

The bus does not normalize, and it should not: normalizing at the boundary would mean the value a script receives is not the value the app holds, which breaks the one property the return path exists to provide. So normalization is a script-level and app-level concern, and it becomes a bug in exactly one shape — a join.

Every cross-app recipe in this book joins on a string. The reconciliation recipe matches attachment filenames from a reference library against object names in a remote store. The reading-list harvest matches DOIs. A mail filter matches folder names. Each of those is a `%{ }` lookup or a set-membership test where the two sides came from two different apps, each of which got its string from a different origin — one from a user typing into a text field, one from a filesystem, one from an HTTP header, one from a document's metadata. The probability that two such origins agree on normalization for accented text is not one, and the failure is the worst kind: the join silently produces fewer matches than it should, and the recipe reports the unmatched items as work to do.

The discipline is the ordinary one and it belongs in the script, at the point of the join, not scattered across the calls:

```

# one helper, applied to both sides of the join. `fold` is the script's
# own normalizing function — the bus supplies none and should not.
fn key($s) { fold($s) }

val %remote = %{}
for val $o (@{ $ftp->call("fs.list", %{ path => $dir }) }) {
  %remote{ key($o->{name}) } = $o
}

for val $it (@{ $cite->call("library.search", %{ q => "" }) }) {
  for val $att (@{ $it->{attachments} }) {
    if !%remote{ key($att->{name}) } {
      p "missing on remote: ${ $att->{name} }"
    }
  }
}
}

```

Two things about that helper are the point. It is one function, so the two sides of the join are guaranteed to agree — a normalization applied on one side only is worse than none, because it converts a match into a mismatch for a different population of inputs. And it is applied to the *key*, not to the value: the map still holds the app's own string, so anything the script hands back to an app — a filename to delete, a title to display — is the app's byte sequence and not the script's reinterpretation of it. Normalize what you compare; carry what you were given.

Case, and where folding belongs

Case is the same problem with a second wrinkle: case folding is language-dependent in a small number of documented cases, and a script folding a title for comparison is making a linguistic decision it has no information to make. For a join key that exists only inside one run and is never shown to anyone, folding is fine and the snippet above does it. For anything the user sees or the app stores, it is not the script's call.

The more useful rule is about *where* the fold happens. A verb that searches — the library search, the message search, the remote listing — is doing a comparison inside the app, against the app's own corpus, with the app's own knowledge of what its data means. A script that pulls a result set across the wire in order to filter it case-insensitively in the VM has done two wrong things at once: it has moved bulk onto the control plane, which Chapter 13 already prices, and it has substituted its own folding rule for the one the app applies to its own data, so its answer will not match what the same query returns in the app's UI. The consistency argument is the stronger of the two. A script whose filter disagrees with the app's filter is a script whose results a user cannot reproduce by hand, and reproducibility by hand is what makes an automation trustworthy.

Byte-shaped values, and the threshold at which a file wins

Some values are not text at all: a thumbnail, a signature, a key, a digest, a certificate. JSON has no byte-string type, so anything of this kind crosses either as an array of numbers or as a text encoding of bytes inside a string — and the second is the only sane option, since the first costs several bytes of decimal digits and a comma per byte.

A base-64 payload costs roughly a third more than the bytes it carries, plus whatever the JSON string escaping adds, plus a decode on the far side, plus the fact that the frame is now unreadable in the `socat` transcript that Chapter 23 depends on. Those costs are all acceptable for something small and all unacceptable for something large, so the question is where the line sits, and it is not a size in bytes. It is a question about what happens to the value next.

If the value's next stop is a decision — a fingerprint the script compares, a checksum it verifies, a short key it

passes as an argument to one other verb — inline is right, because the alternative is a temporary file with a lifecycle, a cleanup obligation, and a residue if the run dies. If the value's next stop is another app or the disk — a rendered page, an exported archive, a captured image — the file is right, and the wire should carry the path, per the rule the second carrier chapter states. The threshold is not a byte count; it is whether the script is going to *look at* the value. Bytes the script inspects can be inline. Bytes the script only forwards should never have entered the script at all.

The ids are ASCII, and that is a rule worth keeping

One last string deserves a sentence. Verb ids, state ids, and event ids are strings on the wire like any other, and nothing in the frame format prevents an app from registering a verb whose id contains an accented letter, an emoji, or a right-to-left mark. The naming conventions read off the shipped surface are lowercase dot-separated words, and every id quoted anywhere in this book is ASCII.

Keep it that way, for reasons that are all mechanical rather than aesthetic. An id that needs normalization is an id that can be typed two ways and matched one way, which turns the verb registry — the one lookup that must never be ambiguous — into another join with a normalization hazard. An id with a bidirectional control character can render in a transcript in an order that does not match its bytes, which defeats the legibility the whole wire format is built around. And an id outside ASCII is an id a shell completion, a grep, and a filename cannot handle uniformly across every tool in the terminal the scripts live in. The surface is a programming interface; its identifiers should be as boring as the rest of the language's are.

Chapter 51 – Time: Timestamps, Deadlines, and the Clock the Bus Does Not Have

JSON has no date, so every app invents one

The value model has a second gap of exactly the same shape as the number seam, and it is larger in practice. JSON has no date type. A timestamp crossing the wire is therefore either a number or a string, the choice is each app's, and nothing in the manifest's type vocabulary — the scalars, the two container forms, the app's own nominal shapes — distinguishes a timestamp from any other number or any other string. A verb declaring `returns: "int"` for a modification time and a verb declaring `returns: "int"` for a row count are indistinguishable to the bridge, to the catalog, and to the script.

That is not a defect to fix at the boundary. Teaching the bridge about dates would mean the bridge holding a semantic notion of a value's meaning, which is exactly what the layering forbids it. It is a convention to settle per surface, and the consequence of not settling it is that a script driving four apps will meet four time representations in one run and convert between them at every seam.

The two representations, priced

An **epoch integer** — seconds, or milliseconds, since the epoch — is unambiguous, totally ordered, cheap to compare, and immune to the normalization and parsing problems of the previous chapter. It carries no timezone because it needs none: it names an instant, and an instant is the same instant everywhere. Its costs are that it is unreadable in a transcript, that the seconds-versus-milliseconds choice is invisible in the value and catastrophic to get wrong, and that it cannot represent what a calendar means — “the first of the month, locally” is not an instant until a zone is supplied.

An **ISO-8601 string** is readable in a transcript, self-describing when it carries an offset, and directly comparable as text when — and only when — every producer emits the same form with the same precision and the same zone. Its costs are that those conditions are easy to violate, that string comparison silently stops sorting correctly the moment two producers disagree on fractional-second digits, and that every consumer needs a parser.

The recommendation the census's own shape supports is: an instant is an epoch integer, a civil date is a string, and no verb returns a form that is neither. A message's receipt time, a transfer's start, an item's date-added are instants. A publication year, an invoice's billing month, a due date are civil values that have no defensible instant. Mixing the two into one field — a “date” that is sometimes a full timestamp and sometimes a year — is the failure that makes downstream code untestable, and it is common precisely because both are dates in English.

Where the number seam bites time specifically

The codec restores an integer when the decoded number is integral and a float otherwise. Applied to time, that rule has a consequence worth holding.

Epoch seconds and epoch milliseconds are integral, so they arrive as integers and arithmetic on them is integer arithmetic — which is what a script wants, and there is nothing to think about. Fractional seconds are not integral, so they arrive as floats — also fine. The trap is the boundary case that the number seam always produces: a fractional-seconds timestamp that happens to land exactly on a whole second arrives as an integer, and a script that formats it with a decimal or feeds it to something expecting a float meets a type change that depends on the value. It will work for months and fail on the one run whose event fired at a whole second.

The remedy is the one the number seam already prescribes, applied at the one place it reliably matters: coerce at the point of use. A script that does arithmetic on times should force the type once, when the value enters the script, rather than trusting a rule whose output depends on the data. This is the single most likely place in a real orchestration for the number seam to produce a bug, because timestamps are the numbers most likely to be fractional and most likely to be occasionally integral.

The deadline needs a clock that does not step

Every call is a bounded wait. A deadline is only as good as the clock it is measured against, and there are two kinds of clock on any machine that runs these apps.

A **wall clock** reports civil time and is allowed to move discontinuously. It is adjusted by a time synchronization daemon, by the user changing the date, by a laptop learning the correct time after a flat battery. It can jump backwards. A deadline computed as “wall clock now, plus the timeout” and checked against a later wall-clock reading is a deadline that a backwards step extends arbitrarily and a forwards step fires early. A call that survives its own verb only to be killed because a sync daemon stepped the clock forward is a failure with no cause anyone will find, and a call that hangs for an hour because the clock stepped back is the hang the timeout exists to prevent.

A **monotonic clock** reports elapsed time from an arbitrary origin and never goes backwards. It is the only correct basis for a timeout, a retry backoff, or a duration measurement, and the audit wrapper’s elapsed-milliseconds field is a duration measurement, so the same rule applies to it: a run whose recorded step durations are computed from wall-clock readings will occasionally record a negative duration, and a negative duration in an audit file is an entry nobody can explain.

There is a second axis, and it is the one that catches laptops. Monotonic clocks differ in whether they advance while the machine is suspended. On Linux, `CLOCK_MONOTONIC` does not count suspended time and `CLOCK_BOOTTIME` does. For a bus deadline the distinction is real: a script that calls a long verb and closes the lid has, on the not-counting clock, paused its own deadline, so the call resumes on wake with its full remaining budget — which is almost always what a user wants. On the counting clock, the same call comes back from a lunch break already dead. An implementer choosing between them should choose deliberately; a script author should know that “the timeout fired” after a suspend means the second choice was made.

What the event frame does not carry

The frame catalog is exhaustive, and reading it for what is *absent* is as instructive as reading it for what is present. An **event** frame carries its subscription id, its event id, and its payload. It carries no timestamp.

That is a deliberate consequence of what an event is on this bus: a live notification, not a persisted record, delivered to whoever is subscribed at the moment it fires. A notification’s time is approximately now by construction, so the frame does not spend a field on it. But the approximation has a width, and the width is the pump. A subscriber’s callback runs when the script next drains its queue, and the ordering guarantee is arrival order, not emission order across different apps. So a script that records “when did this happen” from inside a callback is recording when it *pumped*, not when the app emitted, and the gap is whatever the script’s loop body took.

For most watchers this is irrelevant — a rebuild-on-change flag does not care. For a watcher that is building a

log of when things happened, it matters, and there is exactly one correct fix: the emitting app puts the time in the payload. The payload is the app's own value with the app's own fields, so an app whose events are worth timestamping declares a time field and fills it at the moment of emission, where the true time is known. A script cannot recover that information after the fact and should not pretend to by stamping its own clock on arrival.

The unattended run meets the calendar

A scheduled script inherits every hard part of civil time at once, and two of them produce bugs that only appear on specific days of the year.

The first is “yesterday”. A maintenance run that asks an app for everything since some computed instant is doing calendar arithmetic, and calendar arithmetic across a daylight-saving transition is not subtraction. Twenty-four hours before a particular local midnight is not the previous local midnight on the two days a year the offset changes; it is an hour before it or an hour after. A window computed by subtracting a duration will, on those days, either re-process an hour of work or skip an hour of it. Which of the two is worse depends entirely on whether the verbs being called are idempotent — which is the app's property, not the script's — so the two failure modes are “harmless duplicate” and “silently missed work” and a script author cannot tell which they have without knowing the surface.

The second is the scheduler's own zone. A job runs under whatever zone its scheduling context defines, which is not necessarily the zone the user's session uses and not necessarily the zone the app formats its dates in. A run that computes a window in one zone and compares it against timestamps produced in another is off by the offset, every day, uniformly — which is the good case, because a uniform error is one someone eventually notices. The mixed case, where some values are instants and some are civil strings, is off by the offset for half its inputs, and that is the case nobody notices for a quarter.

The discipline that removes both: compute windows as instants, compare instants to instants, and convert to civil time exactly once, at the point of printing a report a human reads. Every intermediate value in a scheduled run should be an epoch integer. It is not that civil time is hard to get right; it is that it is hard to get right *repeatedly*, and a scheduled script gets a fresh chance to be wrong every night.

Chapter 52 – One User, More Than One Session

What the permission bit actually proves

The security model rests on a file mode. The socket lives in the per-user runtime directory at mode 0600, the trust boundary is the socket file's owner, and the model has two states — you are the user and you have everything, or you are not and you have no socket to dial. That is the right model and this chapter does not argue with it. It asks a narrower question the earlier statement leaves open: the mode names an *owner*, and an owner is a uid, and a uid is not a person, a session, or a login. Several things share a uid, and the differences between them are where a script author's expectations quietly stop matching the machine.

Start with what the bit does not cover. A process running as uid 0 is not subject to the permission check at all; the superuser can open any socket on the machine regardless of its mode. This is not a hole in the bus — a superuser can read the app's memory, its files, and its keychain items too, so a socket is the least of what it reaches — but it does mean “only the owning user can dial it” is precise only if read as “no *unprivileged* other user can dial it”. Say the whole sentence; a security claim that overstates by one word is a claim that will eventually be tested by someone who noticed.

The stronger protection, and the one worth understanding because it holds on both platforms the design targets, is not the socket file's mode at all. It is the parent directory. The bus's path is `zgui/<app>.sock` under a per-user runtime directory, and both of the directories it resolves against are private to one user by their own definition: the runtime directory on Linux is specified as being owned by the user and inaccessible to anyone else, and the temporary directory macOS hands a session is likewise per-user rather than shared. So two users on one machine do not merely fail to open each other's sockets; they never compute the same path in the first place. There is no shared namespace to collide in.

That distinction matters because the historical enforcement of a socket file's own permission bits at connect time has varied between Unix implementations, while the enforcement of directory traversal has not. A design that leaned only on the file's mode would be leaning on the part that varies. This one sets the mode *and* lives inside a private directory, so the weaker of the two guarantees is not load-bearing.

Two sessions, one uid, one socket namespace

Here is the case the two-state model does not describe, and it is not exotic: the same user logged in twice. A graphical session at the console and a terminal session over ssh are two sessions, two process trees, two sets of environment variables — and one uid.

On Linux the runtime directory is per-user, not per-session, so both sessions resolve the same `zgui/` directory. The consequence is direct and mostly delightful: a script run from the ssh session dials the socket of the app running in the graphical session and drives the desktop the user is not sitting at. Nothing special was built to allow this. It falls out of the path being a function of the uid, and it is the closest thing to remote automation the design permits — while remaining, in every sense the security model cares about, entirely local. The frame never leaves the machine; only the human did.

It also produces the one confusion this arrangement generates. The single-instance rule says the last process to bind a name owns it. If the user has a graphical session running the reference manager and then launches

a second copy in some other context under the same uid, the second copy takes the socket name and the first is left serving a socket nobody will find. A script from either session now reaches the *second* copy — which may not be the window the user is looking at. The bus reports this honestly in the sense that `App::list()` shows one name and one process answers it, but the script has no way to ask *which* copy it reached, because the surface exposes no identity beyond the bus name.

The practical rule follows: on a machine where the same user may run an app more than once, treat the bus name as naming *an* instance, not *the* instance the user is watching. Every recipe in this book that reads a selection and acts on it assumes the two are the same, which they are in the intended single-instance world. A script that runs unattended in a second session and reads a selection is asking a question whose answer depends on which copy bound the socket last, and that is the one question the design deliberately does not answer.

The scheduler is a different session

The scheduled maintenance runs assume the script can dial the apps. That assumption depends on the scheduling context resolving the same runtime directory as the user's session, and this is exactly where scheduled bus scripts fail on a fresh machine.

Three cases, and they behave differently. A job scheduled *as the user, inside the user's session* — a per-user timer, a user agent, a shell job launched from a session that is still alive — inherits the same uid and typically the same runtime directory, dials fine, and is the intended shape. A job scheduled *as the user but outside any graphical session* may resolve a different temporary directory, because some platforms hand a session-scoped temporary path rather than a purely uid-scoped one; the script then computes a path where no socket exists and reports that the app is not running, which is true of the directory it looked in and false of the machine. A job scheduled *as a system service*, running as a different uid or as root, is either denied by the directory's mode or looking in the wrong directory entirely, and in the root case may succeed at the file layer while still failing to find the path.

The diagnosis for all three is the same and it is one line. A scheduled script whose first act is to print `App::list()` distinguishes them immediately: an empty list from a scheduler while the same script from the terminal reports five running apps is not an app problem, a permissions problem, or a bus problem — it is the scheduler resolving a different runtime directory. The fix is environmental, not architectural: run the job in the user's session context, or make the runtime directory explicit in the job's environment.

There is a second, quieter reason to prefer the user's own session for scheduled bus work, and it is not about paths. The apps being driven are GUI applications that are running because the user launched them. A job that runs when no session is logged in has nothing to drive: the sockets do not exist because the apps do not exist, launch-on-demand is not in version one, and the maintenance run correctly reports that every target is down. Scheduled bus automation is automation of a desktop that is up — that is not a limitation to work around, it is what a desktop bus is.

Sandboxes and containers

One more boundary belongs here because it produces a failure that looks like a bug and is not. A sandboxed application may be given a private view of the filesystem, including a private temporary directory. If an app is sandboxed that way, its `zgui/<app>.sock` is created inside the sandbox's view, and a script outside the sandbox — running in the user's ordinary session, as the same uid, with every permission it could want — computes the unsandboxed path, finds nothing there, and reports that the app is not running while the app is plainly on screen.

The general statement is that the socket path is resolved independently on both sides and the two sides must agree on what the runtime directory *is*. Everything the design gets for free from “the filename is the registration” depends on the two parties sharing a filesystem view. Same uid is necessary and not sufficient; same namespace is the real requirement, and same uid is only the usual way to get it. The same reasoning

explains the container case without needing a separate discussion: an app in a container and a script outside it are not on the same bus, whatever their uids say.

What would tighten this, and why version one does not

An implementer who wanted the bus to know *who* is calling, rather than only that the caller could open the file, would reach for peer credentials — the mechanism a Unix socket offers for a server to learn the uid and pid of the process on the other end, without trusting anything the client sent. It would let the handler log the calling process, refuse a connection from a uid it did not expect even if the file mode somehow allowed it, and distinguish one dialer from another for auditing.

Version one has none of this and the omission is coherent. Every capability on that list is only useful if there is a decision to make about the caller, and the trust model's whole point is that there is not: inside the boundary everything is trusted equally, so a uid check would confirm what the directory already enforced, and a pid would name a process the model has already decided to trust completely. Adding identity to a system with one principal buys nothing but a field to log.

The honest framing is that this is a thing the design *could* grow, additively, if the principal count ever changed from one — and that the principal count changing from one is the same event that the network refusal is guarding against. The single principal is not an assumption the bus makes casually. It is the assumption every other simplification in the design is paid for by, and the reason peer credentials are absent is the reason there is no capability token, no middle tier, and no authentication handshake in the frame set.

Chapter 53 – Secrets, and What Outlives the Run

The problem is not interception

A chapter on secrets in a local IPC system has to start by saying which problem it is not solving, because the obvious one is already solved by construction. Nothing on this wire is at risk of interception. The socket is a file in a private directory, both ends run as the same user, no bytes reach a network interface, and an attacker who could read the socket could equally read the app's memory. There is no confidentiality problem *in flight*, and adding encryption to a local socket would be theatre.

The problem is residue. A run's values pass through several artifacts that outlive it — a transcript someone captured, a test fixture someone committed, a log a script appended to, a terminal scrollbar someone pasted into a bug report — and the values that pass through them include everything on the audit-trail chapter's list of sensitive arguments, plus at least one thing that list does not cover: a key generated by `wg.genkey` is a secret in a *return value*, arriving in the VM without having been an argument to anything. That chapter made the one decision that matters most here — the wrapper records the verb and the outcome and deliberately not the argument map, because only the script author knows which arguments are secret. This chapter asks the same question everywhere else it arises, because the argument map is not the only place a secret travels and the audit file is not the only thing that outlives a run.

Six places a secret can come to rest

Enumerating them is most of the work, because each has a different owner and three of them are not the script's to fix.

The `socat` transcript. The debugging affordance the wire's legibility exists to provide is a complete, plaintext, unredacted copy of every frame including every argument. This is entirely correct as a debugging tool and entirely unsafe as an artifact. A transcript is a thing people paste into issues.

The app's own logs. The bridge logs acceptance, dispatch, and reply. Read that list carefully: dispatch logs the verb name that arrived, not the arguments it arrived with. So the app-side trace is, by default, safe to share — it says a call to `account.add` was dispatched, not what was in it. That default is a property worth protecting deliberately, because it would be natural for an app author debugging their own engine to add the argument map to that log line and never take it out, converting the one server-side artifact that was safe into another one that is not.

The reply's error string. This is the leak the layering makes easy to miss. The audit wrapper records the error text on failure — that is the whole point of the wrapper, and the entry it writes is the line a human most wants. If a verb composes an error that quotes the credential it just rejected, the secret the script author carefully excluded from the record arrives in the record anyway, through a field they did not choose the contents of. The script author cannot fix this; the app author must, and the rule is short: an error string names *what* failed, never the value that failed.

The event payload. An `emit` fans out to every subscription on that event across every connection, with no acknowledgement, no scoping, and no way for the emitter to know or limit who is listening. Every other value an app produces is produced in answer to a caller; an event payload is the one value it publishes. An app that

puts a credential in an event payload has handed it to every script watching, including ones written by nobody in particular and left running in a forgotten terminal. Event payloads should carry identifiers and outcomes, not contents.

The state query. A `get` takes no arguments and is the cheapest thing a script can call, which makes it the most likely thing a script calls without thinking. A state query that returns an active environment as a full map of its bindings returns whatever those bindings hold, and in an API client's environment they hold tokens. The narrow design — returning the environment's name and identity, with the bindings reachable only through a verb that a script has to mean to call — is not paranoia; it is the difference between a value that shows up in every casual introspection and one that shows up where someone asked for it.

The golden transcript. The testing chapter's fixture is a recorded conversation checked in as an expected result. Recording it against a real account records the real credentials into a file whose entire purpose is to be committed and diffed. Fixtures get synthetic values, always, and a fixture that cannot be synthesized is a test that should run against a throwaway account rather than a real one.

The pattern that removes most of it: name the secret, do not carry it

The strongest mitigation is not redaction anywhere; it is arranging for the secret never to enter the script. The bus already has the shape for this and two apps in the census already use it.

An API client's environments exist so that one saved request runs against staging or production by *activating an environment* rather than by editing the request. The script that calls `environment.activate` with a name and then `request.send` has driven an authenticated request without the credential ever crossing the wire, because the credential lives in the app and the script named it rather than supplying it. A tunnel client's configuration holds keys the same way: a script calls `vpn.connect` and gates on `vpn.status`, and no key is in the script or on the socket.

Read as a surface-design rule it generalizes to every app: a verb that takes a secret as a parameter is a verb that forces every caller to hold that secret, and a verb that takes a *reference* to a secret the app already stores is a verb that lets callers stay ignorant. Provisioning verbs — the ones that install a credential in the first place — genuinely need the value and are the exception; they are also called once, by hand, rather than on a schedule. The test for whether a surface has this right is to ask which of its verbs a scheduled script would have to embed a secret to call. Ideally: none.

What a script does when it cannot avoid the value

Sometimes the value is unavoidable, and the discipline is then about the script's own habits rather than the surface's design.

Keep secrets out of the command line. A bus script is an ordinary program in the shell, which means its arguments are visible in the process table while it runs and in the shell's history afterwards. A run that needs a credential should read it from a file or from the environment, never from `argv`, and the reason is not the bus at all — it is that shell history is a durable, greppable, backed-up record of every command anyone typed.

Do not print returns wholesale. The habit of `p` on a whole map is how a script author learns what a verb returned, and it is exactly right at the prompt and in the REPL. In a script that will run unattended and whose output someone will keep, it prints whatever the app put in the map, forever, including fields the author never read. Print the fields the run is about.

And treat the value's lifetime in the VM as part of the design. A secret decoded into a variable lives as long as the run does, and a run that dies leaves whatever it had produced on disk. A script that fetches a key, writes it to an intermediate file for another app, and then dies has left the key in the per-run directory that the sweeper will remove *eventually* — which is fine for a rendered invoice and not fine for a private key. The second carrier

is the wrong carrier for a secret; values that must not persist should stay on the wire, where nothing writes them down.

Why the bus does not solve this for you

Every mitigation above lives in the app or in the script, and none of them lives in the bus. That placement is not an oversight and it is the same argument the audit trail already made, generalized.

For the bus to redact, it would have to know which parameters of which verbs of which apps are sensitive. That is per-app domain knowledge, and the layer that would hold it is the one layer forbidden to hold any: the module must not know what a verb means. A generic rule — redact anything named `password` or `token` — is worse than nothing, because it produces the appearance of protection while missing the field named `key`, the field named `header`, and the credential embedded in a URL, and it teaches script authors that the layer below is handling it.

So the honest posture is the one the design takes everywhere: put the decision with the party that has the knowledge, and be explicit that it *is* a decision. The app author decides what an error string and an event payload may contain. The script author decides what a record and a print may contain. The bus decides nothing, carries everything faithfully, and does not pretend that faithfulness is a security property.

Chapter 54 – What a Verb Should Say When It Says No

The error field is a string, and that is the whole type system for failure

The reply frame's failure arm carries exactly one field: `error`, a string. There is no code, no category, no structured detail, no machine-readable cause. A bridge rejection and an app's own refusal arrive in the same field, in the same shape, and become the same thing in the VM — a death whose message is that string.

The error reference catalogs the failures the *bus* produces, and they are a closed and small set. This chapter is about the other population, which is unbounded: the failures a verb itself decides to raise. Across a census of this size those vastly outnumber the bus's own, and nothing in the design specifies them. A verb's error strings are not part of the wire, not part of the surface as the manifest describes it, and not part of the module — so of the three things the versioning chapter says the contract consists of, error text is in none of them.

That is worth sitting with, because the consequence is real. A script that branches on *why* a call failed has only the string to branch on. It matches text. The text is unversioned, uncatalogued, and invisible to the generated catalog's diff, so an app author can change it in a routine cleanup and break a script with no signal anywhere in the release process. This is the softest joint in the whole contract, and it is soft precisely because it was left out of the parts that were hardened.

Three rules for the string, in priority order

Given that, an app author's error strings are a de facto interface, and the useful response is to treat them as one deliberately rather than to pretend they are not.

Lead with a stable prefix. The bus's own errors do this and it is why they are usable: `no such verb:`, `missing required argument:`, and the rest put the invariant part first and the variable part after the colon. A script that must branch matches the prefix, which the app can then commit to keeping, while the tail stays free to improve. An error that reads `the verb foo.bar is not something this app knows about` puts the variable part in the middle and offers nothing stable to match.

Name the noun that failed, not the value that failed. The secrets chapter gives the security reason and there is an operational one too: the value is usually already in the script, which supplied it, so repeating it in the error tells the caller nothing it does not have. What the caller does not have is *which* of its inputs the app objected to. `collection not found` plus the caller's own knowledge of what it passed is a better diagnostic than the full argument echoed back.

Say whether the world changed. This is the field the frame does not have and the string must therefore carry. A failed call leaves the script unable to know whether the verb did nothing, did everything and failed to report, or did half. An error string that distinguishes `nothing was sent` from `sent, but the record failed` is the difference between a recovery that is safe to retry and one that will duplicate work. The restart chapter establishes that the effect of a broken call is unknown; an error string is the one place an app can make it known, and almost no app does unless its author was told to.

Raise or return: the decision a verb makes before it writes any text

Before the wording matters, an app author has to decide whether a given outcome is a failure at all. Three cases recur, and the third is the one that goes wrong.

Absence is a value. A search with no matches returns an empty list. A lookup for a thing that does not exist returns nil. Neither is an error, because the verb did exactly what it was asked and the answer is “none”. A surface that raises on empty results forces every caller to wrap every read in a `try`, which converts the failure model's opt-in catching into mandatory ceremony and makes the common path the ugly one.

Impossibility is an error. A verb asked to operate on a collection that does not exist, to send through an account that is not configured, or to export a document that is not open cannot produce the value its `returns` type promises. The manifest declares what the verb hands back; when there is no image of the outcome in that type, the honest reply is `ok:false`. This is the cleanest test available and it comes straight from the surface's own declaration: if `returns` is `item` and there is no item, do not invent one.

Partial success is neither, and it is the case that goes wrong. A verb that acts on many things — a filter run over a folder, a plan applied across a tree, a batch import — will sometimes complete some and fail others. Raising loses the successes: the script gets a death with a string and no way to learn which items went through, so it cannot resume without re-doing work whose safety it cannot assess. Returning `ok:true` with a plain value hides the failures entirely. The correct shape is `ok:true` carrying a *result* — a value with per-item outcomes in it — and it should be the declared return type of any verb whose work is plural. A verb whose reply is one bit for an operation with fifty outcomes has thrown away forty-nine of them, and the script's only recovery is to ask again and diff, which is a whole extra round trip to recover information the app had and discarded.

Note what this is not. The try-per-item idiom exists so a *script* can tolerate one bad item in a loop of single-item calls. The rule here is about a verb that is itself plural, where the loop is inside the app and the script cannot get between the iterations. The two are complementary: the script's tolerance covers the calls it makes, and the result value covers the work it cannot see.

Retryability is the field nobody has

Follow the partial-success case one step further and it exposes the deeper gap. When a call fails, the single most useful thing a script could know is whether calling again is safe and whether it might succeed. Neither is derivable from a string.

A timeout might mean the verb is still running and will finish after the deadline, in which case a retry does the work twice. A refused connection means nothing happened, so a retry is free. An app-level error might be permanent — a malformed argument no repetition will fix — or transient — a remote host that was briefly unreachable, which is precisely what half the transfer and request namespaces exist to talk to. These are three completely different instructions to a script and they arrive in the same shape.

The design's answer today is that the script must know its verbs: idempotence is the app's property, the surface documents it or does not, and a script driving a non-idempotent verb defends itself with a checkpoint. That is workable and it is what the restart chapter builds. What it is not is *self-describing*, and this is one of the few places in the design where a script cannot learn from the surface something it genuinely needs. The frame set is closed by intent and grows by adding fields, so a future `retryable` boolean or a category tag alongside `error` is exactly the shape of change the wire was built to absorb — an addition older readers ignore. Naming it as the gap it is seems more useful than pretending the string covers it.

The error a verb should never raise

One category deserves an outright prohibition. A verb should not raise to signal something the caller could have avoided knowing about — a retry the app performed internally, a cache it missed, a degraded mode it fell into. Every `ok:false` is a death at a call site by default, and using a death to report a condition the script did not ask about converts an implementation detail into a control-flow event in someone else's program.

The failure model earns its strictness by being rare and meaningful. A surface whose verbs raise on conditions the caller cannot act on trains script authors to wrap everything in `try` and to discard the message, and a `try` whose catch block is empty is the exact ceremony the opt-in design was built to avoid. If a verb wants to report a condition without failing, it returns a value with a field in it, and the scripts that care read the field.

Chapter 55 – Retiring a Verb

Removal is the half the additive rule does not cover

The versioning chapter's compatibility rule is stated in one direction: adding is safe, removing and renaming are the breaking moves, and an app that must rename keeps the old id as an alias and drops it after the scripts have moved. Every word of that is right and it leaves a practical question unanswered — *how does anyone know when the scripts have moved?* — along with a technical one the rule's shape hides: not all removals break in the same way, and the loudest ones are the least dangerous.

This chapter is about the residue. It matters more on this bus than on a public API for one specific reason that cuts both ways, and it is worth stating first because it inverts the usual advice.

The caller population is small, private, and enumerable

A public API's deprecation problem is that its callers are unknown and unreachable. This bus's callers are the opposite: they are `.stk` files on one user's machines, plus palette steps stored in app configuration, plus whatever a scheduler runs. The population is small, it belongs to one party, and — crucially — it is on a filesystem that party can search.

So the audit that a public API cannot perform is here a grep. Before removing a verb id, search the script directories, the scheduled jobs, and the palette definitions for the literal id. Verb ids are dotted, specific, and unlikely to collide with ordinary prose, which makes them near-perfect grep targets — a property the naming conventions produce as a side effect of being boring. The answer that search returns is not a guess about the caller population; it is the caller population, modulo scripts on machines not searched.

Two caveats keep it honest. A script that builds a verb id dynamically, from data rather than from a literal, does not appear in the search — the same blind spot the catalog generator has when it reads source rather than a running process. And a palette step's stryke source lives in the app's own configuration rather than in a file the user thinks of as a script, so a search that covers only `~/scripts` covers less than it appears to. Search both, and prefer literal ids in scripts partly for this reason.

Three removals, in descending order of loudness

The kind of thing being removed determines what a stale caller experiences, and the range is wider than the versioning rule's "breaking" suggests.

Removing a verb or a state id is loud. A call to a name in neither the engine's command list nor the registered set is rejected with `no such verb`, and a stale `get` is rejected with `no such state`. The script dies at the call site with the name in the message. This is the best possible outcome for a break: it happens immediately, it names the fault, and no wrong work is done. A removal that produces this can be made on a schedule and the fallout diagnosed in seconds.

Removing a field from a return value is quiet. The bridge does not check a return against its declared type, so a verb that stops populating a field returns a value the boundary accepts happily. The script reads the field, gets nil, and dies — or worse, does not die, because nil is a legitimate value it passes onward. The fault surfaces one or more statements after the call that caused it, which is the exact failure the loud-at-the-call-site design works everywhere else to prevent. The error, when it comes, names a variable rather than a verb.

Removing a parameter is silent. This one follows directly from a rule stated early and for good reasons: an argument the caller supplied that the verb never declared is ignored, so that the surface can grow additively. Run that tolerance backwards. A verb that drops a parameter keeps accepting calls that pass it, ignores it, and does the work *differently* — with a default in place of the value the caller meant. No error, no rejection, no signal of any kind. A search that used to be limited is now unlimited; an export that used to be scoped is now whole. The script appears to work and does the wrong thing, which is the one outcome the error reference says a script should never see, arriving through the surface rather than through the bus.

The ranking has a design consequence. A parameter removal is the change that most needs a deliberate loud failure, and the mechanism is available without any new machinery: keep the parameter declared and have the `run` body raise when it is supplied, with a message naming the replacement. That converts the silent break into a rejected call, at the cost of one condition in the verb and one line in the manifest, for exactly as long as the sunset window lasts.

Events are the removal with no error at all

The fourth case deserves separate treatment because the design does not settle it.

Removing an event is a subscriber's problem and subscribers are not callers. A script that subscribes to an event the app no longer emits does not fail; it waits. Its pump drains an empty queue forever, its callback never fires, and its loop runs correctly and does nothing. A watcher-shaped script — the kind the event chapter's flag-and-rebuild idiom produces — reports no error, consumes no resources worth noticing, and silently stops doing its job. There is no worse failure shape in this book.

What happens at subscription time is the part the design leaves open. The error reference names a rejection for an unknown verb and a rejection for an unknown state, and it names none for an unknown event; the frame catalog says a `sub` gets a reply confirming the registration. If the bridge validates the event id against the declared event list, then removing an event *is* loud, at the moment of subscribing, and this whole failure mode collapses to a normal death. If it does not, the subscription succeeds against a name nothing will ever emit and the watcher waits forever.

This is a real open question rather than a rhetorical one, and it should be settled in favor of validation. The argument is symmetry: `call` validates the verb name and `get` validates the state name, and there is no principled reason the third entry kind should be the one where a typo is accepted in silence. A script that misspells an event id today gets the same experience as one watching a removed event — nothing, forever — and both are diagnosed by reading `verbs()` and noticing the name is not in the list, which is a thing a person does only after suspecting it. Validating at `sub` turns both into one loud rejection at the point of the mistake.

The alias is invisible to the gate

The catalog diff is the changelog, and the compatibility gate is built on the diff. Deprecation interacts with both in a way worth spelling out, because the mechanism the versioning rule recommends defeats the tooling the catalog chapter builds.

An alias is a second id dispatching to the same `run`. In the manifest it is an entry; in the generated catalog it is a line; in the diff between two generations it is an *addition*. So a rename executed correctly — new id added, old id retained as an alias — shows up as one added verb and no removal at all. The diff cannot see the rename, cannot see that a deprecation has started, and has nothing for the gate to block, which is correct as far as it goes: nothing broke.

The consequence is that the gate protects the end of the process and not the beginning. It will fire on the day the alias is finally dropped, which is the day something might break, and it will be silent for the entire deprecation period during which the migration was supposed to happen. Nothing mechanical will remind anyone that a sunset is pending. That reminder has to live somewhere a human reads — the alias's own entry,

marked in prose in the manifest's label, or a note in the release that added the new id — because the artifact that could have carried it mechanically has, by design, nothing to report.

There is a version of this the design could grow, and it is the same additive shape as everything else: a `deprecated` marker on a manifest entry, ignored by every existing reader, surfaced by `verbs()` and rendered in the catalog. Nothing in the current manifest carries one, so nothing in this book depends on it. It is named here as the obvious extension for the same reason the retryability field was named in the previous chapter — a gap identified is worth more than a gap papered over, and the frame and manifest formats were both built so that filling it later is an addition rather than a redesign.

A sunset that actually works

Gathering the mechanics into a procedure, for an author who has to remove something real from a shipped surface.

Search first, because on this bus the caller population is knowable. Add the new id before removing anything, so both work at once and no window exists in which neither does. Keep the old id dispatching to the same `run` — an alias, not a copy, because two bodies drift. Make the silent breaks loud on purpose: a dropped parameter raises rather than being ignored, a dropped return field is removed only after the search comes back empty. Migrate the scripts, then re-run the search. Drop the alias in a release whose catalog diff shows the deletion, so that the gate fires on the one change that can break something and a human confirms it deliberately.

The whole procedure is ordinary API hygiene, and the only part specific to this bus is the first step — that the callers can be found. That single fact is what makes deprecation here a manageable engineering task rather than the open-ended obligation it is for a public interface, and it is worth recognizing as one of the compensations for the single-user, single-machine constraint the design accepted everywhere else.

Chapter 56 — The Bus in the Shell

The `.stk` file is a first-class citizen of the terminal

The out-of-process front-end is a standalone `.stk` file, and calling it “standalone” undersells where it lives: it lives in the terminal, alongside every other executable the shell knows, and it composes with them the way any other program does. A bus script has a shebang — `#!/usr/bin/env stryke` — so it is `chmod-executable` and runnable by name; it reads arguments, it reads `stdin`, it writes `stdout` and `stderr`, and it returns an exit status. None of that is bus machinery; it is `stryke` being an ordinary member of the shell environment, and it matters because it means a bus script is not a special artifact that runs in a special way. It is a program, and the shell composes programs. The bus’s cross-app reach and the shell’s composition are two powers that multiply: a script that drives four apps is also a program a pipeline can feed and a scheduler can run.

Arguments and exit status: the shell’s contract

A bus script that takes arguments and returns a status is a script the shell can drive, and the maintenance and test recipes leaned on exactly this without naming it. A script that `dies` on failure exits non-zero, which a scheduler reads as “this run failed” and a `ff` chain reads as “stop here”; a script that finishes cleanly exits zero, which the same constructs read as success. The API regression runner of Chapter 35 ended in `die` `"${ $fail } assertion(s) failed"` for precisely this reason: its exit status is its verdict, and a CI job runs it as `stk smoke.stk || alert`, needing to know nothing about the bus to act on the result. The bus script speaks the one protocol every program in the terminal speaks — argument in, status out — so it slots into the shell’s world without translation.

Piping data in and out

Because a bus script reads `stdin` and writes `stdout`, it composes with the rest of the terminal by pipe. A script that reads DOIs on `stdin` and files each into the reference manager is fed by anything that produces DOIs — a `grep`, a `cat`, another script — and a script that writes its results to `stdout` feeds anything downstream:

```
#!/usr/bin/env stryke
use App
val $cite = App::open("zcite")
for val $doi (@{ lines(stdin()) }) {           # one DOI per input line
    val $item = try { $cite->call("item.add", %{ doi => trim($doi) }) } catch ($e) { nil }
    if $item { p $item->{id} }                  # emit the new id to stdout
}
```

Run as `cat dois.txt | stk file-dois.stk > new-ids.txt`, this bus script is a filter — DOIs in, ids out — and the reference manager is a stage in a shell pipeline. That is a strange and useful sentence: a GUI reference manager became a `grep`-able stage in a terminal pipe, because the bus made its `item.add` callable and `stryke` made the caller a filter. The composition is the shell’s; the reach into the app is the bus’s; and neither had to know about the other for the two to multiply.

Scheduling and the unattended run

A bus script that returns a status and needs no terminal is a script a scheduler runs, and the reporting and maintenance recipes of Chapter 30 were written for exactly this life. A cron entry or a systemd timer runs `stk maintenance.stk`, which checks `App::list()` for the apps that are up, does what it can, records the rest, and exits non-zero if anything failed. The unattended run leans on every part of the failure model — the list check that skips a down app, the try-per-item that records rather than aborts, the exit status that reports the verdict — and it leans on the at-rest cost of the bus being nil, per Chapter 24, so that apps carrying the bus pay nothing for the hours between scheduled runs. A scheduled bus script is the automation the book has been building toward at its most self-effacing: it drives the whole suite and no one is watching, and it is trusted to because it fails loudly and finishes cleanly.

Bus scripts composing with each other

Because a bus script is an ordinary program with an ordinary contract — arguments in, stream out, status at the end — the composition that matters most is not between a bus script and `grep` but between two bus scripts. This is the point at which the automation stops being a collection of files and becomes a small system, and it works for a reason worth stating: nothing about a bus script makes it a poor citizen of a pipeline, and nothing about the bus makes two scripts driving the same suite interfere by construction.

The useful decomposition is the one the shell has always rewarded. A script that *reads* — producing lines of identifiers on stdout — and a script that *acts* — consuming identifiers on stdin — are more valuable separately than the single script that does both, because each recombines. A reader that emits the DOIs of the current selection feeds the filer, the reporter, and the checker without knowing any of them exist, and each of those can be tested against a fixed input file with no app running at all, which is the script-logic testing of Chapter 21 arriving through the shell's front door rather than through a mock.

The decomposition also gives a natural place for the concurrency discipline of Chapter 40. Two scripts in a pipeline do not run against the app at the same time in any troublesome sense — the reader has finished its reads before the actor begins its writes for a given item — and where they do overlap, the overlap is visible in the pipeline's shape rather than hidden inside one file's control flow. A single script that interleaves reads and writes against one app is doing something harder to reason about than two scripts joined by a pipe, and the pipe is free.

There is one property to be deliberate about. Each script in the pipeline opens its own handles, which means each dials its own connections and holds its own subscriptions, and the apps see them as unrelated clients — which they are. That is the correct model and it costs almost nothing, per the at-rest accounting of Chapter 24, but it does mean a pipeline of five bus scripts is five sets of connections rather than one shared session. Where that matters — where the sequence really is one logical run and the intermediate values are not worth serializing to text — the answer is a single script holding one set of handles, which is what every cookbook recipe in this book is. The rule is the ordinary one: compose by pipe when the stages are independently useful, and compose in one file when they are one operation that happens to have steps.

The bus and `zsh`

The terminal the bus scripts run in is itself part of the stack — the shell is the IDE, the build system, the orchestrator — and the bus is a citizen of it rather than a thing beside it. A bus script is invoked from the shell, completes against the live surface through the completion of Chapter 38, and returns to the shell; and the shell's own history, aliases, and composition apply to it as to any command. The point is not that the bus integrates with the shell — it is that there is no seam to integrate across, because a bus script is a shell program that happens to reach into GUI apps. The suite's discipline that everything is a set of functions to be called reaches its widest form here: the shell calls the script, the script calls the apps, and the whole chain — keystroke to shell to script to socket to verb to engine and back — is one continuous set of calls, each handed

its caller by the layer above. The bus did not build a bridge to the terminal; it noticed the terminal was already the place the caller lives, and put the apps within its reach.

The last sentence

The book began on one sentence and has spent fifty-six chapters earning it, so it ends by saying it plainly one more time, now with every word behind it. One language should be able to drive every app you own, by name, and get an answer back. The language is stryke. The apps are the suite. The driving is a typed verb, called with typed arguments, over a host callback or a newline of JSON. The answer back is the return value, landing in a variable the next call can use. And the reason the pieces that make the sentence true could be small — a manifest, a bridge, a module, a socket, a shell script — is that the architecture had already turned every app into a set of functions waiting to be called. The bus only had to hand them a caller, and this book only had to show that it did.